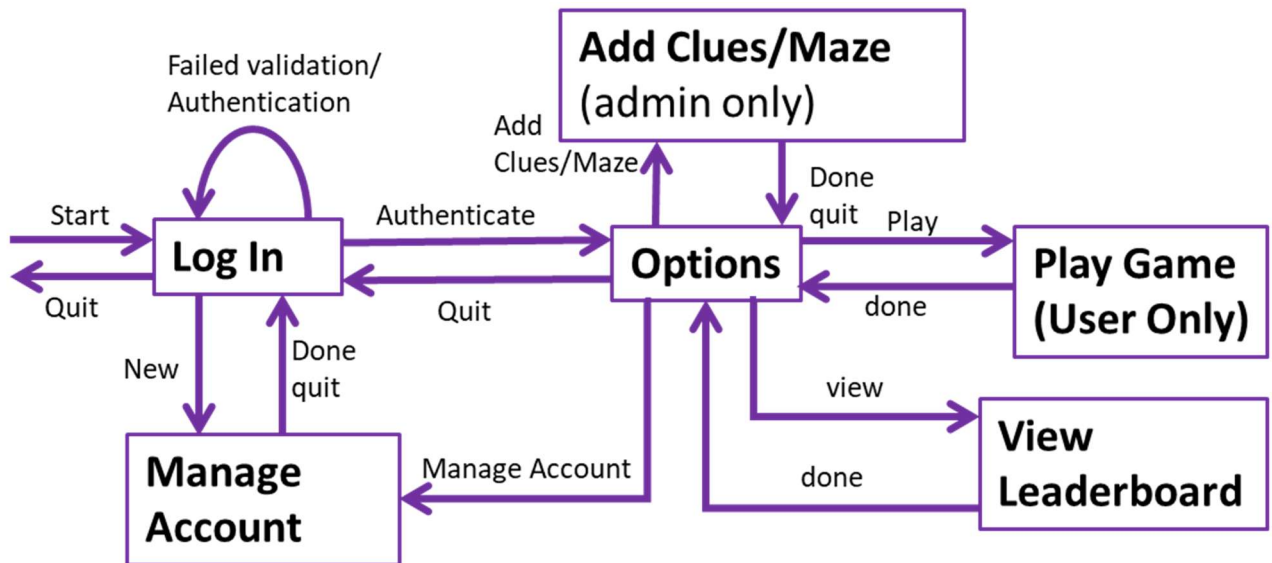


37. 3rd Iteration: Graphical Interface Development



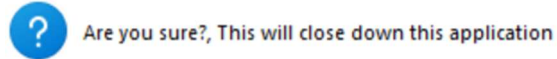
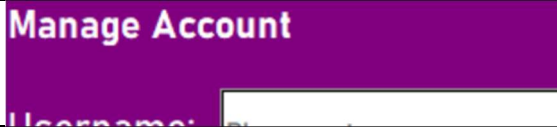
37.1. My game system needs GUIs to enable the stakeholders to interact with it. Previously, I created a State Transition Diagram as shown above. This shows how different GUIs should interact with each other. I will develop each GUI individually, using a similar visual theme throughout.

37.2. **Login GUI**

- a. To Login a stakeholder must enter their username as a string
- b. Valid Characters are 'A' – 'Z', 'a' – 'z', '0' – '9' and selected special characters
- c. Validation:
 - i. Presence check (something entered)
 - ii. Length Check up to 12 characters inclusive
 - iii. Type check – valid char type
 - iv. Maintenance: I would consider sanitisation to prevent SQL injection opportunities in a future prototype.
- d. Authentication: once validated, the username must be authenticated against a username already in the database.
- e. Three login attempts will be allowed.
- f. A Welcome message will be displayed specific to the user if log in is successful
- g. An Error message will be displayed if validation of authentication or number of attempts has failed.
- h. A successful login removes the Login GUI and launches the Options GUI
- i. A new User will have the option of creating an account – the Login GUI will be removed, and the Manage Account GUI will be displayed.
- j. Any User can close the application by quitting the program through an embedded quit button.

37.2.1. Test Plan(White box testing) – development of Log In event handler

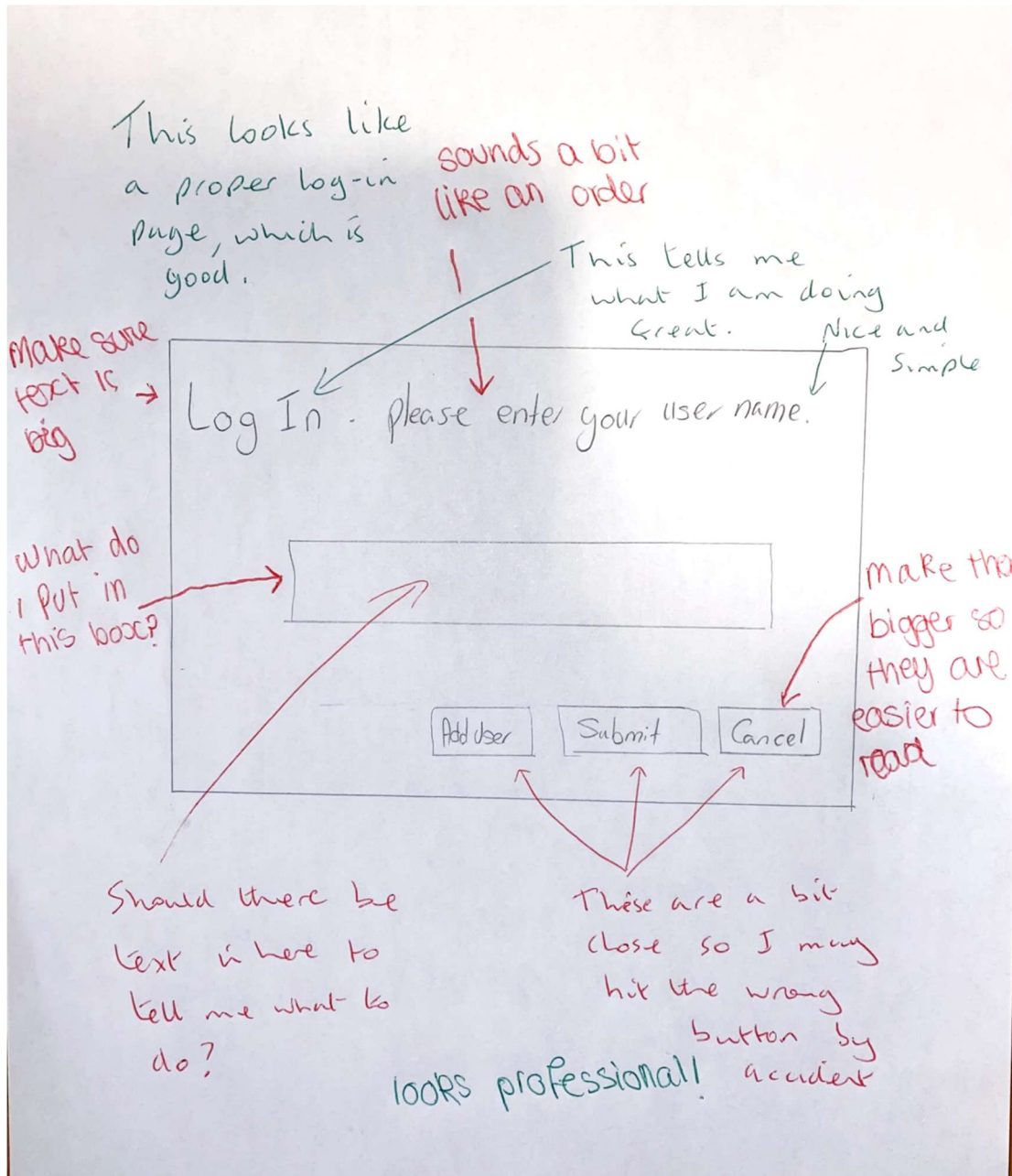
No.	Description	Input	Expected Outcome	Actual Outcome	P/F
1	Normal: enter a valid and authentic username	Username="Test"	Message: "Welcome Test"	 Welcome 'Test'	P
2	Invalid: Enter invalid username				
a.	Invalid: Fails presence check	""(NULL)	Message: "ERROR. Failed validation"	 Error Validation failed, No Username entered	P
b.	Invalid: Failed length check (too long)	Username = "123456789abcde"	Message: "ERROR. Failed validation"	 Error Validation failed, Username: '123456789abcde' is too long	P
c.	Invalid: Failed type check(forbidden chars in a correct length username)	Username = "Hello&"	Message: "ERROR. Failed validation"	 Error Validation failed, Username: 'Hello&' contains forbidden characters	P
3	3 failed attempts to log in closes application	After test 2c above	Message: "ERROR. To many attempts quitting"	 Log In failed after 3 attempts	P
4	After reboot. Invalid: Fails authentication	Username = "Heart"(valid username not in database)	Message: "ERROR. Failed Authentication"	 Error Authentification failed, 'Heart' is not a valid Username	P
5	After 1 fail: Valid authentic log in	Username = "GiraffeLeaf"	Message: "Welcome GiraffeLeaf"	 Welcome 'GiraffeLeaf'	P
6	After restart: Extreme: username is valid	Username = "HelloWorld32" (12 chars)	Message: "Welcome HelloWorld32"	 Welcome 'HelloWorld32'	P
7	Extreme: username is valid	Username = "a" (1 chars)	Message: "Welcome a"	 Welcome 'a'	P

8	Test for button click Cancel	Cancel button clicked	"Are you sure?" Message box displayed	 On yes the GUI shuts down	P
9	Test for button click New User	New User button clicked	Screen moves to New User GUI		P

After Developing my Log In GUI, I decided that it would be beneficial in the case of a failed authentication to give more detail to the users about what error had occurred when signing in. This should improve usability and ease of access.

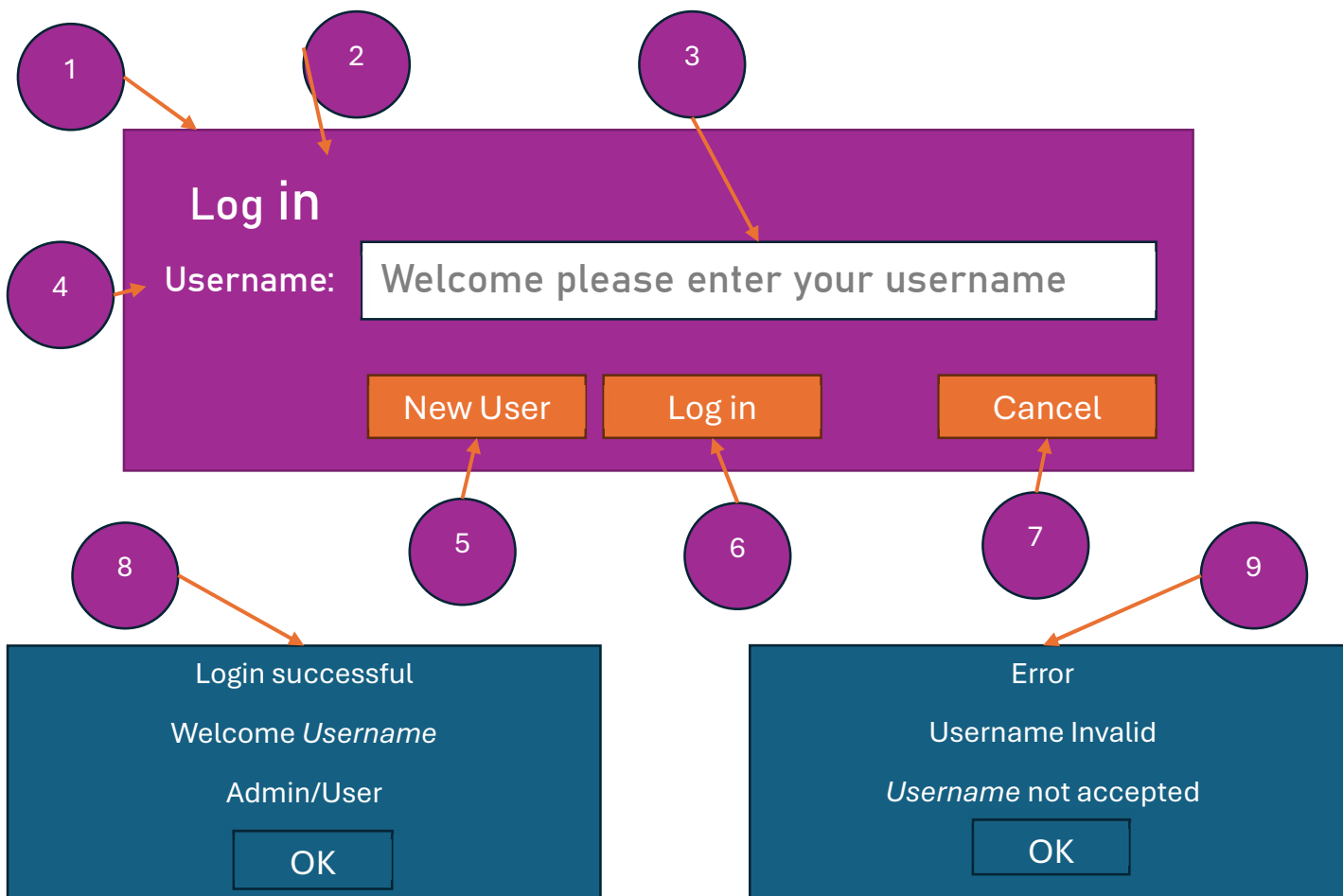
37.2.2.Design and Development

I started with a hand drawing of my GUI design for the stakeholders to comment on. The image below shows my stakeholder comments. They were generally positive, but I asked them to focus on what they would change, so they gave me lots of things to think about.



From this interaction with my stakeholders, I have realised that I need to adjust my initial design to include bigger buttons, a label for the entry box, a more friendly welcome and also potentially I should include temporary text inside the entry box. I am pleased that they liked the initial design for the most part.

Below is my revised design for the Log In GUI as a result of my stakeholder interaction above:



ID	Name	Widget	Justification
1	frmLogIn	Frame(container)	All the other widgets are held within the frame. The widgets are then easy to move within the frame for best layout on the screen
2	lblLogIn	Label	Read only title for this GUI so the user knows where they are at in the system.
3	txtUsername	Entry	User input box for user to enter their username with overwritable instruction text – Entry used because any variety of usernames need to be entered via keyboard
4	lblUsername	Label	Read only identifier for the type of data to input so the user knows what type of data to input.
5	btnNewUser	Button	A Button is an easy option for the user to click for a manage account GUI
6	btnSubmit	Button	A Button is an easy option for the user to click for the validation and Authentication of their username
7	btnCancel	Button	A Button is an easy option for the user to click to quit the game.
8	msgSucessLogIn	messagebox	To welcome user after successful log in Visual display to prove to the user that they are logged in

9	msgErrorLogIn	MessageBox	To report a problem to the user with their username. Visual display to tell the user that there is a problem with log in
---	---------------	------------	---

37.2.3.Log In GUI Development

```
def CreateLogIn():
    #procedure to create the Log in GUI

    global frmLogIn

    BackCol = "purple"      #default bg color
    TextCol = "white"       #default font color

    #create the frame - container

#ref34:https://cs111.wellesley.edu/archive/cs111_fall14/public_html/labs/lab12/tkintercolor.html
#Tkinter named colours
frmLogIn = tk.Frame(root, bg = "green")
frmLogIn.place(x = 5, y = 5, width = 300, height = 100)
#add widgets
lblLogIn = tk.Label(frmLogIn, text = "Log In",
                    bg = BackCol,fg = TextCol,
                    justify = "left" )
lblLogIn.place(x = 5, y = 5, width = 200, height = 15)

lblUsername = tk.Label(frmLogIn, text = "Username:",
                      bg = BackCol,fg = TextCol )
lblUsername.place(x = 5, y = 20, width = 50, height = 25)

txtUsername = tk.Entry(frmLogIn, text = "",
                      bg = TextCol,fg = "black" )
txtUsername.place(x = 70, y = 20, width = 50, height = 25)

btnNewUser = tk.Button(frmLogIn, text = "New User",
                      bg = "DarkOrange1",fg = TextCol )
btnNewUser.place(x = 5, y = 60, width = 60, height = 25)

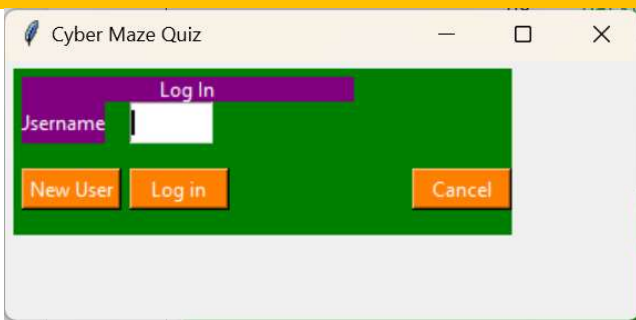
btnSubmit = tk.Button(frmLogIn, text = "Log in",
                      bg = "DarkOrange1",fg = TextCol )
btnSubmit.place(x = 70, y = 60, width = 60, height = 25)

btnCancel = tk.Button(frmLogIn, text = "Cancel",
                      bg = "DarkOrange1",fg = TextCol )
btnCancel.place(x = 240 , y = 60, width = 60, height = 25)

#set event handler

return #end procedure
```

First Development – procedure to contain it and global frmLogIn so the whole frame and its widgets can be easily moved on the screen after it has moved.



This is the output from my first Log In GUI

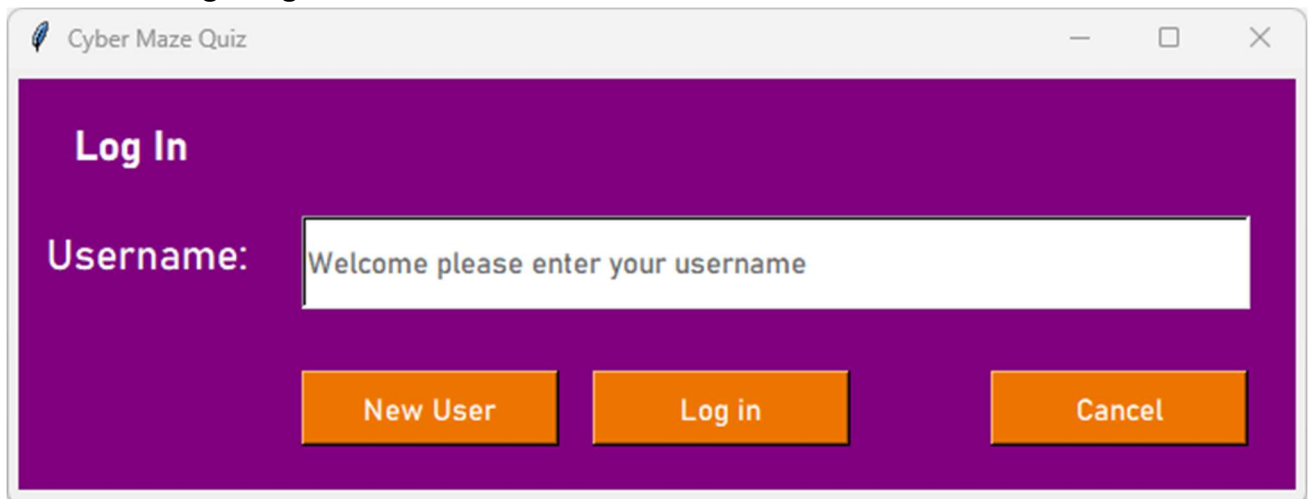
37.2.4. Evaluation

This is my evaluation of this first attempt at creating the Log In GUI to meet the stakeholder requirements. I need to:

- Correct the colours as per the design
- Size the root window to match the frame
- Align the text in the labels to the left
- Increase font size in labels and use bold text for titles
- Make the text box the same size as 12 chars without scrolling
- Make sure the button placement matches the design
- Ensure the text in the labels fits completely in the boxes

37.2.5. Corrections

I made the corrections to match my evaluation findings and this is the second attempt at creating a Log In GUI:



I made the corrections mentioned above as follows and the code is given below:

- Using the pixel measurements from my design PowerPoint, I created a completely accurate GUI according to my design.
- Added temporary text to the input which will empty and change to black text when clicked.

```

def CreateLogIn():
    #procedure to create the Log in GUI

    global frmLogIn, txtUsername

    BackCol = "purple"      #default bg color
    TextCol = "white"       #default font color
    BtnCol = "DarkOrange2"  #Button color
    #create the frame - container

    #ref34:https://cs111.wellesley.edu/archive/cs111_fall14/public_html/Labs/Lab12/tkintercolor.html
    #Tkinter named colours
    frmLogIn = tk.Frame(root, bg = BackCol)
    frmLogIn.place(x = 5, y = 5, width = 645, height = 207)

    #add widgets
    #ref35:https://www.geeksforgeeks.org/what-are-widgets-in-tkinter/
    #types of widget
    lblLogIn = tk.Label(frmLogIn, text = "Log In",
                        bg = BackCol,fg = TextCol,
                        anchor="w", font=('Bahnschrift', 16, 'bold'))
    lblLogIn.place(x = 25, y = 15, width = 201, height = 36)

    lblUsername = tk.Label(frmLogIn, text = "Username:",
                        bg = BackCol,fg = TextCol, anchor="w",
font=('Bahnschrift', 16))
    lblUsername.place(x = 11, y = 70, width = 124, height = 36)

    txtUsername = tk.Entry(frmLogIn, text = "",
                        bg = TextCol,fg = "dim gray", borderwidth=2,
font=('Bahnschrift', 11))
    txtUsername.place(x = 143, y = 68, width = 478, height = 46)

    txtUsername.insert(0, "Welcome please enter your username")

    btnNewUser = tk.Button(frmLogIn, text = "New User",
                        bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )
    btnNewUser.place(x = 143, y = 147, width = 130, height = 38)

    btnSubmit = tk.Button(frmLogIn, text = "Log in",
                        bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )
    btnSubmit.place(x = 290, y = 147, width = 130, height = 38)

    btnCancel = tk.Button(frmLogIn, text = "Cancel",
                        bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )
    btnCancel.place(x = 491 , y = 147, width = 130, height = 38)

    #set event handler

    #temporary text in Entry box
    #ref38:https://www.tutorialspoint.com/how-to-insert-a-temporary-text-in-a-tkinter-entry-widget
    def temp_text(e):
        #ref to change the colour on click

```

The GUI correctly laid out and coloured; added txtUsername as a global so it can be managed for external subroutines

```

txtUsername["fg"] = "black"
txtUsername.delete(0,END)
return e
txtUsername.bind("<FocusIn>", temp_text)

return #end procedure

```

37.2.6.Event Handlers

An event is when something occurs that is expected but it is not known when it will happen, for example the user clicking a button. There are 3 buttons to consider in this frame:

btncancel

```

When clicked,
    Display an 'Are you sure?' message
    If 'Yes', then close the application
    If 'No', return to Login GUI

```

btnNewUser

```

When clicked ,
    hide LogIn GUI and display ManageAccount GUI

```

btnSubmit

```

When clicked
    take the string from txtUsername
    if it is valid then
        authenticate it
        if it passes authentication then
            use messagebox to greet user
            hide LogIn GUI
            display Options GUI
        else //failed authentication
            use message box to display error message
            return to LogIn GUI
        endif
    else //not valid
        use messagebox to display error message
        return to LogIn GUI
    endif

```

Note: But the btnSubmit algorithm does not limit the number of log in attempts to 3.

37.2.7.btnSubmit

The algorithm below is a more detailed breakdown of this button event handler:

```
MaxAttempts = 3           //constant
txtUName = take the string from txtUsername
NumAttempts = 0           //at the start
AuthLogIn = False        //loop control variable

//condition controlled iteration
While not AuthLogIn and NumAttempts < MaxAttempts then
  If IsValid(txtUName) and IsAuthenticated(txtUName) then
    AuthLogIn = True
  else //failed validation or authentication
    use message box to display error message //error failed
  Validation/Authentication
    NumAttempts = NumAttempts + 1
    return to LogIn GUI
endif
endwhile
//why has loop quit?
If AuthLogIn then
  use messagebox to greet user //”Welcome <username>”
  hide LogIn GUI
  display Options GUI
else //maxed out number of login attempts
  use messagebox to display quit message //to many attempts quitting
  close application
endif
```

37.2.8. Development of Login Event Handlers

From the algorithm design, I need to develop two subprograms:

Function IsValid(aStr) - to perform the validation checks and return a Boolean (True for passes validation, False otherwise:)

Function IsAuthenticated(aStr) - to query the database for a matching string, returning a Boolean (True for match, False otherwise)

The algorithm below is for the IsValid function:

```
Function IsValid(aStr)
//Function to perform validation on the aStr parameter according to the validation rules
    GoodChars = "abcdefghijklmnopqrstuvwxyz0123456789_!@"//string = array of char
    If aStr != "" then        //passes presence check
        If aStr.length >=1 and aStr.length <= 12 then        //passes length check
            //linear conditional search type check
            IsGood = True
            CharNum = 0
            While IsGood and CharNum < aStr.Length
                aChar = aStr [CharNum]        //get a char
                if aChar is GoodChars then //aChar is valid
                    CharNum = CharNum + 1    //for next char to test
                Else //failed – no need to look anymore
                    IsGood = False          //quit loop
            Endif
        Endwhile
        //why did loop quit
        If IsGood then
            Return False
            Return True
        Endfunction
```

37.2.8.1. Test Plan(White box testing) - Validation

No.	Description	Input	Expected Outcome	Actual Outcome	P/F
1	An input within the length with Capital letters, lowercase letters and numbers.	IsValid("Agjkrfo23569")	True	<pre>>>> IsValid("Agjkrfo23569") True</pre>	P
2	An input NULL	IsValid("")	False	<pre>>>> IsValid("") False</pre>	P
3	An input with characters that are not allowed	IsValid("%")	False	<pre>>>> IsValid("%") False</pre>	P
4	An input with valid characters just within the lower bound of the range	IsValid("a")	True	<pre>>>> IsValid("a") True</pre>	P
5	An input with valid characters just within the upper bound of the range	IsValid("123456789ABC")	True	<pre>>>> IsValid("123456789ABC") True</pre>	P
6	An input with valid characters outside of the range	IsValid("AEGijheojg114198095")	False	<pre>>>> IsValid("AEGijheojg114198095") False</pre>	P

This is a quick development test to show that the IsValid function is likely to work.

```
#callbacks01.py
#library of subprograms to support the GUIs
#Rosa Madden 02/11/24

def IsValid(aStr):
    #function to perform avalidation checks on aStr
    #Return True if all passed false otherwise
    ValidChars = "abcdefghijklmnopqrstuvwxyz0123456789_"
    #string = array of char

    if aStr != "": #passes validation check

        if len(aStr) >=1 and len(aStr) <= 12: #passes Length check

            #conditional linear search to check char by char
            IsGood = True #boolean loop control valid
            CharNum = 0 #index into a Str

            while IsGood and CharNum < len(aStr):
                aChar = aStr[CharNum] #get a char to check

                if aChar.lower() in ValidChars: #good char
                    CharNum = CharNum + 1 #get next char
                else:
                    IsGood = False #quit checking
            #why did loop quit?
            if IsGood:
                return True #aStr passes validation checks
            else:
                return False #aStr failed type check

        else: #fails length check
            return False

    else: #fails presence check
        return False
```

Quick
development
test shows that it
is likely to work.

```
>>> IsValid("Agjkrfo23569")
True
>>> IsValid("")
False
>>> IsValid("%")
False
>>> IsValid("a")
True
>>> IsValid("AEGijheojgll14198095")
False
>>> IsValid("123456789ABC")
True
```

The algorithm below is for the IsAuthenticated function:

```
Function IsAuthenticated(aStr)
//Function to search for a match to validated aStr in the database
    //create query
    Qry = "SELECT UserName FROM User WHERE UserName = ", aStr
    AnsTable = RunQuery(DBFile, Qry)
    If AnsTable then //match found – usernames are unique
        Return True
    Return False
Endfunction
```

37.2.8.2. Test Plan(White box testing) - Authentication

No.	Description	Input	Expected Outcome	Actual Outcome	P/F
1	The Username input is not in the database.	IsAuth("Frog")	False	>>> IsAuth("Frog") False	P
2	The Username is already in the database.	IsAuth("GiraffeLeaf")	True	>>> IsAuth("GiraffeLeaf") True	P

This is a quick development test to show that the IsAuth function is likely to work.

```
def IsAuth(aStr):
    #function to query the User table in the database to a match to a valid aStr
    #create query
    Qry = '''SELECT UserName FROM User
           WHERE UserName = "%s"''' %aStr
    #run it - dbfile set global in LogOnDatabase
    AnsTable = RunQuery(DBFile, Qry)

    #test result - Usernames are unique
    if AnsTable:    #found match
        return True
    else:
        mbox.showerror("Authentication Failed", "Error Authentication failed, '%s'
is not a valid Username"%aStr)
        return False
```

I simplified IsAuthenticated() to IsAuth() for convenience

Quick development test shows that it is likely to work.

```
>>> IsAuth("Frog")
False
>>> IsAuth("GiraffeLeaf")
True
```


37.2.9.Log in event handler

The code below is assigning subprograms to run when a button is clicked

I used the learning in this reference to understand how to handle a button click event and apply it to my code: <https://www.tutorialspoint.com/how-to-handle-a-button-click-event-in-tkinter>⁴⁰

```
btnSubmit = tk.Button(frmLogIn, text = "Log in",  
                      bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )  
btnSubmit.place(x = 290, y = 147, width = 130, height = 38)
```

```
#set event handler
```

```
btnSubmit['command'] = Submit
```

Assign the click event handler to the Log In button

-note that the handler has **no brackets**

```
def Submit():  
    #event handler for btnSubmit  
    aStr = txtUsername.get()    #get entered username  
    GetUser(aStr)  
    return #end procedure
```

Declare and code the event handler procedure

-note it has empty brackets

Call the function in the imported Callbacks

```
def GetUser(aStr):  
    #function to validate and authenticate a username  
    print(aStr + ":", end = '')  
    if IsValid(aStr) and IsAuth(aStr):  
        print("True")  
        return True  
    else:  
        print("False")  
        return False
```

In Callbacks add some print() statements to support testing *

```
>>> gnfdor:False  
GiraffeLeaf:True
```

Run Interfaces and LogIn with some sample usernames and see the print()s in the shell

37.2.10. Attempt Management

The code below is to manage the number of log in attempts by the user

```
def GetUser(aStr):
    #function to validate and authenticate a username
    print(aStr + ":", end = '')
    C_MaxAttempts = 3 #const - max number of attempts allowed
    NumAttempts = 0 #count the login attempts
    AuthLogIn = False

    #condition-controlled iteration
    while not AuthLogIn and NumAttempts < C_MaxAttempts:
        if IsValid(aStr) and IsAuth(aStr):
            print("True")
            AuthLogIn=True #quit loop
        else: #failed validation or authentication
            print("False")
            NumAttempts = NumAttempts + 1 #increment count
            print("%i Login Attempt(s)" %NumAttempts)

    #why did loop quit?
    if AuthLogIn: #success
        #get users details
        aQry = '''SELECT * FROM User
                WHERE UserName = "%s"''' %aStr
        AnsTable = RunQuery(DBFile,aQry) #is auth proved they are there
        print(AnsTable)
        print("Welcome '%s'" %AnsTable[0][1])
        return True

    else: #failed log in after max goes
        print("Log In failed after %i goes" %NumAttempts)
        return False #end function
```

To greet the user and know their avatar I need to know their details

[(ID, 'UserName', Type, Avatar)]

```
>>> shsrtj:False
1 Login Attempt(s)
False
2 Login Attempt(s)
False
3 Login Attempt(s)
Log In failed after 3 goes
.
```

Error: All attempts are being used at once with a single value of Str.

The code above gave me an Error where all attempts were used at once for a single value of aStr. To attempt to fix this, I tried to put a for loop into the while loop which created an endless loop. This wasn't going to work either. This attempted fix is shown below:

```
while not AuthLogIn:
    for NumAttempts in range(0, (C_MaxAttempts - 1)):
        NumAttempts = C_MaxAttempts #end for loop
```

Looking at the algorithm again, I created another algorithm for how NumAttempts might work better:

Set NumAttempts to 0 FIRST TIME
If failed login
 NumAttempts = NumAttempts+1
 Have another login attempt – NumAttempts must not reset!
NumAttempts must be retained

The fixes I need to create are outlined below:

To make this work I have:

1. globally declared NumAttempts setting it when the program first runs
2. Remove the while statement to stop the same input from being processed multiple times
3. Change the way an authorised login is declared to ensure it only displays that final message after the last attempt.

1

```
global NumAttempts
NumAttempts = 0 #set start values for login attempts
global NumAttempts
AuthLogin = False
```

2

```
AuthLogin = False
if IsValid(aStr) and IsAuth(aStr):
```

3

```
elif NumAttempts == C_MaxAttempts: #failed log in after max goes
```

Changes to make:

1. Reset the number of log in attempts where the temporary text is reset (allows for different users to Login)
2. Get the program to quit after 3 failed login attempts
3. Remove the username after each attempt. (Leaving it blank and ready for new attempts)

```
>>> wrserjg:False
1 Login Attempt(s)
wrserjg:False
2 Login Attempt(s)
wrserjg:False
3 Login Attempt(s)
Log In failed after 3 goes
fkutgu:False
4 Login Attempt(s)
sdfhsd:False
5 Login Attempt(s)
```

The program currently lets you enter more than one attempt.

First part of the fix for limiting the number of attempts:

1. Reset the number of log in attempts where the temporary text is reset (allows for different users to Login)

```
if GetUser(aStr):  
    #ref for destroying widg41  
    #https://www.geeksforgeek41  
    txtUsername.destroy()  
    Username = aStr
```

This is also where the User Attempts must be reset. To do this I have passed the variable attempts to a function in callbacks to ensure that it is reset in the GetUser function

To reset the temporary text I first found out how to remove widgets from a frame by using `txtUsername.destroy()` inside the Submit procedure.

This enabled me to add the `txtUsername` widget back in when I went back to this window and connect up the temporary text again.

```
#options  
def Login():  
    global txtUsername  
    Attempts = 0  
    resetAttempts(Attempts)  
    txtUsername = tk.Entry(frmLogIn, text = "",  
                           bg = TextCol, fg = "dim gray", borderwidth=2,  
                           txtUsername.place(x = 143, y = 68, width = 478, height = 46)  
  
    def temp_text(e):  
        #ref to change the colour on click  
        #ref: https://stackoverflow.com/questions/61199837/change-butto39  
        txtUsername["fg"] = "black"  
        txtUsername.delete(0,END)  
        return e  
    tempreset()  
    txtUsername.bind("<FocusIn>", temp_text)  
  
    return  
  
def tempreset():  
    global txtUsername  
    aStr = txtUsername.get()  
    if aStr != "Welcome please enter your username":  
        txtUsername.insert(0, "Welcome please enter your username")  
    return
```

Used to stop the temporary text from being added multiple times

Second and third parts of the fix for limiting the number of attempts:

2. Get the program to quit after 3 failed login

```
elif NumAttempts == C_MaxAttempts: #failed log in after max goes
    print("Log In failed after %i goes" %NumAttempts)
    Cancel()

def Cancel():
    quit()
    return
```

Calls the cancel procedure to cancel if NumAttempts = C_MaxAttempts. which means that after 3 attempts the program shuts itself down.

3. Remove the username after each go. (Leaving it blank and ready for new attempts)

```
def Submit():
    #event handler for btnSubmit
    aStr = txtUsername.get() #get entered username
    txtUsername.delete(0,END) #remove a Username after input.
    if aStr != "Welcome please enter your username":
        GetUser(aStr)
    else:
        mbox.showerror("No Username Entered", "Error, This Username c
    print(NumAttempts)
    return #end procedure
```

Removes characters in the txtUsername input box from 0 to end.

I also added a filter to make sure that the temporary text could not be entered as an input

37.2.11. Message Boxes

So far, all my messages have been output to the IDLE Shell console so I need to incorporate tkinter messageboxes to display them on the screen. I am using the reference below to work out which type of message boxes exist in order to decide which ones I need to use for different situations: <https://www.geeksforgeeks.org/python-tkinter-messagebox-widget/>⁴²

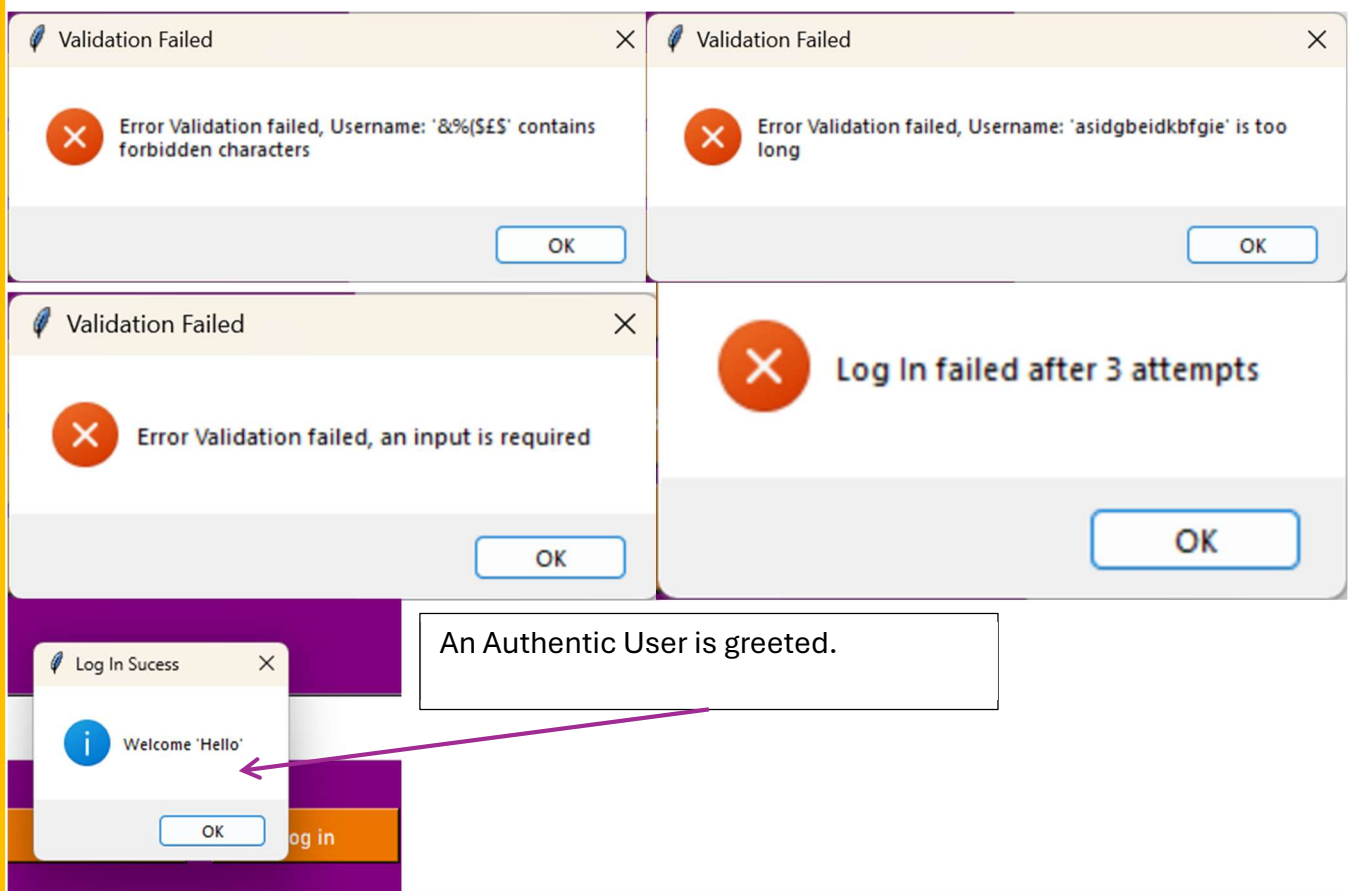
```
mbox.showerror("Authentification Failed", "Error Authentification failed, '%s' is not a valid Username"%aStr)
```

```
mbox.showerror("Validation Failed", "Error Validation failed, Username: '%s' contains forbidden characters"%aStr)
```

```
mbox.showerror("Validation Failed", "Error Validation failed, Username: '%s' is too long"%aStr)
```

```
mbox.showerror("Validation Failed", "Error Validation failed, an input is required")
```

```
mbox.showerror("FAILED", "Log In failed after %i goes" %NumAttempts)
```



37.2.12. Evaluation

The Log In event handling appears to be working although at this point, I have not tested it formally. 3 log in attempts are allowed, validation and authentication works and the correct messages are displayed.

37.2.13. btnCancel

An event is required to close down the window when the cancel button is clicked. To do this, I have created a Cancel procedure which I have used previously to quit after 3 login attempts. I have linked this to the Cancel button which enables it to work.

```
def Exit():  
    Cancel()  
    return #end procedure  
btnCancel['command'] = Exit
```

The Cancel button is connected to the Exit procedure which calls the Cancel procedure in the callbacks.

37.2.14. btnCancel Evaluation

I am yet to add the “Are you sure” message. This is important to make sure a user who didn’t mean to cancel the program doesn’t accidentally do so.

```
def Cancel():  
    action = mbox.askyesno("Cancel", "Are you sure?, This will close down  
    if action:  
        quit()  
    return
```

This will only quit when action is yes due to the if statement.

37.2.15. New User

Even though I currently only have 1 GUI, I know that a new user will have a text entry box which may need to be reset in this connection with the GUI where you add users.

***This was added in later** - after I had developed my Manage Account GUI but I have put it here because it fits in more sequentially

This is the link between the Manage Account GUI and the Log in GUI

```
def MoveMAccount():
    global frmLogIn, frmManage, txtNUser, txtUsername,
    root, pressed

    txtNUser = tk.Entry(frmManage, text = "",
                        bg = TextCol, fg = "dim gray",
    borderwidth=2, font=('Bahnschrift', 11))
    txtNUser.place(x = 143, y = 68, width = 478, height =
46)

    #temporary text in Entry box
    def Ntemp_text(e):
        txtNUser["fg"] = "black"
        txtNUser.delete(0,END)
        return e
    tempNreset()
    txtNUser.bind("<FocusIn>", Ntemp_text)

    frmLogIn.place(x = 5, y = 500, width = 645, height =
207)
    frmManage.place(x = 0, y = 0, width = 645, height =
207)
    width, height = 645, 207
    root.geometry("{}x{}".format(width,height))
    txtUsername.destroy()
    pressed = 1
    return
```

Place in txtNUser to ensure that the temporary text can be loaded in.

To replace the temporary text with no text

Change the frame location.

```
def MoveMAccount():
    global frmLogIn, frmManage, txtNUser, txtUsername, root, pressed

    #place textbox in (may have been previously deleted for temp text
    txtNUser = tk.Entry(frmManage, text = "",
                        bg = TextCol, fg = "dim gray", borderwidth=2, font=('B
    txtNUser.place(x = 143, y = 68, width = 478, height = 46)

    #temporary text in Entry box
    def Ntemp_text(e):
        txtNUser["fg"] = "black"
        txtNUser.delete(0,END)
        return e
    tempNreset()
    txtNUser.bind("<FocusIn>", Ntemp_text)

    #shuffle the frame placement
    frmLogIn.place(x = 5, y = 500, width = 645, height = 207)
    frmManage.place(x = 0, y = 0, width = 645, height = 207)
    width, height = 645, 207
    root.geometry("{}x{}".format(width,height))#make sure the window is the corr
    txtUsername.destroy()
    pressed = 1    #value set for the admin selection default
```

Deletes the necessary text boxes to restore them at a later date (for the temporary text and log in to work multiple times)

37.3. **Options**

Once a Stakeholder has successfully logged in, the options they are given will depend on their account status:

- If they are a standard user, then they can opt to play a game or view a leaderboard
- If they are in the Admin group, then they can opt to add a Maze and/or a Clueset.
- An admin can also view performance stats e.g. Leaderboards
- A user can elect to manage their account e.g. change avatar
- A user can also opt to quit

Manage Account

Whether a user is creating a new account or an existing user wants to change their username and/or avatar, the following options apply:

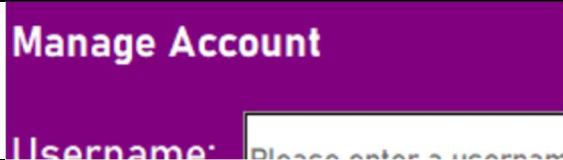
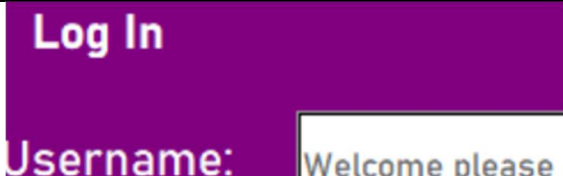
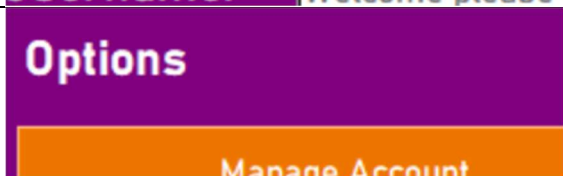
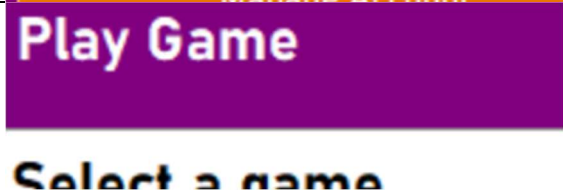
For a new user

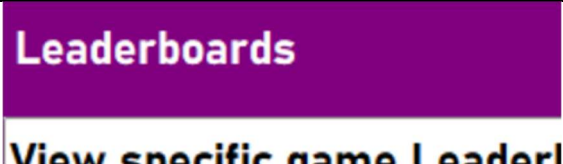
- a. They must enter a username subject to validation:
 - i. Presence check
 - ii. Length check (1 to 12 chars inclusive)
- b. Subject to authentication – the new username must not already be in the database
- c. If the username is unique then they are allowed to select an avatar from a library of available avatars.
- d. If there is a problem with validation and/or authentication, then an error message will be displayed
- e. When they create the account, the new username and avatar are added to the database and a welcome message is displayed before the user is returned to the Log In GUI automatically for them to Login.

For an existing user:

- a. Their current username and avatar selection will be displayed
- b. They can change their username subject to validation and authentication rules, with any relevant error messages displayed
- c. They can change their avatar
- d. When they confirm the changes a “Success” message will be displayed before returning them to the Login GUI (whether they have changed anything or not)
- e. An existing stakeholder should not be able to change their permissions (Admin accounts always remain as Admins, User accounts always remain as Users)

37.3.1. Test Plan(White box testing) – development of Options event handler

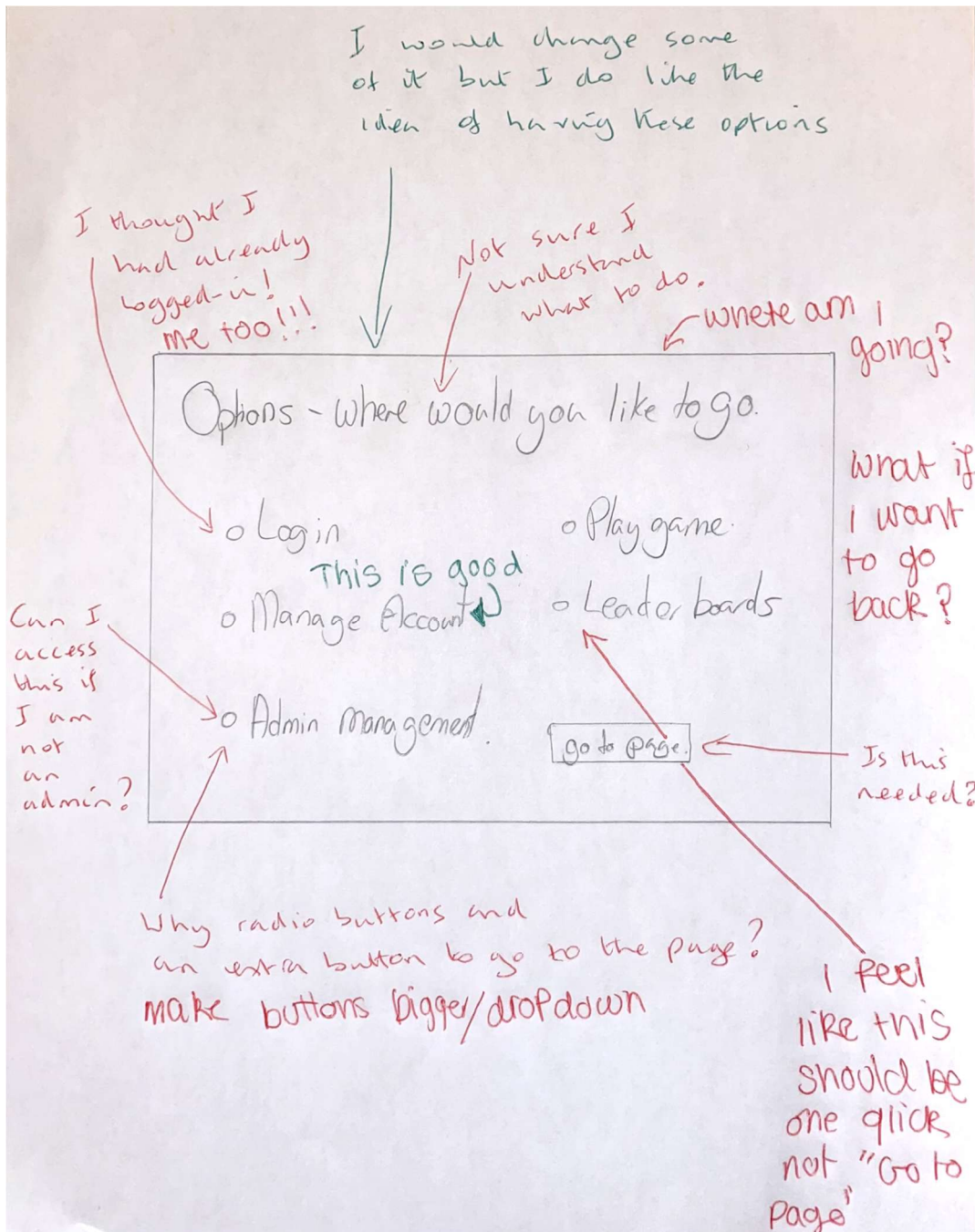
No.	Description	Input	Expected Outcome	Actual Outcome	P/F
1	Test for button click Manage Account	Manage Account button clicked	“Are you sure?” message box	 If you do this you will have to log in again, are you sure?	P
b	Test for click on “Are you sure?” message box	Click “Yes” on the message box	Screen moves to Manage account GUI		P
c	Test for click on “Are you sure?” message box	Click “No” on the message box	Screen remains on Options GUI		P
2	Test for button click back	Back button clicked	“Are you sure?” message box	 If you do this you will have to log in again, are you sure?	P
b	Test for click on “Are you sure?” message box	Click “Yes” on the message box	Screen moves to Log In GUI		P
c	Test for click on “Are you sure?” message box	Click “No” on the message box	Screen remains on Options GUI		P
3	Test for button click Play Game	Play Game button clicked	Screen moves to Play Game GUI		P
4	Test for button click Admin Management	Admin Management button clicked whilst Username = Test	“Incorrect permissions, Access denied”	 User 'Test' is not permitted here	P

				I have made the message more targeted and personal	
		Admin Management button clicked whilst Username = A_Test	Screen moves to Admin Management GUI		P
5	Leaderboard	Leaderboard button clicked	Screen moves to Leaderboard GUI		P

After Developing my Options GUI, I decided to give more detail to the users of what error had occurred when signing in for usability and ease of access.

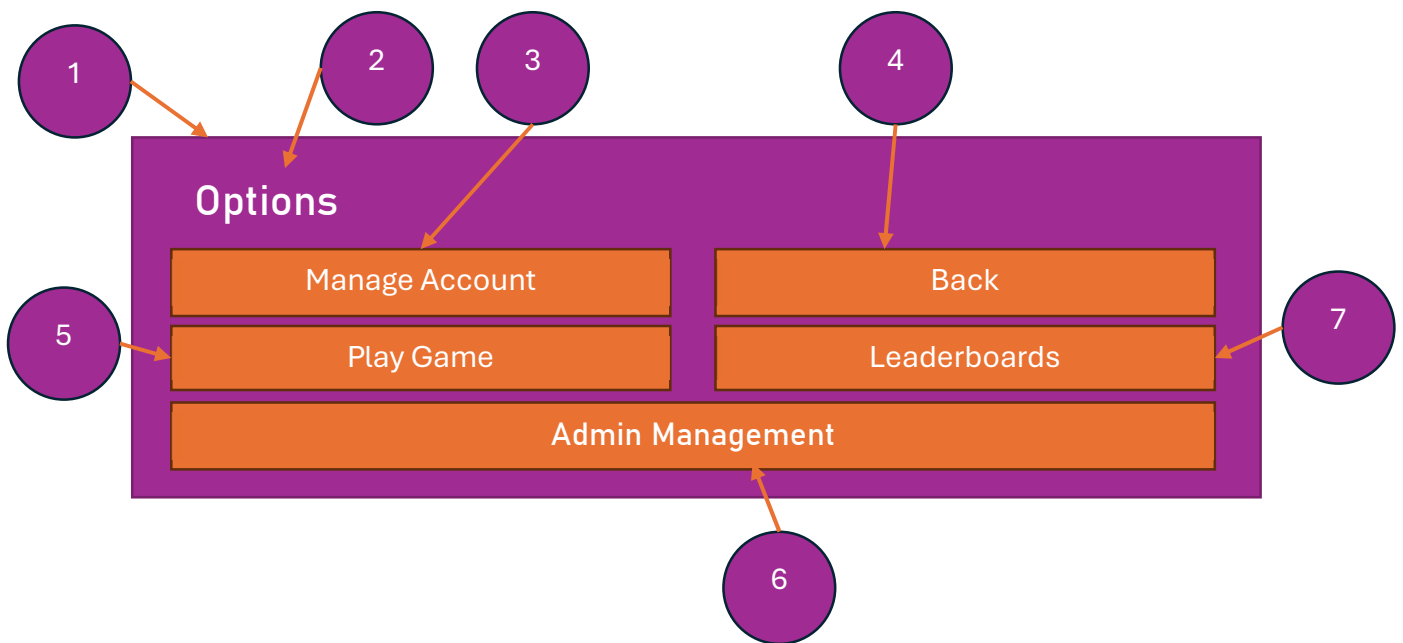
37.3.2.Design and Development

I started with a hand drawing of my GUI design for the stakeholders to comment on. The image below shows my stakeholder comments and they were generally positive but I had asked them to focus on what they would change, so they gave me lots of things to think about.



From my interaction with my stakeholders, I have realised that I need to adjust my initial design to include more buttons, greying out any unusable buttons (e.g. Admin management greyed out for standard users), one click for navigation (rather than two clicks) and a Back button rather than Log in button. I am pleased that they liked the initial design for the most part and liked the button options.

Below is my revised design for the Options GUI as a result of my stakeholder interaction above:



ID	Name	Widget	Justification
1	frmOptions	Frame(container)	All the other widgets are held within the frame. The widgets are then easy to move within the frame for best layout on the screen.
2	lblOptions	Label	Read only title for this GUI so the user knows where they are at in the system.
3	btnManageAccount	Button	A Button is an easy option for the user to click for a manage account GUI
4	btnLogIn	Button	A Button is an easy option for the user to click to go back to the log in GUI.
5	btnPlayGame	Button	A Button is an easy option for the user to Play a game to go to a select game GUI.
6	btnAdmin	Button	A Button is an easy option for the Admin to click to edit the games and be able to shade it out for Users. This Button once clicked will send the Admin user to the Admin Management GUI.
7	btnLeaderboards	Button	A Button is an easy option for the user and Admin to view game leaderboards by being sent to separate Leaderboards GUIs.

37.3.3.Options GUI Development

```
def Options():

    global frmOptions

    BackCol = "purple"      #default bg color
    TextCol = "white"        #default font color
    BtnCol = "DarkOrange2"  #Button color

    #create the frame - container
    #same size as Log on frame
    frmOptions = tk.Frame(root, bg = "green")
    frmOptions.place(x = 5, y = 216.7401575, width = 644.78740157, height =
206.74015748)

    #same position as the Log In Title
    lblOptions = tk.Label(frmOptions, text = "Options",
                           bg = BackCol,fg = TextCol,
                           anchor="w", font=('Bahnschrift', 16, 'bold'))
    lblOptions.place(x = 25, y = 15, width = 201, height = 36)

    btnManageAccount = tk.Button(frmOptions, text = "Manage Account",
                                  bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )
    btnManageAccount.place(x = 25, y = 55, width = 250, height = 38)

    btnLogIn = tk.Button(frmOptions, text = "Back",
                          bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )
    btnLogIn.place(x = 311, y = 55, width = 250, height = 38)

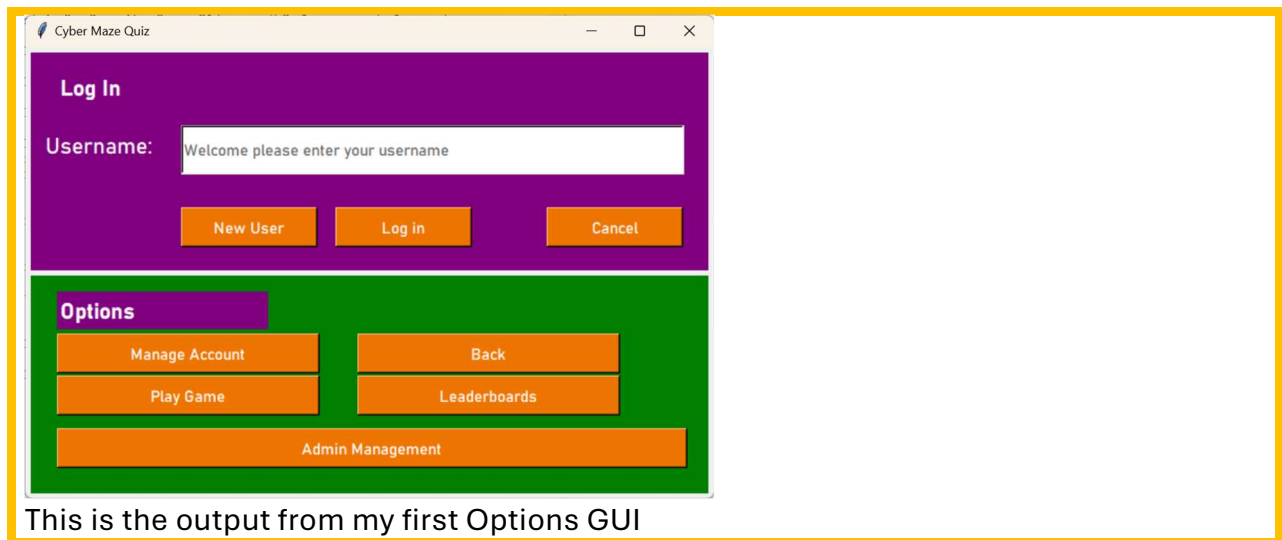
    btnPlayGame = tk.Button(frmOptions, text = "Play Game",
                              bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )
    btnPlayGame.place(x = 25, y = 95, width = 250, height = 38)

    btnAdmin = tk.Button(frmOptions, text = "Admin Management",
                          bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )
    btnAdmin.place(x = 25, y = 145, width = 600, height = 38)

    btnLeaderboards = tk.Button(frmOptions, text = "Leaderboards",
                                  bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )
    btnLeaderboards.place(x = 311, y = 95, width = 250, height = 38)

    #set event handler

    return #end procedure
```



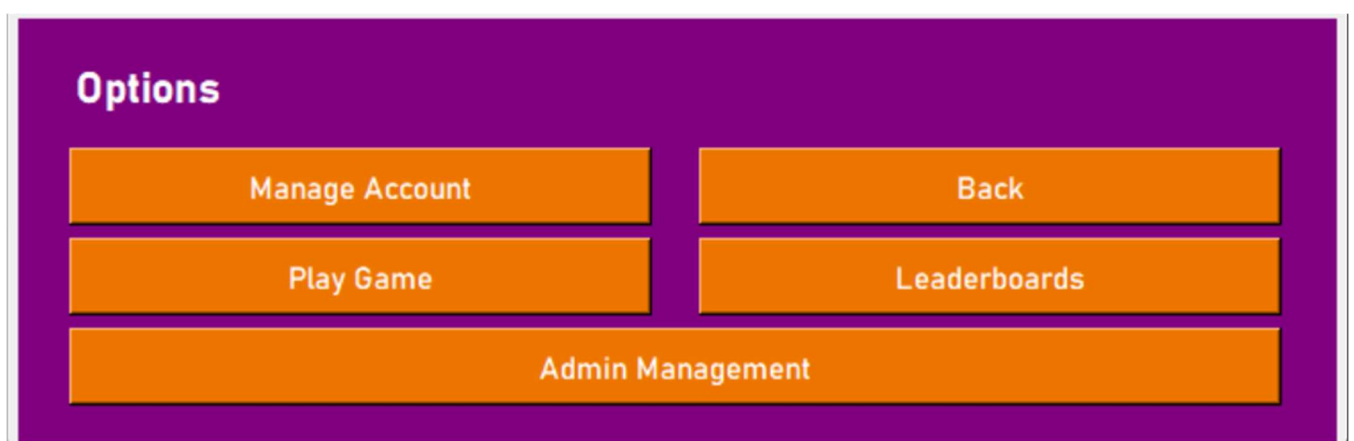
37.3.4. **Evaluation**

This is my evaluation of this first attempt at creating the Options GUI to meet the stakeholder requirements. I need to:

- Correct the colours as per the design (green used for testing)
- Size the root window to match the frame instead of having a tiled window of previous frames
- Place the buttons according to the design

37.3.5. **Corrections**

I made the corrections to match my evaluation findings and this is the second attempt at creating an Options GUI:



I made the corrections mentioned above and the code is given below:

- Using the pixel measurements from my design PowerPoint, I created a completely accurate GUI according to my design.

```
def Options():  
  
    global frmOptions  
  
    BackCol = "purple"      #default bg color  
    TextCol = "white"       #default font color  
    BtnCol = "DarkOrange2" #Button color  
  
    #create the frame - container  
    #same size as Log on frame  
    frmOptions = tk.Frame(root, bg = BackCol)  
    frmOptions.place(x = 5, y = 217, width = 645, height = 207)  
  
    #same position as the Log In Title  
    lblOptions = tk.Label(frmOptions, text = "Options",  
                           bg = BackCol, fg = TextCol,  
                           anchor="w", font=('Bahnschrift', 16, 'bold'))  
    lblOptions.place(x = 25, y = 15, width = 201, height = 36)  
  
    btnManageAccount = tk.Button(frmOptions, text = "Manage Account",  
                                  bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )  
    btnManageAccount.place(x = 25, y = 63, width = 285, height = 38)  
  
    btnLogIn = tk.Button(frmOptions, text = "Back",  
                          bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )  
    btnLogIn.place(x = 333, y = 63, width = 285, height = 38)  
  
    btnPlayGame = tk.Button(frmOptions, text = "Play Game",  
                             bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )  
    btnPlayGame.place(x = 25, y = 107, width = 285, height = 38)  
  
    btnAdmin = tk.Button(frmOptions, text = "Admin Management",  
                          bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )  
    btnAdmin.place(x = 25, y = 151, width = 593, height = 38)  
  
    btnLeaderboards = tk.Button(frmOptions, text = "Leaderboards",  
                                 bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )  
    btnLeaderboards.place(x = 333, y = 107, width = 285, height = 38)  
  
    #set event handler  
  
    return #end procedure
```

37.3.6.Event Handlers

There are 5 buttons to consider in this frame and I have decided to add more message boxes to make the GUI more usable.

btnManageAccount

When clicked,
Display an 'If you do this you will have to log in again, are you sure?' message
If 'Yes', then go to the Manage Account GUI
If 'No', return to Options GUI

btnLogIn

When clicked,
Display an 'If you do this you will have to log in again, are you sure?' message
If 'Yes', then go to the Log In GUI
If 'No', return to Options GUI

btnPlayGame

When clicked,
go to the Play Game GUI

btnAdmin

When clicked,
Verify if Account is Admin
If 'Yes', then go to the Admin GUI
If 'No', stay in Options GUI
Display a 'This User does not have the permissions to access these settings' error message

btnLeaderboards

When clicked,
Go to Leaderboard selection GUI

37.3.7. Development of Options event handlers

Although I have not created most of the GUIs yet, I can now connect this Options GUI to the LogIn GUI and work out the general framework of the connection with the other GUIs that don't exist yet.

37.3.8. btnLogIn

Some of the framework for this Button has already been coded when I was working on the Log in GUI e.g. temp text and resetting attempts. The algorithm below is for resetting the Log in GUI so that it clears the previous attempts and resets it.

```
Function LogInGUI ()
    Use message box to ask if the user would like to return to login
    If yes then
        Reset attempts //(already coded)
        Reset Temp Text //(already coded)
        Replace the options frame with the log in frame
        //(it may be easier to code them moving separately)
    Else:
        Print("returned to options")
        //for test purposes
    Endif
    Return
End function
```

37.3.9. Coding btnLogIn

Below is the code for the movement part of the algorithm above along with connecting the button click to this subprogram:

```
btnLogIn['command'] = LogIn
```

I need to connect the correct subprogram with the button event handler

```
frmLogIn.place(x = 0, y = 0, width = 645, height = 207)
frmOptions.place(x = 5, y = 500, width = 645, height = 207)
width, height = 645, 207
root.geometry("{}x{}".format(width,height))
```

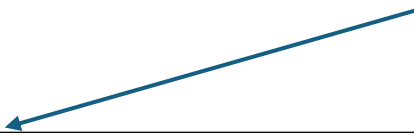
I have decided to use .place() to move around the frames because I was able to use it to reset the Username when I tested the Log In GUI.

I am using the same method for this message box that I used for the Cancel button on the log in page.

```
action = mbox.askyesno("Account", "If you do this you will have to
if action:
```

37.3.10. General Code for moving around frames

Replace the (to be moved) with the frame that the button is associated with moving to.



```
frm(to be moved).place(x = 0, y = 0, width = 645, height = 207)
frmOptions.place(x = 5, y = 500, width = 645, height = 207)
width, height = 645, 207    #total dimensions of frame just in
case others ate different sizes
root.geometry("{}x{}".format(width,height))
```

Maintenance: To improve efficiency, I could try to move all frames in a single procedure. Below I have created the sub program algorithm to do this but due to time constraints, I would implement this in a later prototype:

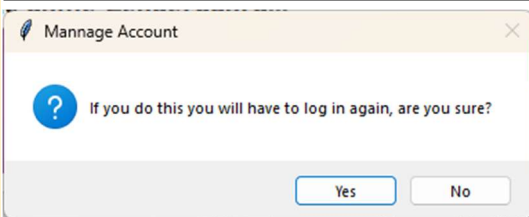
```
procedure swap frames(frame1, frame2):
    Get width and height of frame2
    Move frame1 out of viewable window
    Resize viewable window to accommodate frame2
    Move frame 2 into the window
End procedure
```

37.3.11. btnManageAccount

The code below is to replace the Options GUI with the Manage Account GUI when the Manage Account button is pressed:

```
def ManageAcc():  
    global frmLogIn, frmManage, txtNUser, txtUsername, root  
    action = mbox.askyesno("Mannage Account", "If you do this you will  
have to log in again, are you sure?")  
    if action:  
        #reset temp text if there is any  
        frmOptions.place(x = 5, y = 500, width = 645, height = 207)  
        frmManage.place(x = 0, y = 0, width = 645, height = 207)  
        width, height = 645, 207  
        root.geometry("{}x{}".format(width,height))  
    return
```

This code should work when I have added the Manage Account GUI. If I have temporary text in my Manage Account GUI, I will have to re-define the text box widget in this function



The User input decision message box for the user to clarify what the user would like to do.

37.3.12. btnPlayGame

The code below is to replace the Options GUI with the Play Game GUI when the Play Game button is pressed:

```
def Play_Game():
    global root, frmPlayGame, frmOptions, clbValues
    frmOptions.place(x = 5, y = 500, width = 645, height = 207)
    frmPlayGame.place(x = 0, y = 0, width = 645, height = 207)
    width, height = 645, 207
    root.geometry("{}x{}".format(width,height))
    clbValues = Games()
    print(clbValues)
    clbGameSelect.config(values=clbValues)
    return
```

Get the games to be displayed (This makes sure that the game list will be updated every time the Play Game GUI is open)

Note: I need to make sure I use the same variable names when creating the Play Game GUI.

To update the values I have called a function from the callbacks file. This finds the Maze nicknames and saves them to the array Games which is returned to be saved into clbValues.

```
def Games():
    Qry = '''SELECT * FROM Maze'''
    #run it - dbfile set global in LogOnDatabase
    AnsTable = RunQuery(DBFile, Qry)
    print(AnsTable)
    length = len(AnsTable)
    print(length)
    count = 0
    Game = []
    for count in range (0, length):
        #ref:https://docs.python.org/3/tutorial/datastructures.html#more-on-lists
        Game.append(AnsTable[count][5])
        count = count+1
    print(Game)
    return Game
```

37.3.13. btnAdmin

The code below is to replace the Options GUI with the Admin Management GUI when the Admin button is pressed (this is for Admins only):

In order to make sure that Admin is only accessible to Admin users I need to know what the username of the account is to compare to the users in the database.

This is a global variable declared in the Log In Submit procedure. `Username = aStr`

```
def ManageAdmin():
    if UserAdmin(Username):
        frmOptions.place(x = 5, y = 500, width =
645, height = 207)
        frmAdmin.place(x = 0, y = 0, width =
645, height = 207)
        width, height = 645, 207

    root.geometry("{}x{}".format(width,height))
    else:
        mbox.showerror("Permission Denied",
"User '%s' is not permitted here"%Username)

    return
```

```
def UserAdmin(User):
    #create query
    Qry = '''SELECT * FROM User
        WHERE UserName = "%s"''' %User
    #run it - dbfile set global in LogOnDatabase
    AnsTable = RunQuery(DBFile, Qry)
    print(AnsTable)
    if AnsTable[0][2] == "A":
        return True
    else:
        return False
```

This procedure is controlled by an if statement which calls the function UserAdmin from callbacks to compare the username to admin users in the database.

Once the user permissions are checked the process of changing the frames is the same as the other frames.

The User Admin Function to query if the username is an Admin user.

37.3.14. btnLeaderboards

The code below is to replace the Options GUI with the Leaderboards GUI when the Leaderboards button is pressed:

```
def Leaderboards0():
    global Leader
    frmOptions.place(x = 5, y = 500, width = 645,
height = 207)
    frmLeader.place(x = 0, y = 0, width = 645, height
= 207)
    width, height = 645, 207
    root.geometry("{}x{}".format(width,height))
    Leader = Games()
    print(Leader)
    clbGames.config(values=Leader)
    return
```

Uses the exact same method of collecting the game names as the Play Game GUI. Calls the same function from callbacks to receive the same array returned.

37.3.15. Evaluation

These connections will work correctly when I connect a fully functioning GUI system with the same variable names. The main other point from this evaluation was the need to grey out disabled buttons – the most regularly seen of these will be the Admin button. Below is the code that I have developed to automatically provide greyed out disabled buttons according to user type:

```
if not UserAdmin(Username):
    btnAdmin["bg"] = "DarkOrange3"
    btnAdmin["fg"] = "grey80"
else:
    btnAdmin["bg"] = BtnCol
    btnAdmin["fg"] = TextCol
```

Functions in callbacks which checks if user is admin.

To make this work I also made btnAdmin globally declared

37.4. **Play Game**

The non admin stakeholder can play a game after they have successfully logged in and selected to Play Game from options.

- a. The user will select a game from a list of available games based on selected difficulty.
- b. The chosen game, with its lives, MaxTime and Clueset will be displayed and the user will play it until they win (all Clues solved) or they have lost (run out of time, lost all lives)
- c. When the Game has finished, stats will be recorded against the user in the database.
 - i. Maintenance: Discuss criteria with stakeholders to see if there any other criteria/stats.
 - ii. Score achieved
 - iii. Total Score possible
 - iv. If the game was won or lost
 - v. Lives left
 - vi. Time taken
 - vii. Number of easy clues solved, number of medium clues solved, number of hard clues solved.
- d. The accuracy will be computed using the stats and will be recorded - (algorithm tbd)
- e. When the game is complete it will close and return to the playgame option so the user can select another game to play
- f. When the user has finished playing games then they can return to the options
- g. Note: Admin Users Cannot play games – They can check that mazes they have added are available to be selected.

37.4.1. Development of play game

For the Play Game GUI:

```
Let the User select difficulty level – (Easy medium hard)
Query the maze table in the database
Display list of mazes available – by Nickname, sorted alphabetically (by difficulty)
Record which Nickname is selected by the user
If the User is not admin then
    Query the database for
        The Maze filename (and its associated Clueset filename)
        The Avatar filename of the user
        The Username of the player (for personalized messages)
    Call PlayGame, passing these parameters
    // let the game play until end
    capture stats and update the User_Maze link table in the database
    Compute and add accuracy value for the record in User_Maze
endif
return to play game GUI
```

Note: I have already developed some of the PlayGame function and need to adapt it to accommodate the Username in its call

```
def PlayGame(aMazeFile, aClueFile, Username, HeroAvatarFile):

    # Function that is called to set up and manage a given maze game
    # with a given Clue set and Hero avatar
```

BUT the aMazeFile will not contain the MaxTime value or the Nickname associated with it.

A User record will have their username and avatar filename with it.

So should I pass records to the playgame function of Query the database within playGame

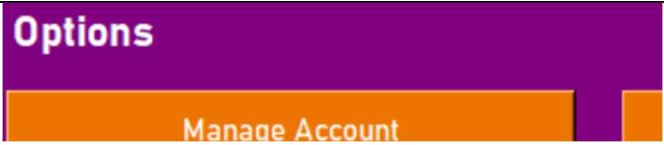
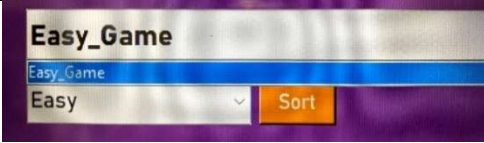
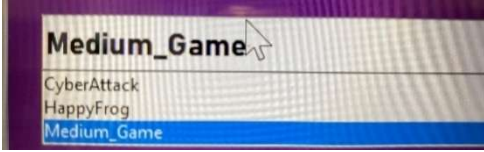
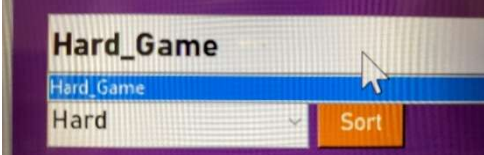
So I would end up with PlayGame (aMazeRecord, aClueFile, aUserRecord) and have to unpack these inside the function.

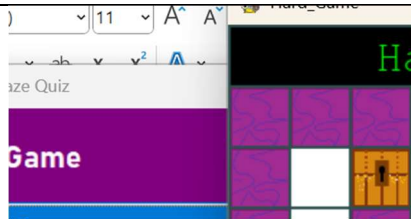
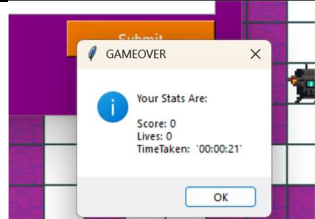
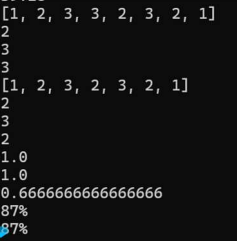
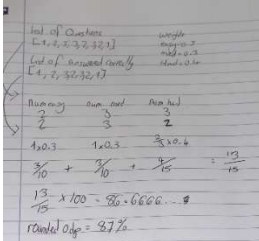
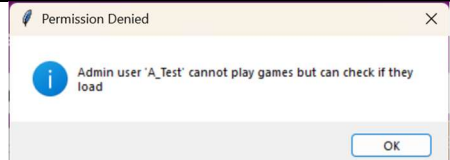
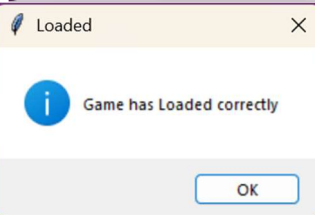
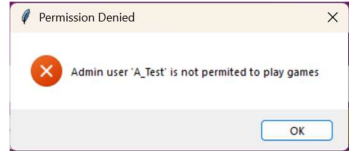
I will do the latter – the event handling mechanism can query the records and pass the records to the PlayGame() function, to be unpacked. This keeps PlayGame ‘clean’ - it only has to manage the game play.

Note: check the authentication within Login to see if whole User record is globally available. –

Results of check: No, only the username is held in the global Username from Login.

37.4.2. Test Plan(White box testing) – PlayGame GUI

No.	Description	Input	Expected Outcome	Actual Outcome	P/F
1	Play Game GUI appears when selected from options GUI and options GUI disappears from view	Button click on the options GUI	The Play game GUI appears on screen		P
2	The Play Game GUI has: A drop down list box showing all the Game Nicknames sorted alphabetically with related difficulty. A button to play. A back button to return to the options GUI	View the PlayGame GUI	The Play Game GUI includes all previously mentioned (that work according to the tests below)		P
3	Clicking the Back button hides the Play Game GUI and returns to the options GUI	Back Button click on the PlayGame GUI	The Options GUI appears on screen		P
4	The user selects an easy Maze	Easy difficulty selected	Only Easy mazes appear in the drop down (should be 1)		P
5	The user selects a Medium maze	Medium difficulty selected	Only Medium mazes appear in the drop down (should be 3)		P
6	The User selects a Hard Maze	Hard difficulty selected	Only Hard mazes appear in the drop down (should be 1)		P

7	The game is launched for the user to play (game window opened)	A game is selected and run.	game window opened (correctly initialized)		P
8	When the game finishes, the correct stats are added to the user maze	Stats a game is run	More stats when I check the database		P
9	The accuracy is calculated and added to the User_Maze record	A game ends	Accuracy is calculated and appears correctly in the console (also in record)	  The results match.	P
10	Admin Users cannot play games but they can check if the game load.	Admin user attempts to play a game	Game Loading is checked but the game window does not open	  	P

After Developing my Play Game GUI, I decided to give more detail to the users of what error had occurred when signing in for usability and ease of access.

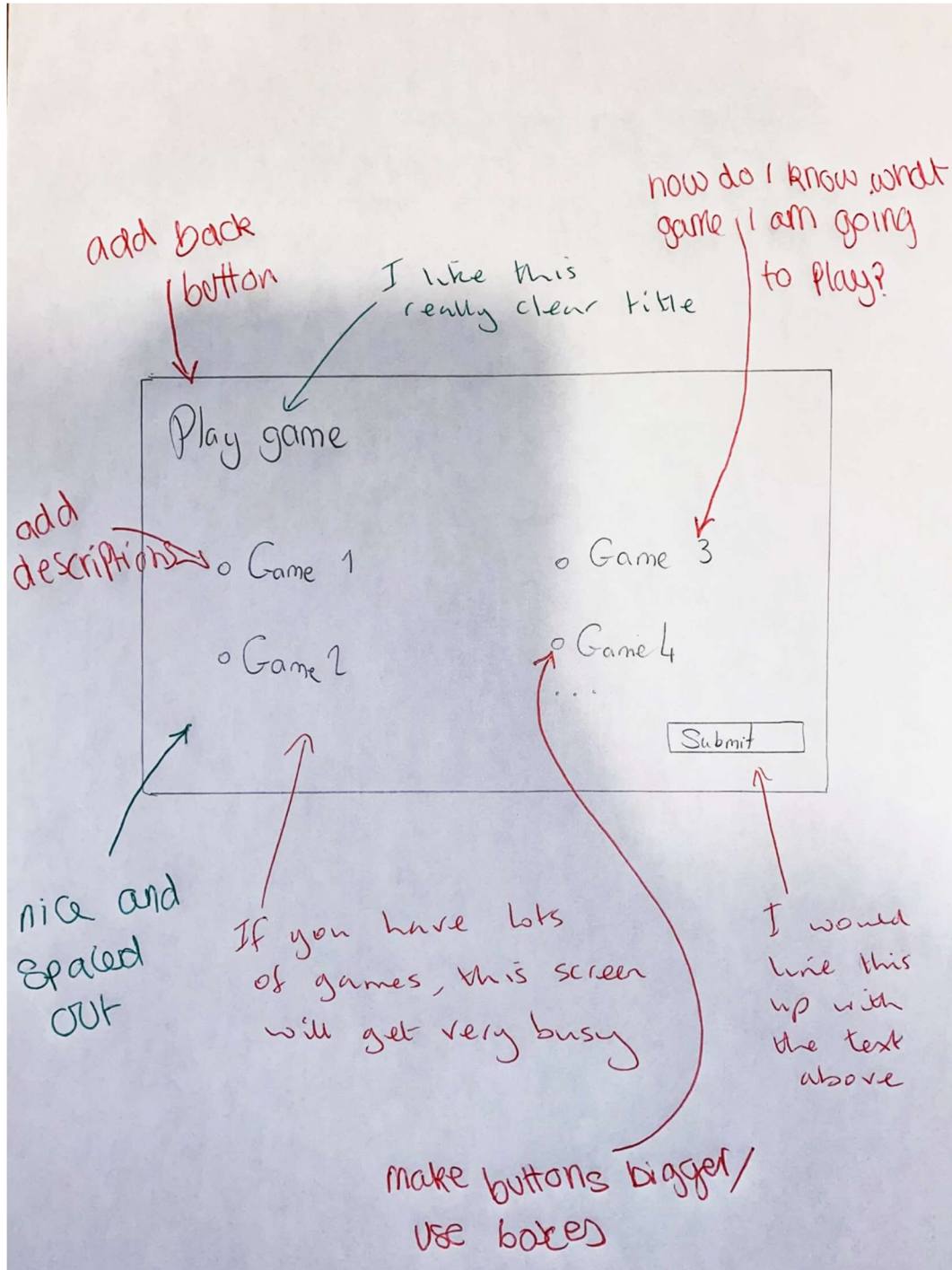
*Note added after completing the testing in the test plan: After running through the tests I noticed that the default text in the difficulty setting was not resetting when I went back into the game. I realised that when I reset the Select game, I forgot to reset the difficulty sort.



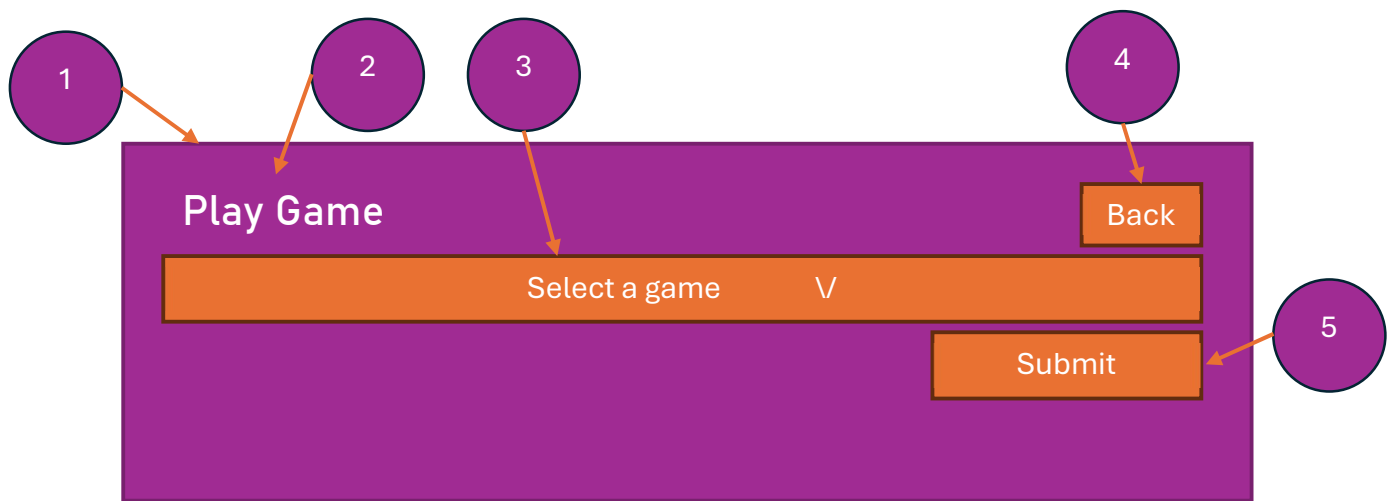
```
root.geometry("{}x{}".format(width,height))
clbGameSelect.set('Select a game...')           #reset clbs
clbGameDiff.set('Sort by Difficulty...')|
return
```

37.4.3. Design and Development

I started with a hand drawing of my GUI design for the stakeholders to comment on. The image below shows my stakeholder comments which included positive feedback and things to consider changing.



From this feedback, I have realised that I need to adjust my initial design to include a nickname for each game (this should also describe the game in some way). I also need to use a drop down instead of radio buttons to stop the screen from getting really busy for the User, a back button to go back to the previous options menu and I need to provide bigger buttons. I am pleased that they liked the spacing in my design and the GUI title.



ID	Name	Widget	Justification
1	frmPlayGame	Frame(container)	All the other widgets are held within the frame. The widgets are then easy to move within the frame for best layout on the screen.
2	lblPlayGame	Label	Read only title for this GUI so the user knows where they are at in the system.
3	clbGameSelect	ComboBox	A drop down menu for game selection meaning the player can only choose one game at a time and more games can be easily added. – The nicknames could be sorted alphabetically. Only available games are listed.
4	btnOptions	Button	A Button is a good option for the user to go back to the options GUI because it just has to be clicked.
5	btnSubmit	Button	A Button is a good option for the user to Play a game to go back to the options GUI because it just has to be clicked.

37.4.4.PlayGame GUI Development

```
def PlayGame():

    global frmPlayGame

    BackCol = "purple"      #default bg color
    TextCol = "white"       #default font color
    BtnCol = "DarkOrange2"  #Button color

    global clbValues

    clbValues = ["Hi", "These", "are", "Temporary", "Values"]  #to be replaced
    once database is connected

    #create the frame - container
    #same size as Log on frame
    frmPlayGame = tk.Frame(root, bg = BackCol)
    frmPlayGame.place(x = 5, y = 429, width = 645, height = 207)

    #same position as previous Title
    lblPlayGame = tk.Label(frmPlayGame, text = "Play Game",
                           bg = BackCol, fg = TextCol,
                           anchor="w", font=('Bahnschrift', 16, 'bold'))
    lblPlayGame.place(x = 25, y = 15, width = 201, height = 36)

    btnOptions = tk.Button(frmPlayGame, text = "Back",
                           bg = BtnCol, fg = TextCol, font=('Bahnschrift', 11) )
    btnOptions.place(x = 561, y = 23, width = 57, height = 36)

    btnSubmit = tk.Button(frmPlayGame, text = "Submit",
                           bg = BtnCol, fg = TextCol, font=('Bahnschrift', 11) )
    btnSubmit.place(x = 464, y = 109, width = 154, height = 38)

    clbGameSelect = ttk.Combobox(frmPlayGame, values = clbValues,
                                  text = "Select a game \/", state= "readonly",
                                  font=('Bahnschrift', 16, 'bold'))
    clbGameSelect.place(x = 23, y = 65, width = 595, height = 38)

    #event handlers

    return
```



37.4.5.Evaluation

My first attempt at creating the PlayGame GUI follows the design that I created from my stakeholder feedback and uses aspects of the other GUIs. The following is an area that I need to work on:

- Connect the database so the values on the drop down are accurate.

37.4.6.Corrections

To get the correct values from the database to be shown in the combo list box, I first need to create a subprogram to retrieve the game identifiers from the Maze table in the database.

```
Function Games          //function to get the game names from the maze table
    Query = ""SELECT MazeFilename FROM Maze""
    Result = RunQuery
    Length = len.Result //number of games
    Count = 0
    For count in range (0 to Length)
        Game = []      //declare array
        AppendGame(Result[count])          //add to separate array
    Next count
    Return Game
EndFunction
```

37.4.7.Coding for games array

```
def Games():
    Qry = '''SELECT MazeFilename FROM Maze'''
    #run it - dbfile set global in LogOnDatabase
    AnsTable = RunQuery(DBFile, Qry)
    print(AnsTable)
    length = len(AnsTable)
    print(length)
    count = 0
```

I have added print statements for testing purposes

```
    for count in range (0, length):
        Game = []
        #ref43:https://docs.python.org/3/tutorial/datastructures.html#more-on-
        #lists
        Game.append(AnsTable[count][0])
        count = count+1
    print(Game)
```

```
[('Maze1.txt',), ('Maze2.txt',), ('Maze3.txt',), ('Maze2.txt',)]
4
['Maze2.txt']
```

There is a Logic error in my code. It is running but only returning the last value in the AnsTable

Correction

After looking through my code I have realised that I need to declare the array Game outside of the for loop otherwise it is declared again overwriting the previous values for every loop.

```
Game = []
for count in range (0, length):
    #ref:https://docs.python.org/3
```

Fix for logic error

37.4.8.Stakeholders Comments

After discussing the design output from the code above with my stakeholders, they said they would like it better if the games had some sort of display name (nickname) to describe the game, which had previously been mentioned in the design development. They also said that it would be nice to be able to show games based on a selected difficulty.

37.4.9.Action on comment

To add a display name to the game, I need to edit the Table definitions in the database and add a Nickname field to the Maze Table.

```
Maze = '''CREATE TABLE IF NOT EXISTS Maze(
MazeID integer primary key autoincrement,
MazeFilename text NOT NULL,
Difficulty integer NOT NULL,
MaxTime time NOT NULL,
ClueID integer NOT NULL,
NickName text NOT NULL,
FOREIGN KEY (ClueID) REFERENCES ClueSet(ClueID)
)'''
```

Added NickName field

#ref: <https://hyperskill.org/learn/step/27151>⁴⁵
 #for time data type sqlite

Add Use Cases

```
aDML = '''INSERT INTO Maze(MazeFilename, Difficulty, MaxTime, ClueID,
NickName) VALUES("Maze1.txt", 1, "00:10:00", 3 , "Cyber Attack")'''
AddData(DBFile, aDML)
```

```
aSQL = "select * from Maze"
print(RunQuery(dbfile, aSQL))
```

```
aDML = '''INSERT INTO Maze(MazeFilename, Difficulty, MaxTime, ClueID,
NickName) VALUES("Maze2.txt", 3, "00:05:00", 2 , "Animal Farm")'''
AddData(DBFile, aDML)
```

```
aSQL = "select * from Maze"
print(RunQuery(dbfile, aSQL))
```

```
aDML = '''INSERT INTO Maze(MazeFilename, Difficulty, MaxTime, ClueID,
NickName) VALUES("Maze3.txt", 2, "00:80:00", 1, "Fun Fire Bombs History")'''
AddData(DBFile, aDML)
```

```
aSQL = "select * from Maze"
print(RunQuery(dbfile, aSQL))
```

From here I used a function that I named “game” in callbacks to query the database for both the Nickname and the difficulty level.

I added two new widgets into the play game function (difficulty and sort) and I used these to sort the games into difficulty level.

```
btnSort = tk.Button(frmPlayGame, text = "Sort",
                    bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )
btnSort.place(x = 199, y = 115, width = 57, height = 36)
```

I created a button called “Sort” to enter the sorted mode and this button has the same style as other buttons in this game. The code for this button design is shown below:

```
Level = ["Easy", "Medium", "Hard"]
clbGameDiff = ttk.Combobox(frmPlayGame, values = Level,
                           text = "Select Game Difficulty", state= "readonly",
                           font=('Bahnschrift', 13))
#ref:https://stackoverflow.com/questions/44711792/ttk-combobox-how-to-place-a-default-text-inside-the-box
clbGameDiff.set('Sort by Difficulty...')
clbGameDiff.place(x = 23, y = 115, width = 170, height = 36)
```

I have used a difficulty combo list box to provide the sort options. (Easy, Medium, Hard)



Further Development Idea: I would like users to have the ability to sort by creator (so that e.g. a class can be told to find a specific game created by their teacher) or by a game id. I am unable to develop this for this prototype due to time constraints.

Maintenance (achieved within first prototype): At present users have to return to the Options GUI and then return to the play game GUI to reset the Sort button. This is likely to annoy the users so in future, the user should be able to reset the filter so that all mazes are shown again.*

Note: I have now added an ‘All’ sort option to the sort drop down to resolve this issue because it was beginning to annoy me as well during development! My code for this solution is shown below:

```
def GameDiff():
    selecteddiff = clbGameDiff.get()
    Easy = []
    Med = []
    Hard = []
    All = []
    length = len(NickDiff)
    for count in range (0, length):
        #ref:https://docs.python.org/3/tutorial/datastructure
        if NickDiff[count][1] == 1:
            Easy.append(NickDiff[count][0])
            print(Easy) #test print
        elif NickDiff[count][1] == 2:
            Med.append(NickDiff[count][0])
            print(Med) #test print
        elif NickDiff[count][1] == 3:
            Hard.append(NickDiff[count][0])
            All.append(NickDiff[count][0]) #add all values
        print(Hard) #test print
    if selecteddiff == "Easy":
        Easy = sorted(Easy)
        clbGameSelect.config(values=Easy)
    elif selecteddiff == "Medium":
        Med = sorted(Med)
        clbGameSelect.config(values=Med)
    elif selecteddiff == "Hard":
        Hard = sorted(Hard)
        clbGameSelect.config(values=Hard)
    elif selecteddiff == "All": #if all are selected
        All = sorted(All)
        clbGameSelect.config(values=All)
    return

Level = ["Easy", "Medium", "Hard", "All"]
clbGameDiff = ttk.Combobox(frmPlayGame, values = Level,
                           text = "Select Game Difficulty", state= "readonly",
                           font=('Bahnschrift', 13))
#ref:https://stackoverflow.com/questions/44711792/ttk-combobox-how-to-place-a-default-text-inside-the-box
clbGameDiff.set('Sort by Difficulty...')
clbGameDiff.place(x = 23, y = 115, width = 170, height = 36)
```

Add All to the drop down.

Set so that all is set all mazes are shown.

37.4.10. **Event Handlers**

There are 2 buttons to consider in this frame:

btnOptions

When clicked,

go to the Options GUI

btnSubmit

when clicked,

check if User is admin if no

take the string from clbGameSelect

Find the Game in the database using its nickname.

Check if the game is online and has all the required features (Maze file, Clue file)

Initialize the game and run it.

Else

Message box to display “Admin users cannot play games”

37.4.11. **btnSubmit**

If not Admin:

txtNickName = take the string from clbGameSelect

GameDetails(txtNickName)

GameMaze = Maze[1]

ClueID = Maze[4]

ClueFile(ClueID)

GameClue = Clue

AvatarLoc(Username)

Avatar = User[3]

Run game in Canvas

PlayGame(GameMaze, GameClue, txtNickName, Avatar)

When game over

Store stats by passing to callbacks and calculating.

37.4.12. Coding loading Game

Code to get values to pass to Play Game function

```
def PlayMSelected(Nickname, Username):
    #function to set up and manage the stats from playing a Maze.

    #need to Query the database for User and Maze Record.
    #User Record
    Qry = '''SELECT * FROM User
            WHERE UserName = "%s"''' % Username
    UserRecord = RunQuery(DBFile, Qry) [0] #get tuple only
    print(UserRecord)
    #Avatar Filename
    Qry = '''SELECT AvatarFilename FROM Avatar
            WHERE AvatarID = "%s"''' % UserRecord[3]
    AvatarFilename = RunQuery(DBFile, Qry) [0] [0] #get tuple only
    print(AvatarFilename)
    #MazeRecord
    Qry = '''SELECT * FROM Maze
            WHERE NickName = "%s"''' % Nickname
    MazeRecord = RunQuery(DBFile, Qry) [0] #get tuple only
    print(MazeRecord)

    #get Clue file
    aQry = '''SELECT ClueFilename FROM ClueSet
            WHERE ClueID = "%s"''' % MazeRecord[4]
    Clue = RunQuery(DBFile, aQry) [0] [0]
    print(Clue)
    #User record contains the - Avatar filename, User Type
    #Maze record
    >>> PlayMSelected("Cyber Attack", "Test")
    (1, 'Test', 'U', 3)
    Avatar03.png
    (1, 'Mazel.txt', 1, '00:10:00', 3, 'Cyber Attack')
    Cyber.txt
```

Get Data to send to the Play Game function so that it can be managed and used to play the game.

To test this, I ran the callbacks file and called the PlayMSelected function to see if the data was being processed correctly.

I also tested this data using my already created GUIs to check that the Username value was being correctly used this was the result.

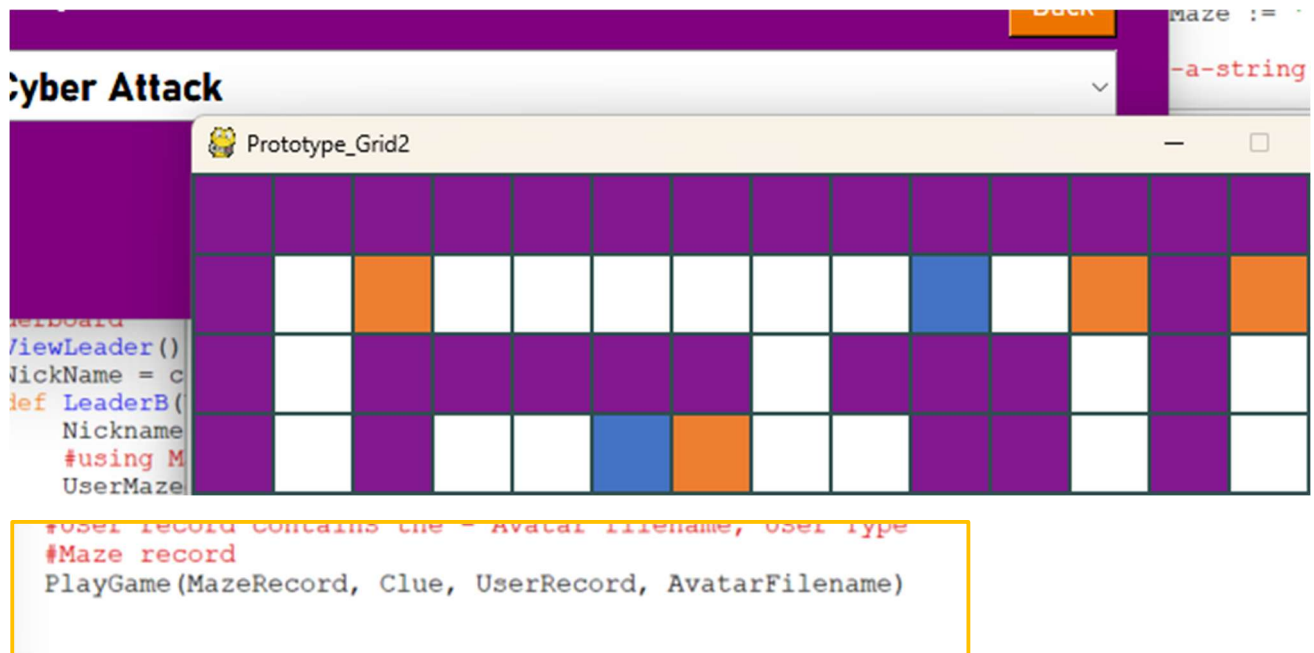


The Username has been correctly stored and passed as well as the Maze nickname I selected.

All the current mazes have been listed and the one selected has been displayed and processed.

```
['Cyber Attack', 'Animal Farm', 'Fun Fire Bombs History', 'Test_Game', 'Automati
c-Death-Trap', 'Hello', 'Test123', 'TestNick']
(1, 'Test', 'U', 3)
Avatar03.png
(6, 'Mazel.txt', 3, '00:30:00', 3, 'Hello')
Cyber.txt
```

To connect to the Play Game function, I called it within the PlayMSelect function in the callbacks as follows:



37.4.13. Check if the files given are valid

```
def ValidFile(MazeRecord, Clue, UserRecord, AvatarFilename):    #to check if files are valid and run the game
    aFileM = MazeRecord[1]
    aFileC = Clue
    Presence = LoadFile(aFileC)

    if Presence != [] and (len(Presence)>8):
        Presence = LoadFile(aFileM)
        if (Presence[0] == (len(Presence[0])*"W")):
            #iterate to remove new lines

            for row in range(0, len(Presence)):
                #ref to replace a value
                #https://www.digitalocean.com/community/tutorials/python-remove-character-from-string18
                Presence[row] = ((str(Presence[row])).upper()).replace("W", "")
                Presence[row] = ((str(Presence[row])).upper()).replace("E", "")
                Presence[row] = ((str(Presence[row])).upper()).replace("C", "")
                Presence[row] = ((str(Presence[row])).upper()).replace("S", "")
                Presence[row] = ((str(Presence[row])).upper()).replace("H", "")
                print(Presence[row])
                print("row:", row)

            #ref remove empty from array
            #https://stackoverflow.com/questions/3845423/remove-empty-strings-from-a-list-of-strings47
            while '' in Presence:
                Presence.remove('')

            print(Presence)

            if Presence == []:
                PlayGame(MazeRecord, Clue, UserRecord, AvatarFilename)
            else:
                print(Presence[0])
                mbox.showerror("File loading Failed", "Error loading file '%s', Game contains forbidden characters"%aFileM)

    else:
        mbox.showerror("File loading Failed", "Error loading file '%s'"%aFileM)

    return
```

37.4.14. Additional Event Handling for added elements.

btnSort

when clicked,

take the string from clbGameDiff

Find all the Games in the Maze table with the selected difficulty(this may be a separate subprogram)

Sort the Array of the selected difficulty alphabetically

Set the sorted array as the values displayed in clbGameSelect

Coding btnSort

```
def GameDiff():
    selecteddiff = clbGameDiff.get()
    Easy = []
    Med = []
    Hard = []
    length = len(NickDiff)
    for count in range(0, length):
        #ref:https://docs.python.org/3/tutorial/datastructures.html#more-on-lists43
        if NickDiff[count][1] == 1:
            Easy.append(NickDiff[count][0])
            print(Easy)
        elif NickDiff[count][1] == 2:
            Med.append(NickDiff[count][0])
            print(Med)
        elif NickDiff[count][1] == 3:
            Hard.append(NickDiff[count][0])
            print(Hard)
    if selecteddiff == "Easy":
        Easy = sorted(Easy)
        clbGameSelect.config(values=Easy)
    elif selecteddiff == "Medium":
        Med = sorted(Med)
        clbGameSelect.config(values=Med)
    elif selecteddiff == "Hard":
        Hard = sorted(Hard)
        clbGameSelect.config(values=Hard)
    return
```

Create arrays of each difficulty level.

This array also sets the values for the clbGameSelect dependant on what value is in the sort drop down.

- Maintenance (achieved within first prototype): I would like to disallow Admin users from playing games with a greyed out submit button and display an access denied message once clicked. I may develop this in the future. *Note: I developed this, shown in the code below:

```
if not UserAdmin(Username):
    PlayMSelected(SelectedGame, Username)
else:
    mbox.showerror("Permission Denied", "Admin user '%s'" % Username)
return
```

To change the colouration of the Submit button to look greyed out if User is Admin

This is in the Playgame function to replace the options GUI with the play game GUI



Using callbacks UserAdmin Function to check if the user is an admin or not.

If they are an admin then permission is denied and the PlayMSelected function is not run.

```
if UserAdmin(Username):
    btnSubmitGame["bg"] = "DarkOrange3"
    btnSubmitGame["fg"] = "grey80"
else:
    btnSubmitGame["bg"] = BtnCol
    btnSubmitGame["fg"] = TextCol

#reload games just in case
NickDiff = Games()
```

37.4.15. Saving Game Stats (developed after game functionality work)

Now that my game is playable for all the outputs that are saveable, I need to create a procedure to save the data returned to callbacks from playgame. This procedure must be able to get all the values and store them correctly in the User_Maze Database table. To do this I have created a Save_to_DB function which is connected to the PlayMSelected procedure which runs the game. The algorithm for this function is shown below:

```
Function SaveData(pass values executed by the game)
    Query to find the related IDs for the Maze and User records mentioned
    Query to see if there is any data for these values already in the table.
    If Not Data then                                //different actions if there is data present
        Add a record to the table with the ID values from before and the values from
the PlayGame Function
    Else
        Update previous data with the ID values from before and the values from the
PlayGame Function
    //check if the data has been added
    Query to check the data for the all the values just added to the table
    If not Query then                                //returns true if the data has been correctly added.
        Return false
    Else then
        Return true
Endfunction
```

The code below is for the function outlined above:

```
def Save_to_DB(Score, Lives, TimeTaken, NumEasy, NumMed, NumHard, Accuracy, Comp, Username, Nickname):
    aQry = '''SELECT UserID From User WHERE UserName = "%s"'''%(Username)
    UserID = RunQuery(DBFile, aQry)
    UserID = UserID[0][0]                                #find the User ID for the named record
    print(UserID)                                         #test
    aQry = '''SELECT MazeID From Maze WHERE NickName = "%s"'''%(Nickname)
    MazeID = RunQuery(DBFile, aQry)
    MazeID = MazeID[0][0]                                #find the User ID for the named record
    print(MazeID)                                         #test
    aQry = '''SELECT * From User_Maze WHERE MazeID = %i AND UserID = %i'''%(MazeID, UserID)
    GameUsers = RunQuery(DBFile, aQry)
    print(GameUsers)
    if not GameUsers:                                    #is there already a record under these details?
        aDML= '''INSERT INTO User_Maze(Score, TimeTaken, NumEasy, NumMedium, NumHard, Accuracy, UserID, MazeID, COMP)
VALUES(%i, "%s", %i, %i, %i, "%s", %i, %i, %d)''' %(Score, TimeTaken, NumEasy, NumMed
        AddData(DBFile, aDML)                            #new record
        print("UserMaze Added")
    else:
        aDML= '''UPDATE User_Maze SET Score = %i, TimeTaken = "%s", NumEasy = %i, NumMedium = %i,
        NumHard = %i, Accuracy = "%s", UserID = %i, MazeID = %i, COMP = %d)''' %(Score, TimeTaken,
        AddData(DBFile, aDML)                            #append record
        print("Update UserMaze Added")
    aQry = '''SELECT Score, TimeTaken, NumEasy, NumMedium, NumHard, Accuracy, UserID, MazeID, COMP From User_Maze'''
    GameUsers = RunQuery(DBFile, aQry)
    print(GameUsers)                                     #check if successful
    if GameUsers[0] == (Score, TimeTaken, NumEasy, NumMed, NumHard, Accuracy, UserID, MazeID, Comp):
        return True
    else:
        return False
```

Get all the values
needed to be added to
the User_Maze record

After trying to run this an error has occurred as follows:

```
Exception in Tkinter callback
Traceback (most recent call last):
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.8_3.0.0_x64_qbz5n2kfra8p0\lib\tkinter\__init__.py", line 1892, in __call__
    return self.func(*args)
  File "C:\Users\RosaB\Downloads\Cyber-Quiz-Maze\interfaces.py", line 232, in playMSelected
    PlayMSelected(SelectedGame, Username)
  File "C:\Users\RosaB\Downloads\Cyber-Quiz-Maze\callbacks01.py", line 434, in save_to_DB
    Save = Save_to_DB(Score, Lives, TimeTaken, NumEasy, NumMed, NumHard, Accuracy, Comp, Username, Nickname)
  File "C:\Users\RosaB\Downloads\Cyber-Quiz-Maze\callbacks01.py", line 504, in save_to_DB
    AddData(DBFile, aDML) #append record
  File "C:\Users\RosaB\Downloads\Cyber-Quiz-Maze\LogOnDataBase.py", line 10, in AddData
    curs.execute(aScript) #add record
sqlite3.OperationalError: near ")": syntax error
  (Score, TimeTaken, NumEasy, NumMed, NumHard, Accuracy, Comp, Username, Nickname)
  (Score, TimeTaken, NumEasy, NumMed, NumHard, Accuracy, Comp, Username, Nickname)
```

This error mentions a syntax error. Tracing this error back I believe it originates from the save_to_DB function.

This is most likely due to a careless mistake whilst copying the long list of variables from the Insert option to the update

Accuracy Error

I have noticed that the accuracy is not being displayed correctly in the leaderboard. I think this is an issue with saving the data / retrieving the data as the leaderboard only retrieves data.

```
'<function Accuracy at 0x000001AE4D64BD30>',
```

After looking at the sub-program (PlayMSelected) I have noticed a spelling inconsistency with the accuracy stat as follows due to the word “accuracy” missing a “c”.

```
#validate the files
if ValidFile(MazeRecord, Clue, UserRecord, AvatarFilename):
    if not UserAdmin(Username):
        Score, Lives, TimeTaken, NumEasy, NumMed, NumHard, Accuracy, Comp = PlayGame(MazeRecord, Clue, UserRecord, AvatarFilename)
        Save = Save_to_DB(Score, Lives, TimeTaken, NumEasy, NumMed, NumHard, Accuracy, Comp, Username, Nickname)
        if Save:
            mbox.showinfo("Added", "Data correctly added to database")
        else:
            mbox.showerror("Error", "There was an error saving your game details to the database") #for testing
    else:
        mbox.showinfo("Loaded", "Game has Loaded correctly")
        mbox.showerror("Permission Denied", "Admin user '%s' is not permitted to play games"%Username)
```

This part of the sub program is in charge of calling save_to_DB and getting back the results from PlayGame. I have noticed that I am accidentally calling a variable version of the play game subprogram when sending the values to be saved.

Now that the spelling is correct the correct data will be saved.

```
NumHard, Accuracy, Comp = PlayGame(MazeRecord, Clue,
NumEasy, NumMed, NumHard, Accuracy, Comp, Username,
Lives, TimeTaken, NumEasy, NumMed, NumHard, Accuracy, Comp = PlayGame(MazeRecord, Clue, UserRecord,
Save_to_DB(Score, Lives, TimeTaken, NumEasy, NumMed, NumHard, Accuracy, Comp, Username, Nickname)
```

37.5. **Manage Account**

- a. To manage an account a stakeholder must enter a username as a string (this can be new or current).
- b. Valid Characters are 'A' – 'Z', 'a' – 'z', '0' – '9' and selected special characters
- c. Validation: (must be the same as Log in)
 - i. Presence check (something entered)
 - ii. Length Check 1 to 12 characters inclusive
 - iii. Type check – valid char type
 - iv. Maintenance: I would consider sanitisation to prevent SQL injection opportunities in a future prototype.
- d. Authentication: once validated the username must be authenticated against a stakeholder already in the database.
 - i. If the username is already in the database the user will be able to edit the Avatars of that username (Usernames cannot be changed once created).
 - ii. If the username is not already in the database it will be added.
- e. An edited/created message will be displayed specific to the user if the Account is successful saved.
- f. An Error message will be displayed if validation has failed.
- g. A User will have the option of going to the Login page to Login either for the first time or into an account that has already been created (the Login GUI will be removed and the Manage account GUI will be displayed).

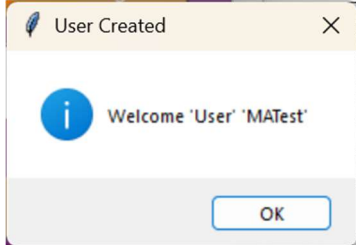
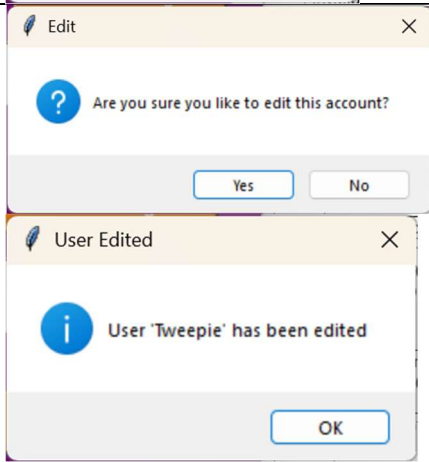
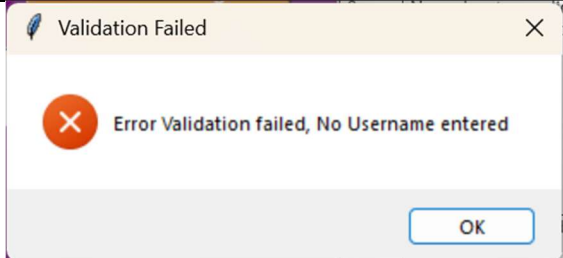
Maintenance: From the state transition diagram, in a future prototype, it would be good if:

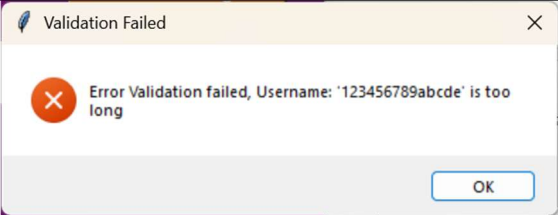
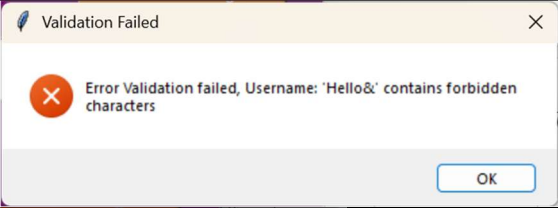
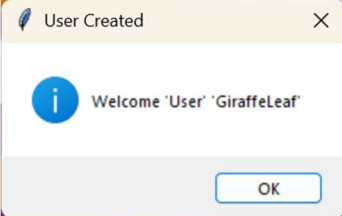
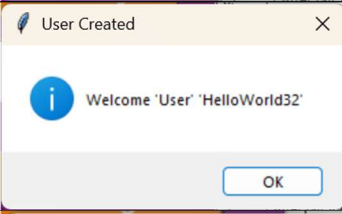
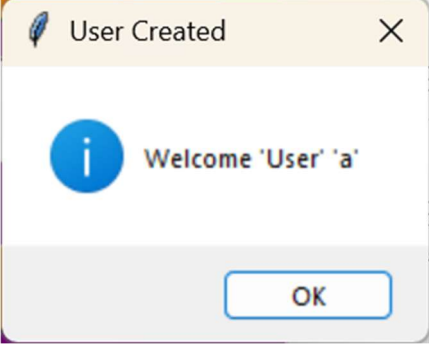
1. (achieved within first prototype) When accessed from the log in gui the username field should be blank
2. When accessed from the options gui the username could contain the current username

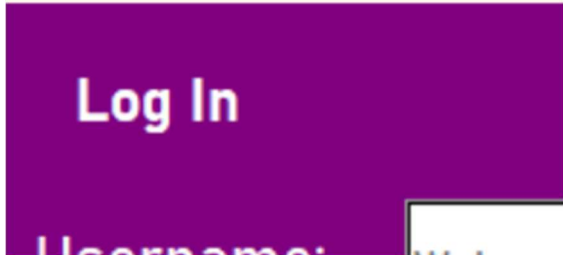
This would make the Manage Account process seem more friendly and allow the User to change their username.

***Limitation:** This is a very important limitation that must be resolved urgently. A delete button should be added to allow a user to delete an account. This is important due to the Data Protection Act of 1998 which requires unnecessarily held personal data to be deleted. But an admin may wish to review performance of all users including deleted accounts, so this will need to be resolved when the fix is put in. This could be resolved by connecting deleted users to an unknown status for their username, thus anonymising their data and allowing for account deletion whilst still being able to view the game data. I do not know how I could achieve this currently, but I know it is an urgent issue that needs to be solved as soon as possible in a next prototype.

37.5.1. Test Plan(White box testing) – development of Manage Account event handler

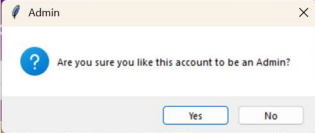
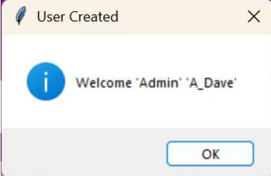
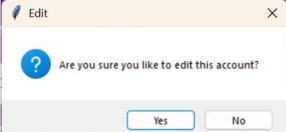
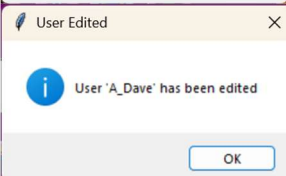
No.	Description	Input	Expected Outcome	Actual Outcome	P/F
1	Normal: enter a valid username(Not Authentic)	Username="MATest" Avatar = 3	Message "Welcome User MATest"	 A dialog box titled "User Created" with a close button (X). It contains an information icon (i) and the text "Welcome 'User' 'MATest'". At the bottom is an "OK" button.	P
2	Normal: enter a valid username(Authentic)	Username = "Tweepie"	Message "User Tweepie has been edited"	 Two stacked dialog boxes. The top one is titled "Edit" with a close button (X), contains a question mark icon (?), and the text "Are you sure you like to edit this account?". It has "Yes" and "No" buttons. The bottom one is titled "User Edited" with a close button (X), contains an information icon (i), and the text "User 'Tweepie' has been edited". It has an "OK" button.	P
3	Invalid: Enter invalid username				
a.	Invalid: Fails presence check	""(NULL)	Message : "ERROR. Failed validation"	 A dialog box titled "Validation Failed" with a close button (X). It contains an error icon (X) and the text "Error Validation failed, No Username entered". At the bottom is an "OK" button.	P
b.					

c.	Invalid: Failed length check(too long)	Username = "123456789abcde"	Message : "ERROR. Failed validation"	 A dialog box titled "Validation Failed" with a close button (X). It contains an orange error icon and the text "Error Validation failed, Username: '123456789abcde' is too long". There is an "OK" button at the bottom right.	P
	Invalid: Failed type check(forbidden chars in a correct length username)	Username = "Hello&"	Message : "ERROR. Failed validation"	 A dialog box titled "Validation Failed" with a close button (X). It contains an orange error icon and the text "Error Validation failed, Username: 'Hello&' contains forbidden characters". There is an "OK" button at the bottom right.	P
5	After fail:Valid authentic log in	Username = "GiraffeLeaf"	Message "Welcome User GiraffeLeaf"	 A dialog box titled "User Created" with a close button (X). It contains a blue information icon and the text "Welcome 'User' 'GiraffeLeaf'". There is an "OK" button at the bottom right.	P
6	Extreme: username is valid	Username = "HelloWorld32" (12 chars)	Message "Welcome User HelloWorld32"	 A dialog box titled "User Created" with a close button (X). It contains a blue information icon and the text "Welcome 'User' 'HelloWorld32'". There is an "OK" button at the bottom right.	P
7	Extreme: username is valid	Username = "a" (1 chars)	Message "Welcome User a"	 A dialog box titled "User Created" with a close button (X). It contains a blue information icon and the text "Welcome 'User' 'a'". There is an "OK" button at the bottom right.	P

9	Test for button click Log in	Log in button clicked	Screen moves to Log on GUI		P
---	------------------------------	-----------------------	----------------------------	---	---

After Developing my Manage Account GUI, I decided to give more detail to the users of what error had occurred when creating an account for usability and ease of access.

Tests for Maintenance points addressed within this section of the prototype

1	The Type is changed to Admin when A_ is at the start of the username	"A_Dave"	An Admin User will be created instead of a regular User	 	P
2	Editing an already made account	"A_Dave"	A pop up will appear asking if you want to edit an account -No → nothing happens and the pop up disappears -Yes → A Message will appear with "User A_Dave has been edited"	 	P

3	Deleting the database to check if the Avatar records are added	Delete	Avatar 1 – 4 Added to the database.	<pre> [] Avatar Added Avatar1.png [] Avatar Added Avatar2.png [] Avatar Added Avatar3.png [] Avatar Added Avatar4.png [(1, 'Avatar1.png'), (2, 'Avatar2.png'), (3, 'Avatar3.png'), (4, 'Avatar4.png')] [(1, 'Avatar1.png')] [(2, 'Avatar2.png')] [(3, 'Avatar3.png')] [(4, 'Avatar4.png')] [(1, 'Avatar1.png'), (2, 'Avatar2.png'), (3, 'Avatar3.png'), (4, 'Avatar4.png')] </pre>	P
---	--	--------	-------------------------------------	--	---

I decided that I needed to have a default option for the Avatar. After testing this, I realised that the values didn't reset when I reopened the GUI so I consulted my stakeholders about it. They said they would like for the avatar to be clearly preselected. (I decided to default to avatar 1)

Maintenance (achieved within first prototype):

- If the Username entered begins with "A_" the account type is automatically set to Admin with a message to say what has been done when it is saved.

37.5.2. Additional Development After Initial Testing

After developing a few other GUIs, I realised that I needed to delete the database because it was full of unnecessary test value. It was at this point that I realised that since I have developed my Manage Account GUI, the Avatar files wouldn't be changing and it would be easier if they were automatically added and checked whenever the program runs. This would ensure no errors occur when a game is played because the avatar file is always called correctly.

This procedure is called in Callbacks to manage this:

```
CheckAvatar():          #function to ensure the Avatars are correct
for check in range(0, 4):
    ID = str(check + 1)
    AvatarFilename = "Avatar0" + ID + ".png"      #Concatenate the correct filename
    aQry = '''SELECT * From Avatar WHERE AvatarID = %i''' % (check+1)      #query the database
    AnsTable = RunQuery(DBFile, aQry)
    print(AnsTable)

    if not AnsTable:      #is there a record?
        aDML = '''INSERT INTO Avatar(AvatarID, AvatarFilename)
                VALUES(%i, "%s")''' % ((check+1), AvatarFilename)      #add a record
        AddData(DBFile, aDML)
        print("Avatar Added")
        print(AvatarFilename)
    elif AnsTable[0][1] != AvatarFilename:      #change the Avatar filename
        aDML = ('''UPDATE Avatar SET AvatarFilename="%s"
                WHERE AvatarID= %i''' % (AvatarFilename, (check+1)))
        AddData(DBFile, aDML)
        print("Avatar Edited")
    else:
        print("The Avatars are correct")      #the avatar record is correct

aQry = '''SELECT * From Avatar'''
AnsTable = RunQuery(DBFile, aQry)
print(AnsTable)      #display records for testing
return
```

First, I have used an If statement to iterate through the 4 different options for Avatars.

Then I queried the database to get the Avatar record for that index.

If the AnsTable is empty, a record is added to the database with the correct file input.

Otherwise if there is an incorrect filename, then the record will be updated with the correct filename.

At this point I also want to add Users quickly including Admin users without going into the Shell console. This was the maintenance point that I recorded after finishing my test plan. I have decided that all Admin User accounts will begin with “A_”. If it is not a current admin account, then an admin account will be added. Otherwise, the current Admin account will be edited as normal. My algorithm and code for this is shown below:

```
If aStr.left(2) == "A_" then
    Type = "A"
    Msg = Are you sure?
Else then
    Type = "U"
    Msg = False
If not Msg then
    Add User to DB
Else then
    Error message.
```

```
if aStr[:2] == "A_":
    Type = "A"
    action = mbox.askquestion("Admin", "Are you sure you like this account to be an Admin?")
else:
    Type = "U"
    action = False
if action == False:
    if aStr != "Please enter a username":
        AvatarID = pressed
        AddUser(aStr, AvatarID, Type)
    else:
        mbox.showerror("No Username Entered", "Error, This Username cannot be submitted")
else:
    mbox.showinfo("Admin not added", "User not added, please enter a Username without 'A_' at the start")
return
```

This is adapted from the previous version where the account type was not passed to the procedure in callbacks. By changing this I can easily add admin accounts by simply putting the prefix “A_” before them.

After coding this I realised that if I wanted to edit a pre-existing admin account it would ask the “Are you sure” question for account creation rather than allow the account to just be edited. This will need to be resolved as shown below:

```

def Save():
    #event handler for btnSubmit
    aStr = txtNUser.get()    #get entered username
    AvatarID = pressed
    edit = False
    ##    print(aStr[:2])    #test print
    if UserPresence(aStr):    #check if there is already a user
        edit = mbox.askyesno("Edit", "Are you sure you like to edit this account?")
        #print(edit) #test print

    else:
        edit = True

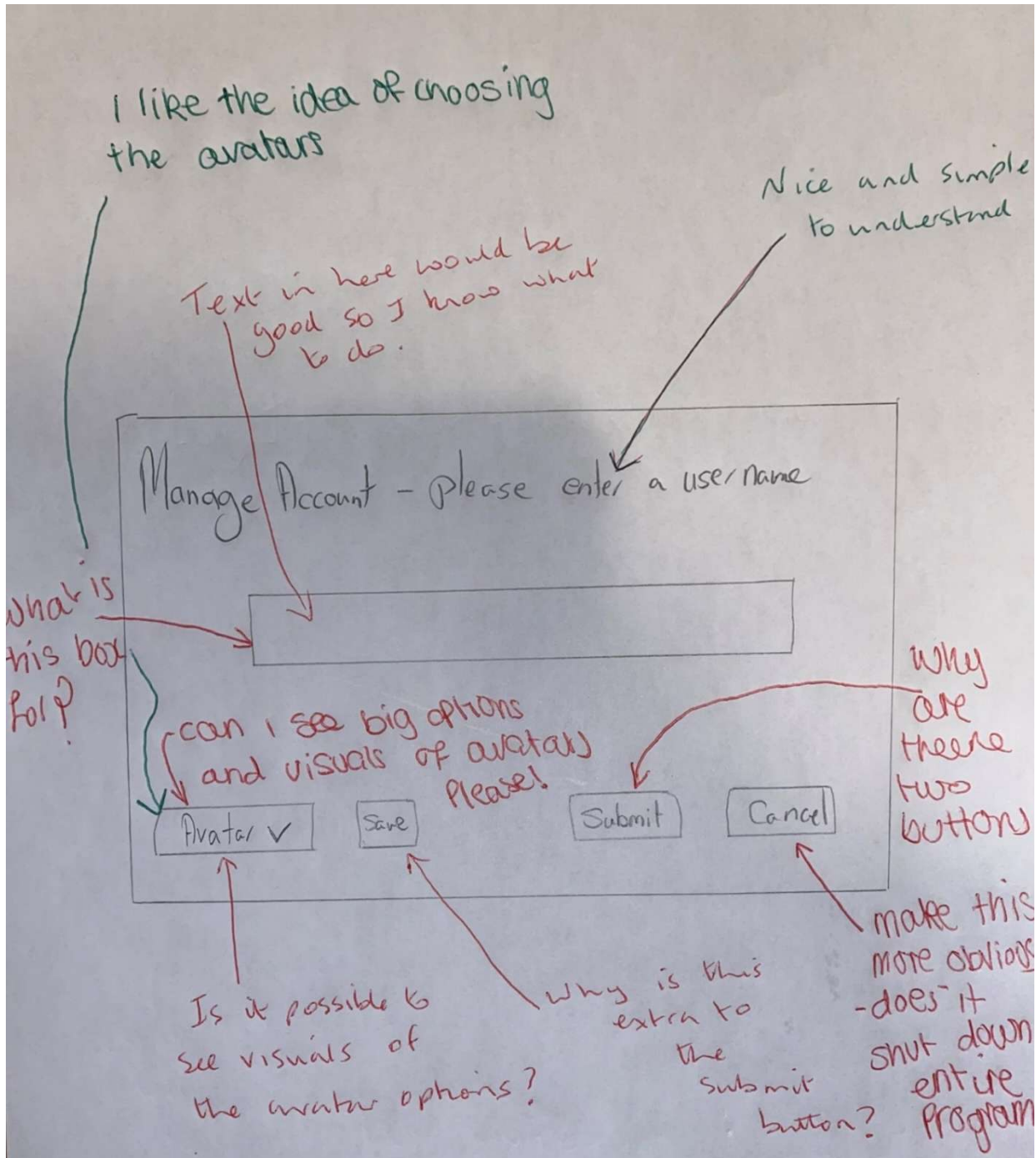
    if edit:
        action = False
        if aStr[:2] == "A_":
            Type = "A"
            if not UserPresence(aStr):    #check if user is for editing
                action = mbox.askyesno("Admin", "Are you sure you like this account to be an Admin?")
            else:
                action = True
        else:
            Type = "U"
            action = True
        if action:
            if aStr != "Please enter a username":
                AvatarID = pressed
                AddUser(aStr, AvatarID, Type)
            else:
                mbox.showerror("No Username Entered", "Error, This Username cannot be submitted")
        else:
            mbox.showinfo("Admin not added", "User not added, please enter a Username without 'A_' at the start")
    return

```

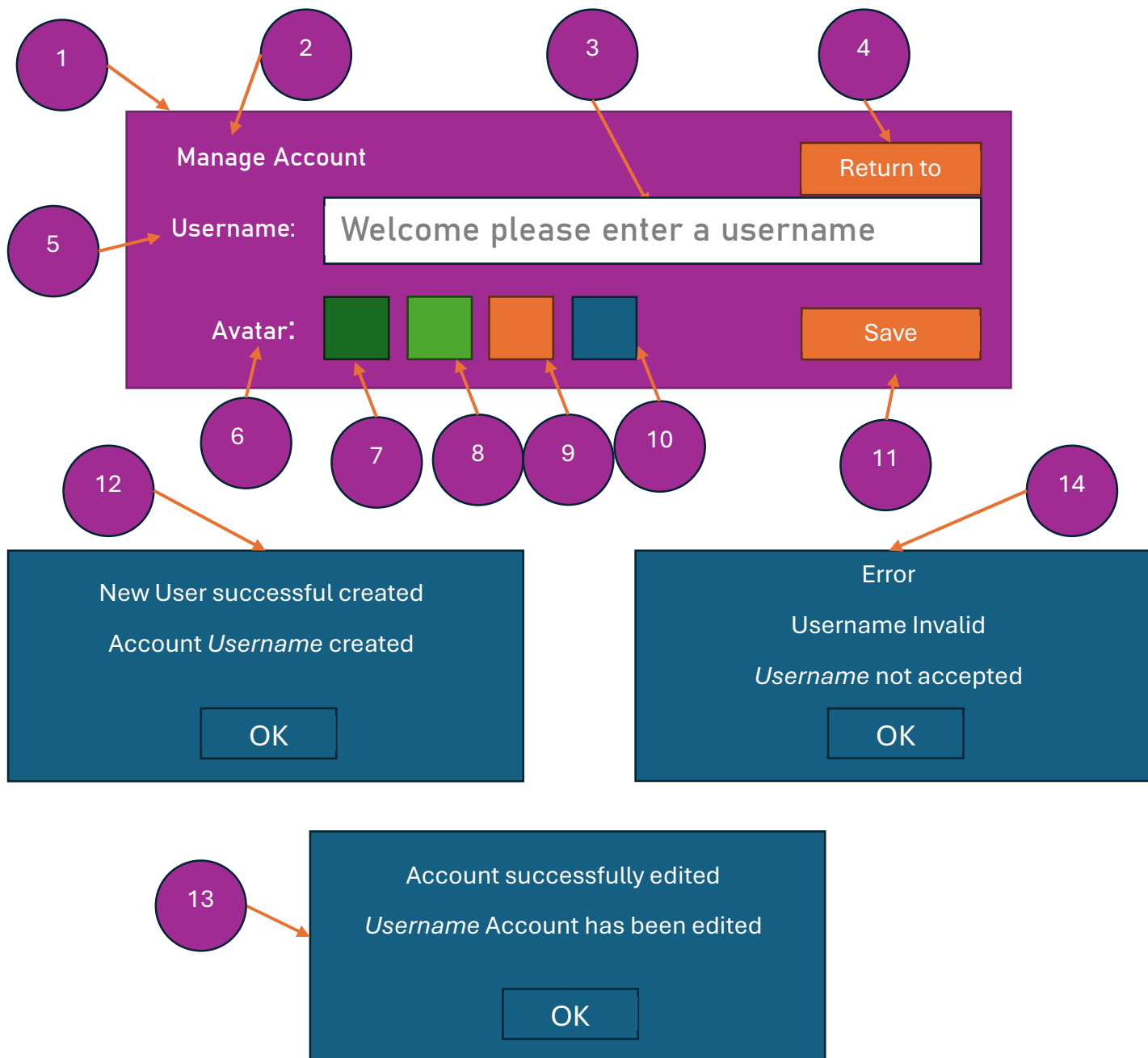
I needed to change the type of message box to askyesno as well because it ensures that a Boolean would be output.

37.5.3. Design and Development

I started with a hand drawing of my GUI design for Manage Account for the stakeholders to comment on. The image below shows my stakeholder comments which included positive feedback and things to consider changing.



After discussing this with my stakeholders, I have realised that I need to adjust my initial design to include a label for my input box and perhaps add some temporary text, Avatar visuals for the icons and I should have the same button for submit and save. I am pleased that they liked my plan to have the ability to choose avatars even if some areas of the design need improvements.



ID	Name	Widget	Justification
1	frmManage	Frame(container)	All the other widgets are held within the frame. The widgets are then easy to move within the frame for best layout on the screen.
2	lblManage	Label	Read only title for this GUI so the user knows where they are at in the system.
3	txtNUser	Entry	User input box for user to enter their User name with overwritable instruction text – Entry used because any variety of usernames need to be entered via keyboard

4	btnLogIn	Button	A Button is a good option for the user to go back to the LogInGUI.
5	lblUsername	Label	Read only identifier for the type of data to input so the user knows what type of data to input.
6	lblAvatar	Label	Read only identifier for the icons on the right so the user knows what they are.
7	imgAvatar1	button(image)	I needed to learn how to add images to buttons in tkinter and I found this reference to be a useful starting point: https://www.geeksforgeeks.org/how-to-add-an-image-in-tkinter/ ⁴⁸ An Image button is a good option for the user to select a specific avatar based on looks. Avatar1 is default (starts off shaded)
8	imgAvatar2	button(image)	An Image button is a good option for the user to select a specific avatar based on looks.
9	imgAvatar3	button(image)	An Image button is a good option for the user to select a specific avatar based on looks.
10	imgAvatar4	button(image)	An Image button is a good option for the user to select a specific avatar based on looks.
11	btnSubmit	Button	A Button is a good option for the user to save their details to the database or amend details already there.
12	msgSucessCreation	messagebox	To welcome user after successful User Creation Visual display to prove to the user that they have successfully created an account which is now saved to the database.
12	msgEditUser	messagebox	To welcome user after successful editing of a User account Visual display to prove to the user that they have successfully edited their account which is now appended in the database.
9	msgErrorLogIn	Messagebox	To report a problem to the user with their username. Visual display to tell the user that there is a problem with log in

37.5.4.ManageAccount GUI Development

```
def ManageAccount():
    global frmManageAccount, txtNUser, imgAvatar1, imgAvatar2, imgAvatar3,
    imgAvatar4

    BackCol = "purple"      #default bg color
    TextCol = "white"        #default font color
    BtnCol = "DarkOrange2"  #Button color
    #create the frame - container

    frmManage = tk.Frame(root, bg = BackCol)
    frmManage.place(x = 655, y = 5, width = 645, height = 207)

    #add widgets
    #ref35:https://www.geeksforgeeks.org/what-are-widgets-in-tkinter/
    #types of widget
    lblManage = tk.Label(frmManage, text = "Manage Account",
                        bg = BackCol,fg = TextCol,
                        anchor="w", font=('Bahnschrift', 16, 'bold'))
    lblManage.place(x = 25, y = 15, width = 201, height = 36)

    lblUsername = tk.Label(frmManage, text = "Username:",
                        bg = BackCol,fg = TextCol, anchor="e",
font=('Bahnschrift', 16))
    lblUsername.place(x = 11, y = 70, width = 122, height = 36)

    txtNUser = tk.Entry(frmManage, text = "",
                        bg = TextCol,fg = "dim gray", borderwidth=2,
font=('Bahnschrift', 11))
    txtNUser.place(x = 143, y = 68, width = 478, height = 46)

    txtNUser.insert(0, "Please enter a username")

    btnLogIn = tk.Button(frmManage, text = "Return to Login",
                        bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )
    btnLogIn.place(x = 464, y = 15, width = 154, height = 38)

    lblAvatar = tk.Label(frmManage, text = "Avatar:",
                        bg = BackCol,fg = TextCol, anchor="e",
font=('Bahnschrift', 16))
    lblAvatar.place(x = 40, y = 150, width = 89, height = 36)

    btnSave = tk.Button(frmManage, text = "Save",
                        bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )
    btnSave.place(x = 464, y = 150, width = 154, height = 38)

    #1 Ref49
    #https://www.geeksforgeeks.org/python-pillow-a-fork-of-pil/
    # Load the image using PIL
    original_image = Image.open("Avatar1.png") # Replace with your image path

    # Scale the image (change width and height as needed)
    width, height = 45, 45 # Desired dimensions
    scaled_image = original_image.resize((width, height), Image.LANCZOS)

    # Convert the scaled image to a Tkinter-compatible image
```

```

tk_image = ImageTk.PhotoImage(scaled_image)

# Create a button and set the image
imgAvatar1 = tk.Button(frmManage, image=tk_image, anchor="center",
command=lambda imgAvatar1 = 1: which_button(imgAvatar1))
imgAvatar1.place(x = 142, y = 138, width = 47, height = 47)

# Keep a reference to avoid garbage collection
imgAvatar1.image = tk_image

#2
#Load the image using PIL
original_image2 = Image.open("Avatar2.png") # Replace with your image path

# Scale the image (change width and height as needed)
width, height = 45, 45 # Desired dimensions
scaled_image2 = original_image2.resize((width, height), Image.LANCZOS)

# Convert the scaled image to a Tkinter-compatible image
tk_image2 = ImageTk.PhotoImage(scaled_image2)

# Create a button and set the image
imgAvatar2 = tk.Button(frmManage, image=tk_image2, anchor="center",
command=lambda imgAvatar2 = 2: which_button(imgAvatar2))
imgAvatar2.place(x = 205, y = 138, width = 47, height = 47)

# Keep a reference to avoid garbage collection
imgAvatar2.image = tk_image2

#3
#Load the image using PIL
original_image3 = Image.open("Avatar3.png") # Replace with your image path

# Scale the image (change width and height as needed)
width, height = 45, 45 # Desired dimensions
scaled_image3 = original_image3.resize((width, height), Image.LANCZOS)

# Convert the scaled image to a Tkinter-compatible image
tk_image3 = ImageTk.PhotoImage(scaled_image3)

# Create a button and set the image
imgAvatar3 = tk.Button(frmManage, image=tk_image3, anchor="center",
command=lambda imgAvatar3=3: which_button(imgAvatar3))
imgAvatar3.place(x = 264, y = 138, width = 47, height = 47)

# Keep a reference to avoid garbage collection
imgAvatar3.image = tk_image3

#4
#Load the image using PIL
original_image4 = Image.open("Avatar4.png") # Replace with your image path

# Scale the image (change width and height as needed)
width, height = 45, 45 # Desired dimensions

```

```

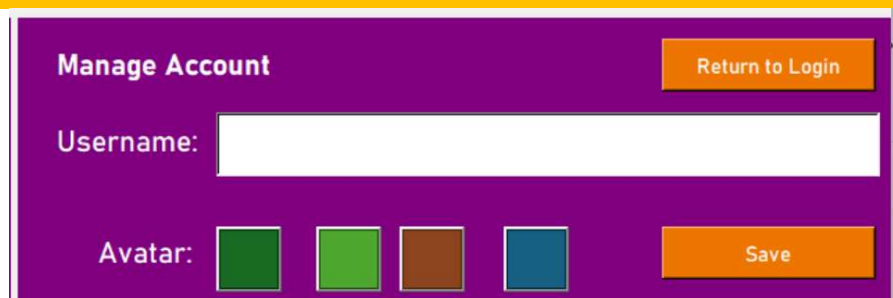
scaled_image4 = original_image4.resize((width, height), Image.LANCZOS)

# Convert the scaled image to a Tkinter-compatible image
tk_image4 = ImageTk.PhotoImage(scaled_image4)

# Create a button and set the image
imgAvatar4 = tk.Button(frmManage, image=tk_image4, anchor="center",
command=lambda imgAvatar4 = 4: which_button(imgAvatar4))
imgAvatar4.place(x = 325, y = 138, width = 47, height = 47)

# Keep a reference to avoid garbage collection
imgAvatar4.image = tk_image4

```



37.5.5. Evaluation

My first attempt at creating the Manage Account GUI follows the design above using aspects of my previously developed GUIs. To improve:

- Change the positions of the Avatar buttons.
- Add actual Avatars to the buttons

37.5.6. Corrections



```
def ManageAccount():
    global frmManage, txtNUser, imgAvatar1, imgAvatar2, imgAvatar3, imgAvatar4

    BackCol = "purple"      #default bg color
    TextCol = "white"        #default font color
    BtnCol = "DarkOrange2"  #Button color
    #create the frame - container

    frmManage = tk.Frame(root, bg = BackCol)
    frmManage.place(x = 655, y = 5, width = 645, height = 207)

    #add widgets
    #ref35:https://www.geeksforgeeks.org/what-are-widgets-in-tkinter/
    #types of widget
    lblManage = tk.Label(frmManage, text = "Manage Account",
                        bg = BackCol, fg = TextCol,
                        anchor="w", font=('Bahnschrift', 16, 'bold'))
    lblManage.place(x = 25, y = 15, width = 201, height = 36)

    lblUsername = tk.Label(frmManage, text = "Username:",
                        bg = BackCol, fg = TextCol, anchor="e",
    font=('Bahnschrift', 16))
    lblUsername.place(x = 11, y = 70, width = 122, height = 36)

    txtNUser = tk.Entry(frmManage, text = "",
                        bg = TextCol, fg = "dim gray", borderwidth=2,
    font=('Bahnschrift', 11))
    txtNUser.place(x = 143, y = 68, width = 478, height = 46)

    txtNUser.insert(0, "Please enter a username")

    btnLogIn = tk.Button(frmManage, text = "Return to Login",
                        bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )
    btnLogIn.place(x = 464, y = 15, width = 154, height = 38)

    lblAvatar = tk.Label(frmManage, text = "Avatar:",
                        bg = BackCol, fg = TextCol, anchor="e",
    font=('Bahnschrift', 16))
    lblAvatar.place(x = 40, y = 150, width = 89, height = 36)

    btnSave = tk.Button(frmManage, text = "Save",
                        bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )
    btnSave.place(x = 464, y = 150, width = 154, height = 38)

    #1 Ref49
```

```

#https://www.geeksforgeeks.org/python-pillow-a-fork-of-pil/
# Load the image using PIL
original_image = Image.open("Avatar1-dark70.png") # Replace with your image
path

# Scale the image (change width and height as needed)
width, height = 45, 45 # Desired dimensions
scaled_image = original_image.resize((width, height), Image.LANCZOS)

# Convert the scaled image to a Tkinter-compatible image
tk_image = ImageTk.PhotoImage(scaled_image)

# Create a button and set the image
imgAvatar1 = tk.Button(frmManage, image=tk_image, anchor="center",
command=lambda imgAvatar1 = 1: which_button(imgAvatar1))
imgAvatar1.place(x = 142, y = 138, width = 47, height = 47)

# Keep a reference to avoid garbage collection
imgAvatar1.image = tk_image

#2
#Load the image using PIL
original_image2 = Image.open("Avatar2.png") # Replace with your image path

# Scale the image (change width and height as needed)
width, height = 45, 45 # Desired dimensions
scaled_image2 = original_image2.resize((width, height), Image.LANCZOS)

# Convert the scaled image to a Tkinter-compatible image
tk_image2 = ImageTk.PhotoImage(scaled_image2)

# Create a button and set the image
imgAvatar2 = tk.Button(frmManage, image=tk_image2, anchor="center",
command=lambda imgAvatar2 = 2: which_button(imgAvatar2))
imgAvatar2.place(x = 205, y = 138, width = 47, height = 47)

# Keep a reference to avoid garbage collection
imgAvatar2.image = tk_image2

#3
#Load the image using PIL
original_image3 = Image.open("Avatar3.png") # Replace with your image path

# Scale the image (change width and height as needed)
width, height = 45, 45 # Desired dimensions
scaled_image3 = original_image3.resize((width, height), Image.LANCZOS)

# Convert the scaled image to a Tkinter-compatible image
tk_image3 = ImageTk.PhotoImage(scaled_image3)

# Create a button and set the image
imgAvatar3 = tk.Button(frmManage, image=tk_image3, anchor="center",
command=lambda imgAvatar3=3: which_button(imgAvatar3))
imgAvatar3.place(x = 264, y = 138, width = 47, height = 47)

```

```

# Keep a reference to avoid garbage collection
imgAvatar3.image = tk_image3

#4
#Load the image using PIL
original_image4 = Image.open("Avatar4.png") # Replace with your image path

# Scale the image (change width and height as needed)
width, height = 45, 45 # Desired dimensions
scaled_image4 = original_image4.resize((width, height), Image.LANCZOS)

# Convert the scaled image to a Tkinter-compatible image
tk_image4 = ImageTk.PhotoImage(scaled_image4)

# Create a button and set the image
imgAvatar4 = tk.Button(frmManage, image=tk_image4, anchor="center",
command=lambda imgAvatar4 = 4: which_button(imgAvatar4))
imgAvatar4.place(x = 325, y = 138, width = 47, height = 47)

# Keep a reference to avoid garbage collection
imgAvatar4.image = tk_image4

return

```

37.5.7.Event Handling

An event is when something occurs that is expected but it is not known when it will happen, for example the user clicking a button. There are 6 buttons to consider in this frame:

Individual Avatar buttons

```

Function Button_identifier:    //function to identify which avatar has been pressed
    If Button_identifier == imgAvatar1 then    //create a selection visual for the user
        Edit image button to become darker    //showing selected
        Check the other buttons are back to original    //make sure to remove
previous selections.
        Return avatar identifier
    Elseif Button_identifier == imgAvatar2...4 then    //the same for other
image buttons
        Edit image button to become darker    //showing selected
        Check the other buttons are back to original    //make sure to remove
previous selections.
        Return avatar identifier

```

btnSave - Uses the IsAuth and IsValid functions from the LogIn GUI

```
Function Save
    Get string from username input box           //check against database
    If String != temptext then                   //to stop the temporary text from being
entered
        Call function from callbacks and pass String value and button
identifier
    Else:
        Print("temp text cannot be entered")    //will eventually be an
error message to tell the use what is wrong
    Return
```

```
Function Add/Edit a user (string, Avatar identifier) //the two values are passed to
the function.
    Valid = false
    If IsValid
        Valid = true
    Else
        Print("failed")

    If Valid
        If Is Auth:    //username is currently in database
            Edit the Avatar user currently in the database for that
username to the new one
            Print("Username has been edited")
        Else:          //username not in database
            Insert the new User into the User table    //default user
            Check if the user is in the User table
            Print check ("user successfully added to database")
```

btnLogin

```
When clicked,
    go to the Log In GUI
```

37.5.8. Development of Login Event Handlers

From the algorithm design, I can use the subprograms I made for the Login GUI:

IsValid() – Used to validate the data and check that the username entered is valid. This is equally important in this GUI even to check usernames already in the database to remove the possibility of SQL injections working on this portal.

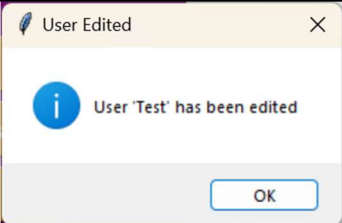
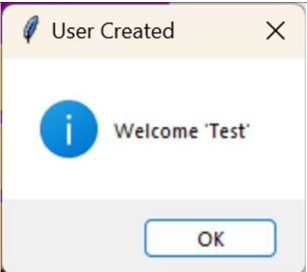
IsAuth() – Used to check if a Username is already in the database. This changes how the data is input into the database.

To combine these subprograms in the way I want, I have used the function AddUser.

```
def AddUser(aStr, AvatarID):
    #function to validate and authenticate a username
    print(aStr + ":", end = '')
    Valid = False

    if IsValid(aStr):
        print("Username is valid")
        Valid = True
    else:
        print("not valid")
    if Valid:
        if NotAuth(aStr):
            aDML = '''INSERT INTO User(Username,Type, AvatarID) VALUES("%s", "%s", %i)''' %
(aStr, "U", AvatarID)
            AddData(DBFile, aDML)
            aQry = '''SELECT * FROM User
                WHERE Username = "%s"''' %aStr
            AnsTable = RunQuery(DBFile,aQry) #is auth proved they are there
            print(AnsTable)
            mbox.showinfo("User Created", "Welcome '%s'" %AnsTable[0][1])
        else:
            #get users details
            aQry = '''SELECT * FROM User
                WHERE Username = "%s"''' %aStr
            AnsTable = RunQuery(DBFile,aQry) #is auth proved they are there
            aDML = ('''UPDATE User SET AvatarID=%i WHERE Username="%s"'''%(AvatarID, aStr))
            AddData(DBFile, aDML)
            #get users details to test
            aQry = '''SELECT * FROM User
                WHERE Username = "%s"''' %aStr
            AnsTable = RunQuery(DBFile,aQry)
            print(AnsTable)
```

37.5.9. Test Plan – btnSave (White-Box testing)

No.	Description	Input	Expected Outcome	Actual Outcome	P/F
1	The Username input is not in the database.	AddUser("Test",3)	The username to be added to the database and clarification sent	(An error is output before the user is added to the database) To solve this I have duplicated the IsAuth function backwards and saved it as a NotAuth function.	F → P
2	The Username is already in the database.	IsAuth("Test", 5)	The Avatar for that username is updated no matter what.	 [[4, 'Test', 'U', 1]]	P
3	The Username input is not in the database.	AddUser("Test",3)	The username to be added to the database and clarification sent.	 [[4, 'Test', 'U', 3]]	P

37.5.10. Save event handler

```
def Save():  
    #event handler for btnSubmit  
    aStr = txtNUser.get()    #get entered username  
    AvatarID = pressed  
    edit = False  
  
    if UserPresence(aStr):    #check if there is already a user  
        edit = mbox.askyesno ("Edit", "Are you sure you like to edit this account?")  
        #print(edit) #test print  
  
    else:  
        edit = True  
  
    if edit:  
        if aStr != "Please enter a username":  
            AvatarID = pressed  
            AddUser(aStr, AvatarID)  
        else:  
            mbox.showerror("No Username Entered", "Error, This Username cannot be submitted")  
  
    return
```

The “pressed” is a parameter that I will use to declare which Avatar button has been clicked. Account usernames cannot be changed because that is currently the parameter for finding Users.

Maintenance: in future, I would get the UserID when entering the Manage Account GUI from options in order to allow for an account name to change. (pre-enter username). I would implement this in a later prototype due to time constraints.

The code below is used to identify “pressed” before it is used in the Save procedure above.

37.5.11. Avatar button selection

```
def which_button(button_press):  
    global pressed  
    if button_press == 1:  
        #change view of button 1  
        original_image = Image.open("Avatar1-dark70.png")  
        # Scale the image (change width and height as needed)  
        width, height = 45, 45 # Desired dimensions  
        scaled_image = original_image.resize((width, height), Image.LANCZOS)  
        # Convert the scaled image to a Tkinter-compatible image  
        tk_image = ImageTk.PhotoImage(scaled_image)  
        imgAvatar1.config(image=tk_image)  
        imgAvatar1.image = tk_image  
  
        #maintain other button view  
        original_image = Image.open("Avatar02.png")  
        # Scale the image (change width and height as needed)  
        width, height = 45, 45 # Desired dimensions  
        scaled_image = original_image.resize((width, height), Image.LANCZOS)  
        # Convert the scaled image to a Tkinter-compatible image  
        tk_image = ImageTk.PhotoImage(scaled_image)
```

```

imgAvatar2.config(image=tk_image)
imgAvatar2.image = tk_image

#maintain other button view
original_image = Image.open("Avatar03.png")
# Scale the image (change width and height as needed)
width, height = 45, 45 # Desired dimensions
scaled_image = original_image.resize((width, height), Image.LANCZOS)
# Convert the scaled image to a Tkinter-compatible image
tk_image = ImageTk.PhotoImage(scaled_image)
imgAvatar3.config(image=tk_image)
imgAvatar3.image = tk_image

#maintain other button view
original_image = Image.open("Avatar04.png")
# Scale the image (change width and height as needed)
width, height = 45, 45 # Desired dimensions
scaled_image = original_image.resize((width, height), Image.LANCZOS)
# Convert the scaled image to a Tkinter-compatible image
tk_image = ImageTk.PhotoImage(scaled_image)
imgAvatar4.config(image=tk_image)
imgAvatar4.image = tk_image
##      print("Avatar 1 selected")      #test print
pressed = 1
return 1

elif button_press == 2:
#change view of button 2
original_image = Image.open("Avatar01.png")
# Scale the image (change width and height as needed)
width, height = 45, 45 # Desired dimensions
scaled_image = original_image.resize((width, height), Image.LANCZOS)
# Convert the scaled image to a Tkinter-compatible image
tk_image = ImageTk.PhotoImage(scaled_image)
imgAvatar1.config(image=tk_image)
imgAvatar1.image = tk_image

#maintain other button view
original_image = Image.open("Avatar2-dark70.png")
# Scale the image (change width and height as needed)
width, height = 45, 45 # Desired dimensions
scaled_image = original_image.resize((width, height), Image.LANCZOS)
# Convert the scaled image to a Tkinter-compatible image
tk_image = ImageTk.PhotoImage(scaled_image)
imgAvatar2.config(image=tk_image)
imgAvatar2.image = tk_image

#maintain other button view
original_image = Image.open("Avatar03.png")
# Scale the image (change width and height as needed)
width, height = 45, 45 # Desired dimensions
scaled_image = original_image.resize((width, height), Image.LANCZOS)
# Convert the scaled image to a Tkinter-compatible image
tk_image = ImageTk.PhotoImage(scaled_image)
imgAvatar3.config(image=tk_image)
imgAvatar3.image = tk_image

```

```

        #maintain other button view
        original_image = Image.open("Avatar04.png")
        # Scale the image (change width and height as needed)
        width, height = 45, 45 # Desired dimensions
        scaled_image = original_image.resize((width, height), Image.LANCZOS)
        # Convert the scaled image to a Tkinter-compatible image
        tk_image = ImageTk.PhotoImage(scaled_image)
        imgAvatar4.config(image=tk_image)
        imgAvatar4.image = tk_image
##        print("Avatar 2 selected")    #test print
        pressed = 2
        return 2

elif button_press == 3:
    #change view of button 3
    original_image = Image.open("Avatar01.png")
    # Scale the image (change width and height as needed)
    width, height = 45, 45 # Desired dimensions
    scaled_image = original_image.resize((width, height), Image.LANCZOS)
    # Convert the scaled image to a Tkinter-compatible image
    tk_image = ImageTk.PhotoImage(scaled_image)
    imgAvatar1.config(image=tk_image)
    imgAvatar1.image = tk_image

    #maintain other button view
    original_image = Image.open("Avatar02.png")
    # Scale the image (change width and height as needed)
    width, height = 45, 45 # Desired dimensions
    scaled_image = original_image.resize((width, height), Image.LANCZOS)
    # Convert the scaled image to a Tkinter-compatible image
    tk_image = ImageTk.PhotoImage(scaled_image)
    imgAvatar2.config(image=tk_image)
    imgAvatar2.image = tk_image

    #maintain other button view
    original_image = Image.open("Avatar3-dark70.png")
    # Scale the image (change width and height as needed)
    width, height = 45, 45 # Desired dimensions
    scaled_image = original_image.resize((width, height), Image.LANCZOS)
    # Convert the scaled image to a Tkinter-compatible image
    tk_image = ImageTk.PhotoImage(scaled_image)
    imgAvatar3.config(image=tk_image)
    imgAvatar3.image = tk_image

    #maintain other button view
    original_image = Image.open("Avatar04.png")
    # Scale the image (change width and height as needed)
    width, height = 45, 45 # Desired dimensions
    scaled_image = original_image.resize((width, height), Image.LANCZOS)
    # Convert the scaled image to a Tkinter-compatible image
    tk_image = ImageTk.PhotoImage(scaled_image)
    imgAvatar4.config(image=tk_image)
    imgAvatar4.image = tk_image
##        print("Avatar 3 selected")    #test print
        pressed = 3

```

```

        return 3

    elif button_press == 4:
        #change view of button 4
        original_image = Image.open("Avatar01.png")
        # Scale the image (change width and height as needed)
        width, height = 45, 45 # Desired dimensions
        scaled_image = original_image.resize((width, height), Image.LANCZOS)
        # Convert the scaled image to a Tkinter-compatible image
        tk_image = ImageTk.PhotoImage(scaled_image)
        imgAvatar1.config(image=tk_image)
        imgAvatar1.image = tk_image

        #maintain other button view
        original_image = Image.open("Avatar02.png")
        # Scale the image (change width and height as needed)
        width, height = 45, 45 # Desired dimensions
        scaled_image = original_image.resize((width, height), Image.LANCZOS)
        # Convert the scaled image to a Tkinter-compatible image
        tk_image = ImageTk.PhotoImage(scaled_image)
        imgAvatar2.config(image=tk_image)
        imgAvatar2.image = tk_image

        #maintain other button view
        original_image = Image.open("Avatar03.png")
        # Scale the image (change width and height as needed)
        width, height = 45, 45 # Desired dimensions
        scaled_image = original_image.resize((width, height), Image.LANCZOS)
        # Convert the scaled image to a Tkinter-compatible image
        tk_image = ImageTk.PhotoImage(scaled_image)
        imgAvatar3.config(image=tk_image)
        imgAvatar3.image = tk_image

        #maintain other button view
        original_image = Image.open("Avatar4-dark70.png")
        # Scale the image (change width and height as needed)
        width, height = 45, 45 # Desired dimensions
        scaled_image = original_image.resize((width, height), Image.LANCZOS)
        # Convert the scaled image to a Tkinter-compatible image
        tk_image = ImageTk.PhotoImage(scaled_image)
        imgAvatar4.config(image=tk_image)
        imgAvatar4.image = tk_image
    ##        print("Avatar 4 selected")    #test print
        pressed = 4
        return 4

    else:
        pressed = 1
        return 1
        #default value

```

I have used the same code for all the buttons, just with different chosen pictures including a pre-darkened version to represent the image that the user has selected (all the other images are reset to their normal lighter colour scheme to show they haven't been selected).

Maintenance (achieved within first prototype):in future, I would only reset the previously selected image and currently selected image for efficiency purposes.

37.5.12. Testing “Pressed”

No.	Description	Input	Expected Outcome	Actual Outcome	P/F
1	Default when manage account GUI is opened.	Opening Manage Account GUI	Pressed = 1	Avatar:    	P
2	The Second Avatar is clicked	Avatar 2 click	Pressed = 2	Avatar:    	P
3	The Third Avatar is clicked	Avatar 3 is clicked	Pressed = 3	Avatar:    	P
4	The Fourth Avatar is clicked	Avatar 4 is clicked	Pressed = 4	Avatar:    	P
5	The First Avatar is clicked	Avatar 1 is clicked	Pressed = 1	Avatar:    	P

After testing, to make the code more efficient, I decided to only load and change the images that had been changed by the user selecting them and causing them to go darker or by the user selecting another image and causing the original selection to go lighter. First, I created a set code to change the original “pressed” back to its original colour scheme then, Second, I changed the currently selected button to the “selected” darker version of picture. Below is the more efficient code:

```
#manage account
def which_button(button_press):
    global pressed

    #for previous button
    #maintain other button view
    Imagereset = ("Avatar0%i.png"%pressed)
    original_image = Image.open(Imagereset)
    # Scale the image (change width and height as needed)
    width, height = 45, 45 # Desired dimensions
    scaled_image = original_image.resize((width, height), Image.LANCZOS)
    # Convert the scaled image to a Tkinter-compatible image
    tk_image = ImageTk.PhotoImage(scaled_image)
    if pressed == 1: #declare the image for the correct previously selected
avatar
        imgAvatar1.config(image=tk_image)
```

```

        imgAvatar1.image = tk_image
    elif pressed == 2:
        imgAvatar2.config(image=tk_image)
        imgAvatar2.image = tk_image
    elif pressed == 3:
        imgAvatar3.config(image=tk_image)
        imgAvatar3.image = tk_image
    elif pressed == 4:
        imgAvatar4.config(image=tk_image)
        imgAvatar4.image = tk_image

    #change view of selected button
    imagedarker = ("Avatar%i-dark70.png"%button_press)
    original_image = Image.open(imagedarker)
    # Scale the image (change width and height as needed)
    width, height = 45, 45 # Desired dimensions
    scaled_image = original_image.resize((width, height), Image.LANCZOS)
    # Convert the scaled image to a Tkinter-compatible image
    tk_image = ImageTk.PhotoImage(scaled_image)

    if button_press== 1: #set to a specific button
        imgAvatar1.config(image=tk_image)
        imgAvatar1.image = tk_image
        pressed=1
    ##      print(pressed) #test print
    return 1 #return the currently selected
    elif button_press == 2:
        imgAvatar2.config(image=tk_image)
        imgAvatar2.image = tk_image
        pressed=2
    ##      print(pressed) #test print
    return 2
    elif button_press == 3:
        imgAvatar3.config(image=tk_image)
        imgAvatar3.image = tk_image
        pressed=3
    ##print(pressed) #test print
    return 3
    elif button_press == 4:
        imgAvatar4.config(image=tk_image)
        imgAvatar4.image = tk_image
        pressed=4
    ##      print(pressed) #test print
    return 4

```

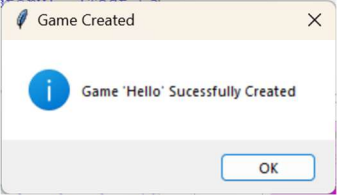
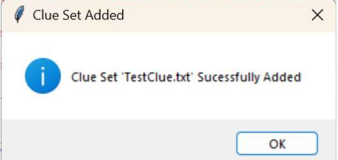
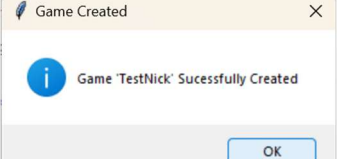
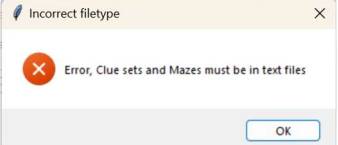
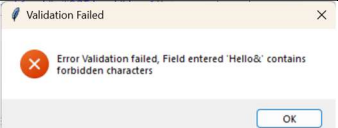
This shortened code only updates necessary image buttons instead of all of them. (works in the same way as previously but is more efficient and a lot shorter than the previous version).

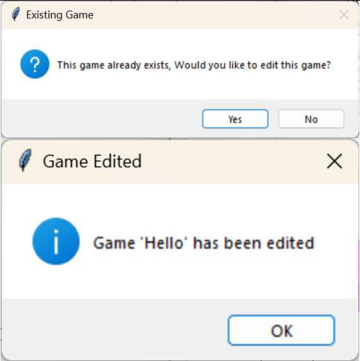
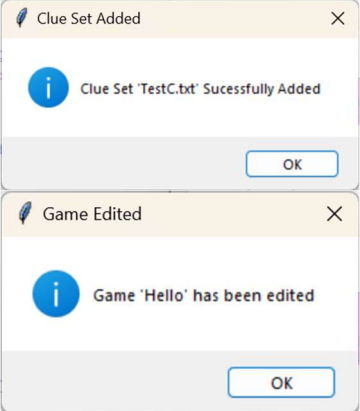
37.6. **Admin Management**

To create a new game, an Admin User must enter a Display Name, a Clue Set Filename and a Maze Filename as strings:

- a. Valid Characters are 'A' – 'Z', 'a' – 'z', '0' – '9' and selected special characters
- b. Validation:
 - i. Presence check (something entered)
 - ii. Length Check – 5 – 20 characters inclusive for Display name
 - iii. Type check – valid char type
 - iv. Maintenance: I would consider sanitisation to prevent SQL injection opportunities in a future prototype.
- c. Authentication: once validated the username must be authenticated to check if the game is being edited or not.
- d. A success message will be displayed specific to the game if it is successful added to the database
- e. An error message will be displayed if validation fails.
- f. After a Game has successfully been added to the database, delete the data in the boxes.
- g. Ask if a User would like to edit before editing a game. – display a message saying that the game is already in the database.
- h. If the "Cancel" button is clicked – the Admin Management GUI will be removed and the Options GUI will be displayed.

37.6.1. Test Plan(White box testing) – development of Admin Management event handler

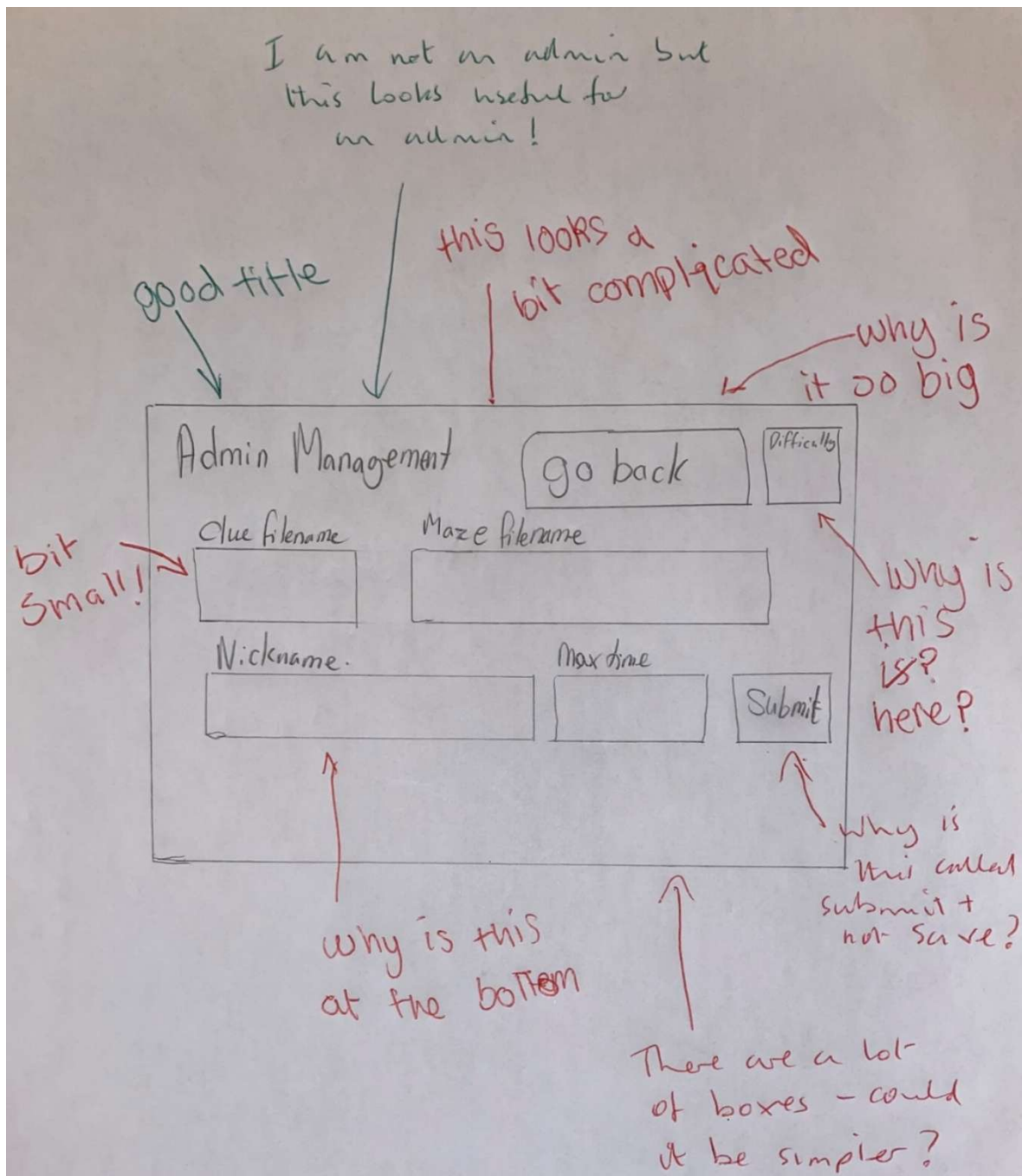
No.	Description	Input	Expected Outcome	Actual Outcome	P/F
1	Successful: Add a game to the database with a clueset already in the database.	Nickname = Hello ClueSet = Cyber.txt Maze = Maze1.txt MaxTime=00:15:00 Difficulty = Medium	Message “Game Hello Created”		P
2	Successful: Add a game to the database with a clueset not already in the database.	Nickname = TestNick ClueSet = TestClue.txt Maze = Maze1.txt MaxTime=00:15:00 Difficulty = Medium	Message “Clue Set TestClue.txt added” Message “Game TestNick Created”	 	P
3	Invalid: Filenames of incorrect filetype	ClueSetFilename = Hi	Message : “ERROR. Incorrect Filetypes”		P
4	Invalid: Any Empty fields including drop downs	Display Name = “” MaxTime = “Max Time...”	Message : “ERROR. All fields are required”		P
5	Invalid: Failed type check(forbidden chars in a correct length username)	Display Name = “Hello&”	Message : “ERROR. Failed validation”		P
6	Invalid: Display name too short/long	Display Name = “Hi” Display Name = “aishgaoilghaodighe9fg egae”	Message : “ERROR. Failed validation”		P

4	Successful: edit a game to the database with a clueset already in the database.	Nickname = Hello ClueSet = Cyber.txt Maze = Maze1.txt MaxTime=00:20:00 Difficulty = Hard	Message “Game Hello edited”		P
4	Successful: edit a game to the database with a clueset already in the database.	Nickname = Hello ClueSet = TestC.txt Maze = Maze1.txt MaxTime=00:20:00 Difficulty = Hard	Message “Game Hello edited”		P

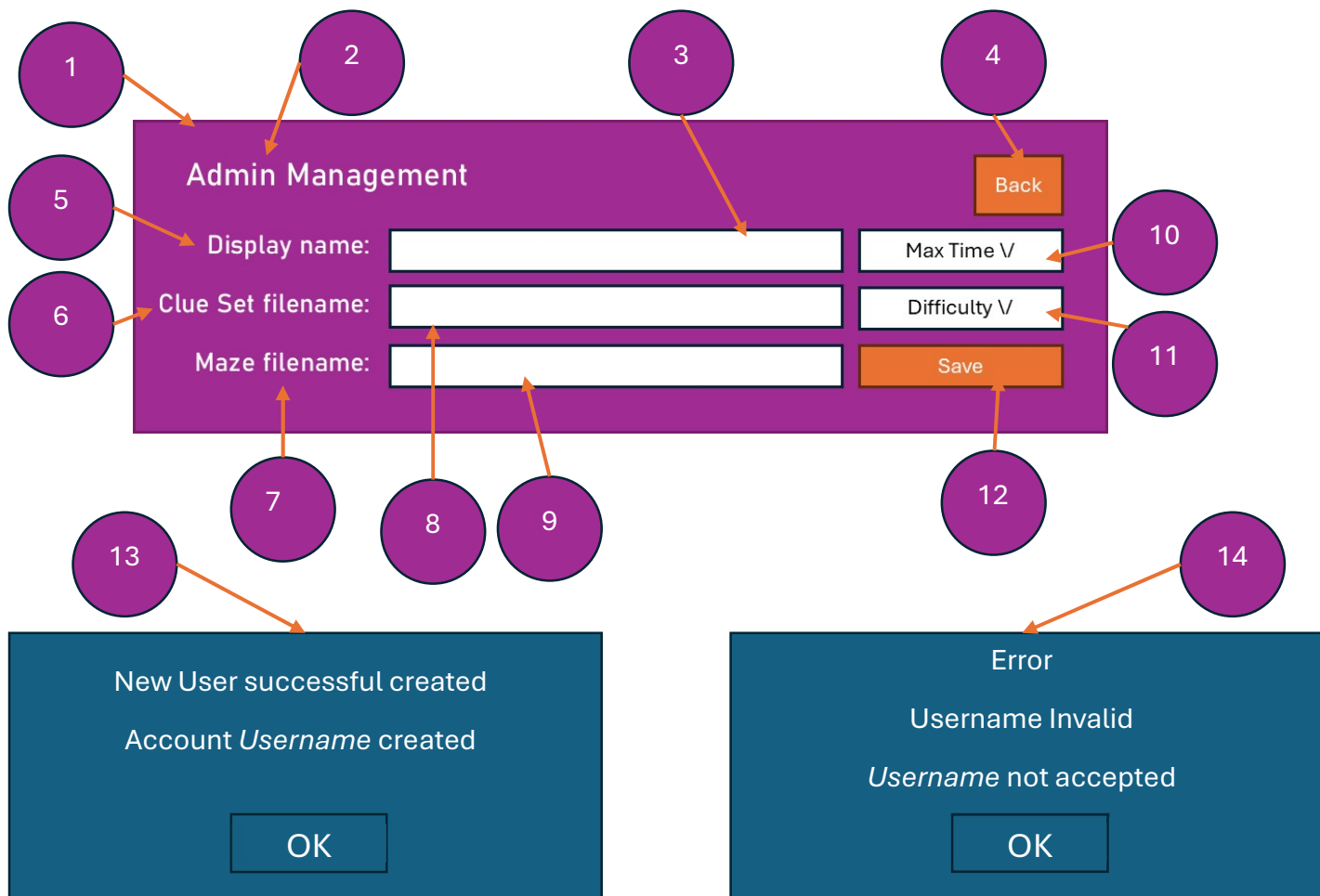
After developing my Admin Management GUI, I decided to give more detail to the users of what error had occurred when signing in for usability and ease of access. Maintenance (achieved within first prototype): I had the idea of entering some pre-filled text into the filename fields with the text files, so I added this to my code.

37.6.2. Design and Development

I started with a hand drawing of my GUI design for Admin Management for the stakeholders to comment on. The image below shows my stakeholder comments which included positive feedback and things to consider changing. I had my sixth form stakeholders comment on this even though they wouldn't be admin and told them to think like a teacher. This was because my teacher stakeholder was unavailable and I wanted to keep making progress. I later checked this with my teacher stakeholder and they were happy with the comments.



After discussing this with my stakeholders, I have realised that I need to adjust my initial design to include more consistent spacing which will make it look simpler. I need to have a save button rather than a submit button and have a clearer input box arrangement.



ID	Name	Widget	Justification
1	frmAdmin	Frame(container)	All the other widgets are held within the frame. The widgets are then easy to move within the frame for best layout on the screen.
2	lblAdmin	Label	Read only title for this GUI so the user knows where they are at in the system.
3	txtNick	Entry	User input box for user to enter the Game Nickname – Entry used because any variety of Nicknames need to be entered via keyboard
4	btnBack	Button	A Button is a good option for the Admin to go back to the Options GUI.
5	lblNick	Label	Read only identifier for the type of data to input so the Admin knows what type of data to input.
6	lblClue	Label	Read only identifier for the type of data to input so the Admin knows what type of data to input.
7	lblMaze	Label	Read only identifier for the type of data to input so the Admin knows what type of data to input.
8	txtClue	Entry	User input box for user to enter the Clue Set filename– Entry used because any variety of Nicknames need to be entered via keyboard

9	txtMaze	Entry	User input box for user to enter the Maze filename– Entry used because any variety of Nicknames need to be entered via keyboard
10	clbMaxTime	ComboBox	A drop-down menu for game selection meaning the Admin can only choose from a selection of max times meaning they are not extreme and are always in the correct format – The times are ordered chronologically.
11	clbDifficulty	ComboBox	A drop-down menu for game difficulty meaning the Admin can only choose one of the available difficulties and they are connected to be added to the database correctly.
12	btnSave	Button	A Button is a good option for the Admin to save their Game details to the database or amend details already there.

37.6.3.Admin Management GUI Development

```
def Admin():
    global frmAdmin, txtNick, txtMaze, txtClue, clbMaxTime, clbDifficulty

    frmAdmin = tk.Frame(root, bg = BackCol)
    frmAdmin.place(x = 655, y = 220, width = 645, height = 207)

    lblAdmin = tk.Label(frmAdmin, text = "Admin Management",
                        bg = BackCol, fg = TextCol,
                        anchor="w", font=('Bahnschrift', 16, 'bold'))
    lblAdmin.place(x = 25, y = 15, width = 201, height = 36)

    lblNick = tk.Label(frmAdmin, text = "Display Name:",
                      bg = BackCol, fg = TextCol,
                      anchor="e", font=('Bahnschrift', 12, 'bold'))
    lblNick.place(x = 24, y = 60, width = 136, height = 31)

    txtNick = tk.Entry(frmAdmin, text = "",
                      bg = TextCol, fg = "black", borderwidth=2, font=('Bahnschrift',
12))
    txtNick.place(x = 170, y = 67, width = 300, height = 28)

    lblClue = tk.Label(frmAdmin, text = "Clue Set filename:",
                      bg = BackCol, fg = TextCol,
                      anchor="e", font=('Bahnschrift', 12, 'bold'))
    lblClue.place(x = 24, y = 103, width = 136, height = 31)

    txtClue = tk.Entry(frmAdmin, text = "",
                      bg = TextCol, fg = "black", borderwidth=2, font=('Bahnschrift',
12))
    txtClue.place(x = 170, y = 105, width = 300, height = 28)

    lblMaze = tk.Label(frmAdmin, text = "Maze filename:",
                      bg = BackCol, fg = TextCol,
                      anchor="e", font=('Bahnschrift', 12, 'bold'))
    lblMaze.place(x = 24, y = 145, width = 136, height = 31)

    txtMaze = tk.Entry(frmAdmin, text = "",
                      bg = TextCol, fg = "black", borderwidth=2, font=('Bahnschrift',
12))
    txtMaze.place(x = 170, y = 140, width = 300, height = 28)

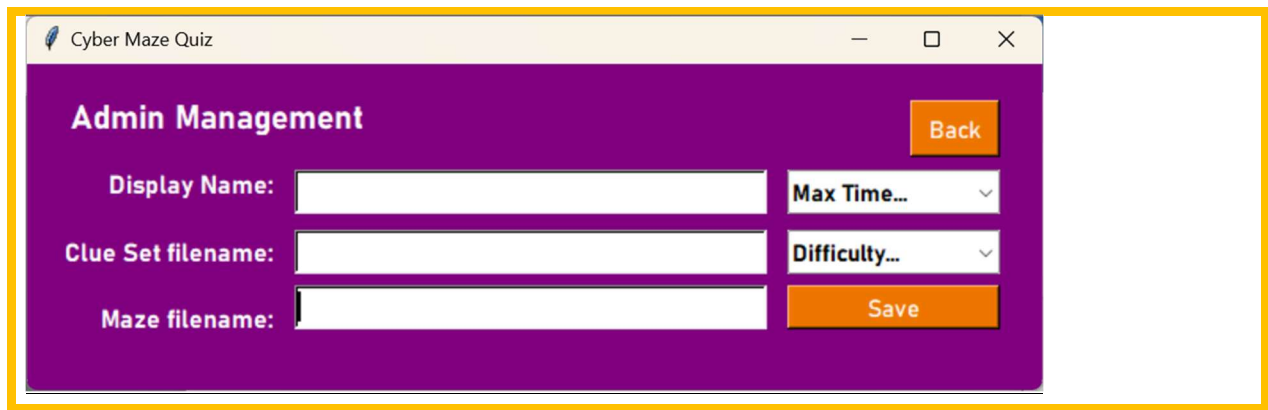
    btnSave = tk.Button(frmAdmin, text = "Save",
                       bg = BtnCol, fg = TextCol, font=('Bahnschrift', 11) )
    btnSave.place(x = 483, y = 140, width = 135, height = 28)

    clbTimeValues = ["00:02:00"], ["00:05:00"], ["00:10:00"], ["00:15:00"], ["00:20:00"],
["00:25:00"], ["00:30:00"]
    clbMaxTime = ttk.Combobox(frmAdmin, values = clbTimeValues, state= "readonly",
                             font=('Bahnschrift', 11, 'bold'))
    clbMaxTime.set('Max Time...')
    clbMaxTime.place(x = 483, y = 67, width = 135, height = 28)

    clbLValue = ["Easy"], ["Medium"], ["Hard"]
    clbDifficulty = ttk.Combobox(frmAdmin, values = clbLValue, state= "readonly",
                                font=('Bahnschrift', 11, 'bold'))
    clbDifficulty.set('Difficulty...')
    clbDifficulty.place(x = 483, y = 105, width = 135, height = 28)

    btnOptions = tk.Button(frmAdmin, text = "Back",
                          bg = BtnCol, fg = TextCol, font=('Bahnschrift', 11) )
    btnOptions.place(x = 561, y = 23, width = 57, height = 36)

    return
```



37.6.4. Evaluation

My first attempt at creating the Admin Management GUI follows the design outlined above and uses aspects of the previously developed GUIs. Corrections needed:

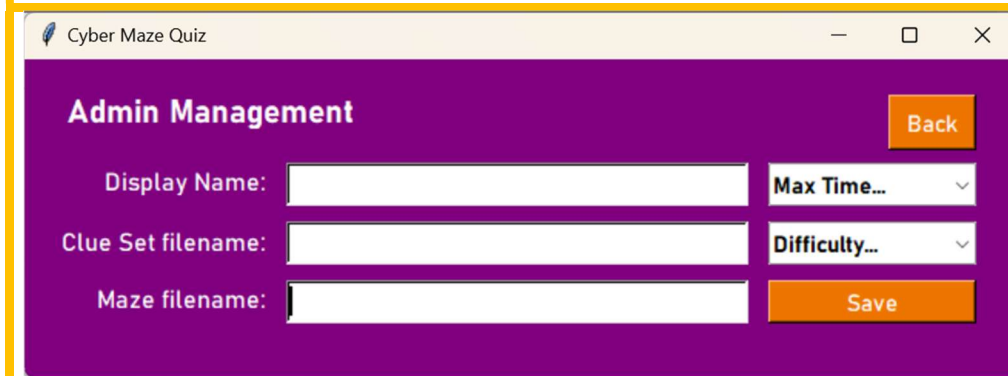
- Change the positions of some of the widgets according to my design.
- Remove the bold on the identifier labels.

37.6.5. Corrections

```
lblNick = tk.Label(frmAdmin, text = "Display Name:",
                    bg = BackCol, fg = TextCol,
                    anchor="e", font=('Bahnschrift', 12))
lblNick.place(x = 24, y = 63, width = 136, height = 31)

lblClue = tk.Label(frmAdmin, text = "Clue Set filename:",
                    bg = BackCol, fg = TextCol,
                    anchor="e", font=('Bahnschrift', 12))
lblClue.place(x = 24, y = 102, width = 136, height = 31)

lblMaze = tk.Label(frmAdmin, text = "Maze filename:",
                    bg = BackCol, fg = TextCol,
                    anchor="e", font=('Bahnschrift', 12))
lblMaze.place(x = 24, y = 139, width = 136, height = 31)
txtMaze.place(x = 170, y = 143, width = 300, height = 28)
btnSave.place(x = 483, y = 143, width = 135, height = 28)
```



37.6.6.Event Handling

An event is when something occurs that is expected but it is not known when it will happen, for example the user clicking a button. There are 2 buttons to consider in this frame:

btnSave

```
when button save is clicked,  
    get values from txtNick, txtClue, txtMaze, clbMaxTime and clbDifficulty.  
    Check if the values are not NULL  
    If not NULL  
        Check all the inputs are valid  
        Find the ClueID for the Clue set filename in the database  
        Add all the information to the database Maze table  
    Display message box "Game successfully added to the database"
```

btnBack

```
when button save is clicked,  
    go to the Options GUI
```

37.6.7.Coding the event handlers

I have used the basis of the Validation function that I used in the Login GUI to create a validation function for the text inputs.

BtnSave

```
Procedure AddGame          //procedure to add a game to the database  
    Display = Get input from txtNick  
    Clue = Get input from txtClue  
    Maze = Get input from txtMaze  
    Time = Get input from clbMaxTime  
    Level = Get input from clbDifficulty  
    If Display, Clue, Maze, Time, Level != NULL then          //checks for empty fields  
        If Clue, Maze(-4) == '.txt' then //makes sure that txt files have been entered  
            ManGame(Display, Clue, Maze, Time, Level) //link to function in callbacks  
        Else then  
            Show error (Incorrect filetype)  
        Endif  
    Else then  
        Show error(Invalid Input)  
    Endif  
    Return  
End procedure
```

```

//function in callbacks
Function ManGame (Display, Clue, Maze, Time, Level)           //to add the values to the database
    Valid = False

    If Difficulty == Hard then           //to get difficulty into the correct format
        Difficulty = 3
    Else if Difficulty == Medium then
        Difficulty = 2
    Else then
        Difficulty = 1

    If ValidInp(Clue) then
        Print("Clue Set is valid")
    If ValidInp(Maze) then
        Print("Maze Filename is valid")
    If ValidInp(Display) then
        Print("Display name is valid")
        Valid = True    //if statement control
    Else then
        Print("not valid")

    If Valid then
        If InMaze(NickName) //to check if the game is already in the maze
            action = message box(Do you want to edit this existing game?)
            if action then           -I added this after a chat with my teacher Stakeholder
                Table = AddClues(Clue)           //to check if the clueset is already in the
database if it is not it will add the clueset to the database.
                'UPDATE Maze SET MazeFileName = %s, Difficulty = %i, MaxTime = %s, ClueID =
%i'%(MFilename, Difficulty, MaxTime, Table, NickName)

```

37.7. Leaderboards

My sixth form and teacher stakeholders will want to view performance statistics ranked in a leaderboard. There are several performance stats I could use for each individual maze leaderboard, as follows:

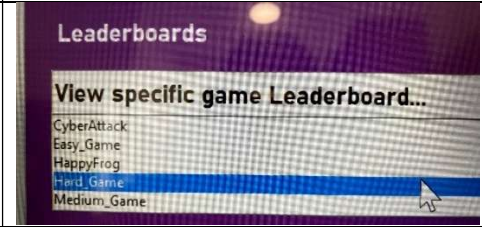
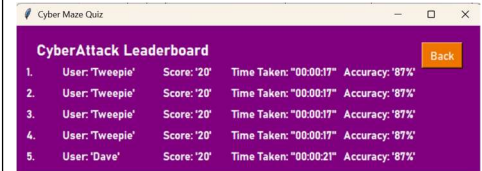
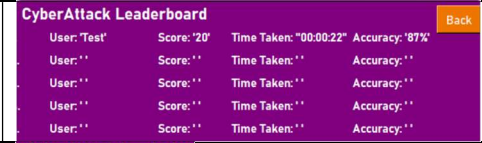
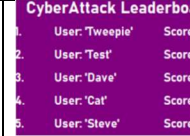
- Score
- Fastest Time taken (completed games only)
- Accuracy
- Number of each question answered correctly per difficulty level.

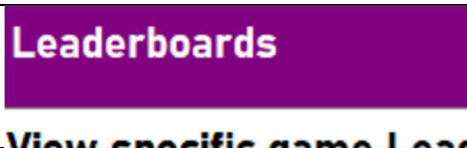
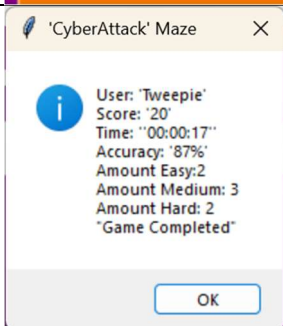
Future Development Idea: It might be a good future idea to be able to track performance over multiple games to measure/visualise improvement.

For the Leaderboards, my development criteria are as follows:

- a. The User must select a Maze game from a list of mazes they have played and completed.
- b. Admin Users can select a maze from all the mazes available, whether completed or not (this is because Admin users can only monitor, they cannot play the games).
- c. Only the top 5 results for Score must be displayed for a user (less than 5 results for a user means that the display will be blank).
- d. Admin users may need to view all players results ranked by performance.
- e. Maintenance: Currently, an admin user can view all players who have ever played a specific game so, this view could run into hundreds. In future, I could add a filter for the Admin to select a date range for when the game was played. This will make the view more manageable. I cannot achieve this for this prototype due to time constraints.
- f. Viewing a leaderboard must be selectable from the Options GUI
- g. A “Back” button will return the user to the Options GUI.

37.7.1. **Test Plan**(White box testing) – development of Leaderboard Selection event handler

No.	Description	Input	Expected Outcome	Actual Outcome	P/F
1	The User must select a Maze game from a list of mazes they have played and completed	Combo List box with game Nickname.	List with games played by the user.	List of all available games. (I would like Admin Users and Generic Users to use the same initial GUI).	F
2	Admin Users can select a maze from all the mazes available, whether completed or not	Combo List box with game Nickname.	Full list of all available games		P
3	Only the top 5 results for Score must be displayed for a user (less than 5 results = display all + blank).	5 User_Maze records	List of all five results in order	ERROR – One of my user's results has been duplicated multiple times 	F->P
		Less than 5 User_Maze Records	1 result with 4 blank rows		P
		More than 5 User_Maze Records	List of the top 5 (excluding anyone below)	 The record below is not on the leaderboard. <pre>[(10, 10, "00:00:20", 2, 2, 1, '63%', 7, 1, 1)]</pre>	P

4	Admin users may need to view all players results ranked by performance.	View list of users on the Admin user drop down.	The drop-down contains all users who have played the game in performance order.		P
5	Viewing a leaderboard must be selectable from the Options GUI	Leaderboards are selected from the Options GUI.	The Options GUI is replaced by the Leaderboard GUI.		P
6	A “Back” button will return to the Options GUI.	“Back” is selected on the Leaderboard GUI.	The Leaderboard GUI is replaced with the Options GUI.		P
7	Pop-up after a User is selected in the Admin Leaderboard with Stats	A User in the Admin Leaderboard drop down	A pop up with the most recent stats for that user for that game		P

Future Development Idea: It would be good to only replace the result if it is greater than the current result. This will allow for a more competitive leaderboard and the best stats to be seen in the Admin view for each user.

Error Correction Following Testing (Failed Tests)

*ERROR 1

I decided to change how I wished to develop the Leaderboard GUI, so I knew this test would fail. I want all users to be able to view all of the leaderboards, irrespective of whether they have played the games in a given leaderboard. This also means that all account types can use the same Leaderboard game selection rather than developing a separate algorithm for the values within the combo list box in the Leaderboard GUI. By offering visibility of all leaderboards, I have been able to reduce the processing required in this GUI.

*ERROR 3(a). Below is the identification of the problem and the fix for the error which duplicated the results of one user multiple times:

```
>>> RunQuery(DBFile, "SELECT * FROM User_Maze WHERE UserID = 2")
[]
>>> RunQuery(DBFile, "SELECT * FROM User_Maze WHERE UserID = 3")
[(1, 20, "'00:00:17'", 2, 3, 2, '87%', 3, 1, 1), (2, 20, "'00:00:17'", 2, 3, 2, '87%', 3, 1, 1), (3, 20, "'00:00:17'", 2, 3, 2, '87%', 3, 1, 1), (4, 20, "'00:00:17'", 2, 3, 2, '87%', 3, 1, 1)]
>>> RunQuery(DBFile, "SELECT * FROM User_Maze WHERE UserID = 1")
[(8, 20, "'00:00:16'", 2, 3, 2, '87%', 1, 1, 1)]
>>> RunQuery(DBFile, "SELECT * FROM User_Maze WHERE UserID = 4")
[(5, 20, "'00:00:21'", 2, 3, 2, '87%', 4, 1, 1)]
>>> RunQuery(DBFile, "SELECT * FROM User_Maze WHERE UserID = 5")
[(6, 14, "'00:00:19'", 1, 2, 2, '62%', 5, 1, 1)]
>>> RunQuery(DBFile, "SELECT * FROM User_Maze WHERE UserID = 6")
[(7, 14, "'00:00:16'", 2, 3, 1, '73%', 6, 1, 1)]
>>> RunQuery(DBFile, "SELECT * FROM User_Maze WHERE UserID = 7")
[]
else:
    if len(GameUsers) == 1:
        aDML= '''UPDATE User_Maze SET Score = %i, TimeTaken = "%s", NumEasy = %i, NumMedium = %i,
                NumHard = %i, Accuracy = '%s', UserID = %i, MazeID = %i, COMP = %d WHERE MazeID = '%s' AND UserID = '%s' ''' %(Score, TimeTaken, NumEasy,
        AddData(DBFile, aDML) #append record
        ## print("Update UserMaze Added") #test print
    else:
        aDML= '''DELETE FROM User_Maze WHERE MazeID = '%s' AND UserID = '%s' ''' %(MazeID, UserID)
        AddData(DBFile, aDML) #delete duplicate records
        aDML= '''INSERT INTO User_Maze(Score, TimeTaken, NumEasy, NumMedium, NumHard, Accuracy, UserID, MazeID, COMP)
                VALUES(%i, "%s", %i, %i, %i, "%s", %i, %i, %d)''' %(Score, TimeTaken, NumEasy, NumMed, NumHard, Accuracy, UserID, MazeID, Comp)
        AddData(DBFile, aDML) #new record
```

It should not be possible to have more than 1 record for a single maze but here there are 4 identical duplicates.

Therefore the error must be in Save_to_DB()

I have changed it to specify which record to update and then to update that single record. (When it failed the test, it was updating all of the records in User_Maze)

I realised after solving Error 3A above that users with the same score would be ordered alphabetically by name so I changed this so that they would be ordered by time as a secondary value, as follows:

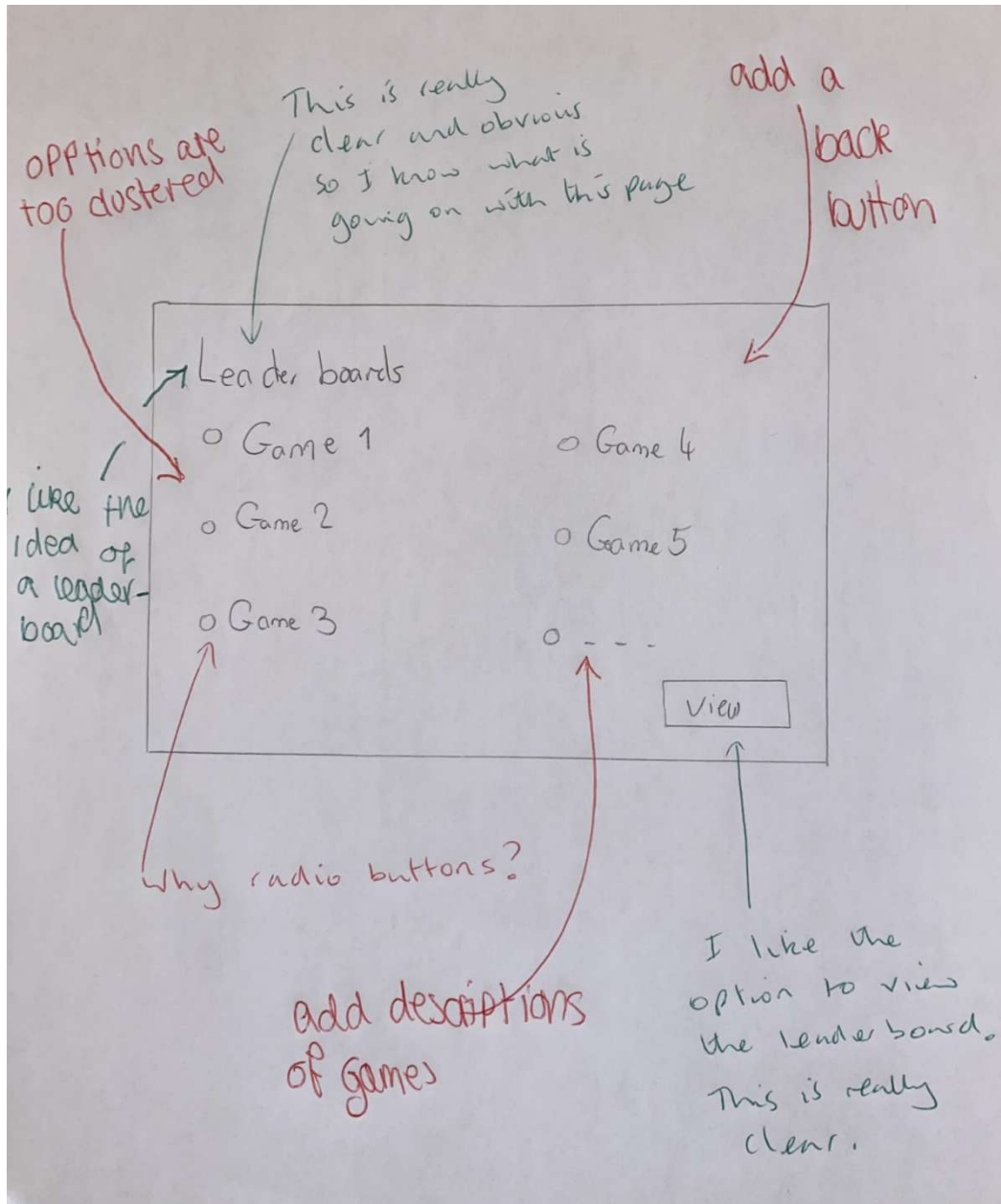
```
#Order these scores into the top 5.  
#ref to order by Score(with time as a back up)  
#https://www.geeksforgeeks.org/python-sort-list-of-list-by-specified-index/51  
#https://www.geeksforgeeks.org/sort-a-list-of-objects-by-multiple-attributes-in-python/52  
#  
UserMazeorder =sorted(UserMazeorder, key = lambda x: (-x[1], x[2]))
```

I realised that previously I had my Admin lists ordered alphabetically so I have used the method that I have developed above, to allow the usernames to be ordered by completion (and score as a secondary value), as shown below:

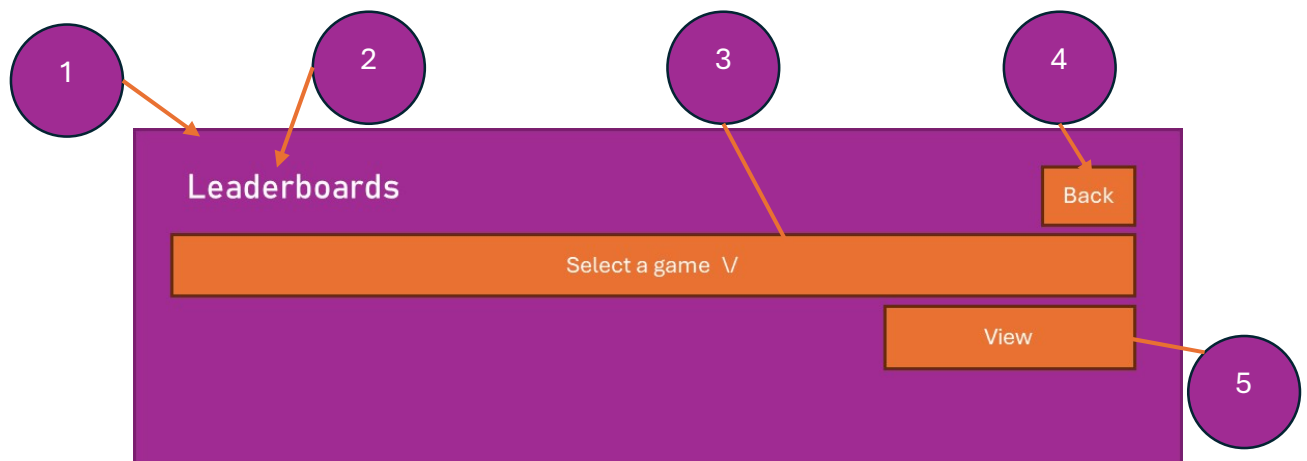
```
Completed      Same Completed  
    UserDetails.append((User, Scr, TT, Acc, Easy, Med, Hard, Completed))  
    count = count + 1  
#    print(UserDetails)  #test print  
    UserDetails =sorted(UserDetails, key = lambda x: (x[7],-x[1]))  
  
    return UserDetails
```

37.7.2. Design and Development

I started with a hand drawing of my GUI design for Leaderboard Selection for the stakeholders to comment on. The image below shows my stakeholder comments which included feedback on positive things and what I might consider changing.



After discussing this with my stakeholders, I have realised that I need to adjust my initial design to include a “Back” button, to use a drop-down for game selection that that the GUI doesn’t get over cluttered and add nicknames to describe the games. I am pleased that they liked the clear functionality of the GUI and they thought the Title was good.



ID	Name	Widget	Justification
1	frmLeader	Frame(container)	All the other widgets are held within the frame. The widgets are then easy to move within the frame for best layout on the screen.
2	lblLeader	Label	Read only title for this GUI so the user knows where they are at in the system.
3	clbGames	ComboBox	A drop-down menu for game selection meaning the Stakeholder can only choose one of the available games and can't query something that doesn't exist.
4	btnBack	Button	A Button is a good option for the Stakeholder to go back to the Options GUI.
5	btnView	Button	A Button is a good option for the stakeholder to go to a GUI to view the selected game leaderboard.

37.7.3. Leaderboards GUI Development

```
def Leader():
    global frmLeader, Leader, clbGames

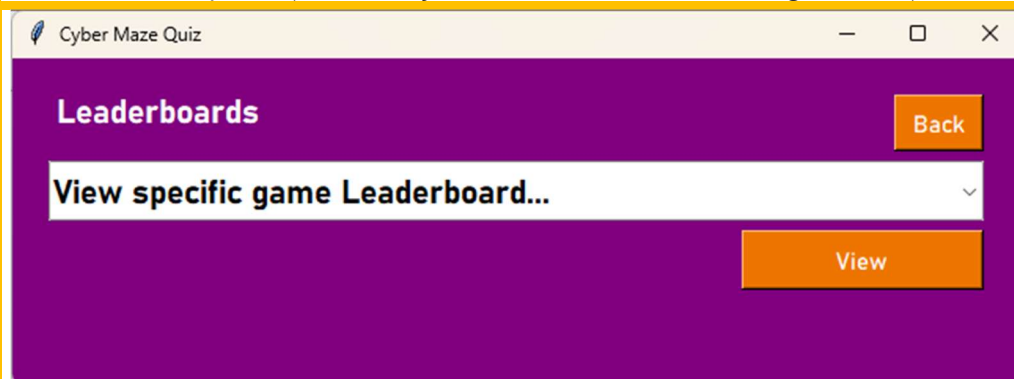
    frmLeader = tk.Frame(root, bg = BackCol)
    frmLeader.place(x = 655, y = 429, width = 645, height = 207)

    #same position as previous titles
    lblLeader = tk.Label(frmLeader, text = "Leaderboards",
                          bg = BackCol, fg = TextCol,
                          anchor="w", font=('Bahnschrift', 16, 'bold'))
    lblLeader.place(x = 25, y = 15, width = 201, height = 36)

    btnView = tk.Button(frmLeader, text = "View",
                        bg = BtnCol, fg = TextCol, font=('Bahnschrift',
11) )
    btnView.place(x = 464, y = 109, width = 154, height = 38)

    btnBack = tk.Button(frmLeader, text = "Back",
                        bg = BtnCol, fg = TextCol, font=('Bahnschrift',
11) )
    btnBack.place(x = 561, y = 23, width = 57, height = 36)

    Leader=["Temporary"],["Leaderboard"], ["Values"]
    clbGames = ttk.Combobox(frmLeader, values = Leader,
                            text = "View specific game leaderboards", state=
"readonly",
                            font=('Bahnschrift', 16, 'bold'))
    clbGames.set('View specific game Leaderboard...')
    clbGames.place(x = 23, y = 65, width = 595, height = 38)
```



37.7.4. Evaluation

My first attempt at creating the Leaderboards GUI follows the design outlined above and uses aspects of the previously developed GUIs (specifically the Play Game GUI). Due to this GUI being an almost exact duplicate of the Play Game GUI with just a few text changes. It doesn't need any corrections because the corrections were already implemented when I developed the Play Game GUI.

37.7.5.Event Handling

An event is when something occurs that is expected but it is not known when it will happen, for example the user clicking a button. There are 2 buttons to consider in this frame:

btnBack

When clicked,
go to the Options GUI

btnView

when clicked,
 check if User is admin if no
 take the string from clbGameSelect
 Find the Game in the database using its nickname.
 Check if the game is online and has all the required features (Maze file,
Clue file)
 Initialize the game and run it.
Else
 Message box to display “Admin users cannot play games”

37.7.6.Coding the event handlers

Expanding from the second algorithm above, below is the algorithm for coding the View Leaderboards Event Handler.

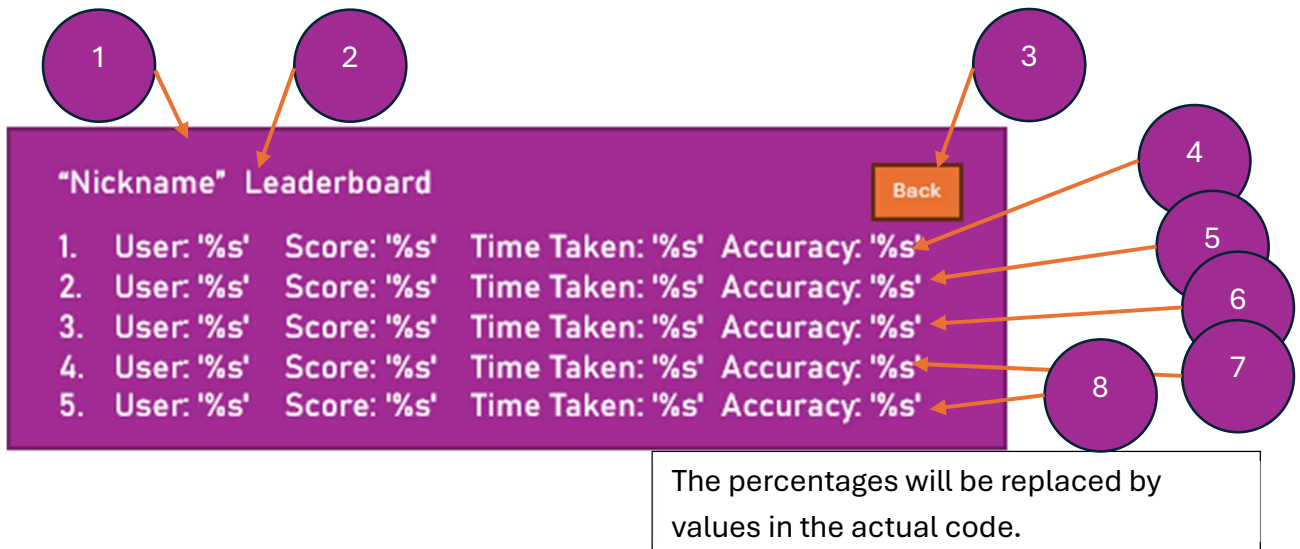
btnView

```
procedure ViewLeader
    NickName = clbGames.Value
if not Admin:
    Top5 = Top5Out(Nickname)           //get values for the game ordered by
score
    //all games must be completed //create frame for temporary gui
    frmLeaderboard.create           //same settings as other interfaces.
    frmLeader.move           //move Leader out of the way.
    //title
    Nickname = Nickname, " Leaderboard"
    lblTitle.create(Nickname)       //same settings as other interface titles
    //back button
    btnBack.create(Back) // same settings as other interface backs

    //display
    displaceY = 50
    For indx in range(0,5)           //top 5
        User, Score, Time, Acc = Top5[indx]
        //placeholders
        If User == "" then
            User = " "
        If Time == "" then
            Time = " : : "
        If Acc == "":
            Acc = " %"
        displaceY = displaceY +(29*indx)
        Stats = ("%i. User:'%s'"%(indx+1),User), "Score:%i  "%(Score),
        "Time Taken" '%s' "%(Time), "Accuracy:  '%s'"%(Acc))
        lbl+(indx).create(Stats)//similar style to rest of interfaces
        //implement by factors each time
        //Event Handlers
        btnBack['command'] = Temp_Back
        clbGames.set('View specific game Leaderboard...') //reset clb
    return
    //admin leaderboard will be here in future
end procedure
```

37.7.7.Coding the User Leaderboard

I have created a simple design for the User Leaderboard GUI within the Leaderboards GUI and this is based on my previous GUIs shown in the simple plan below:



ID	Name	Widget	Justification
1	frmLeaderboard	Frame(container)	All the other widgets are held within the frame. The widgets are then easy to move within the frame for best layout on the screen.
2	lblTitle	Label	Read only title for this GUI so the user knows where they are at in the system.
3	btnBack	Button	A Button is a good option for the Stakeholder to go back to the Options GUI.
4	lbl1	Label	First User label containing the user information with the highest score.
5	Lbl2	Label	Second User label containing the user information with the second highest score.
6	Lbl3	Label	Third User label containing the user information with the third highest score.
7	Lbl4	Label	Fourth User label containing the user information with the fourth highest score.
8	Lbl5	Label	Fifth User label containing the user information with the fifth highest score.

37.7.8. User Leaderboard GUI Development

```
def ViewLeader():
    NickName = clbGames.get()
    if not UserAdmin(Username):
        #call from function in callbacks
        Top5 = Top50Out(NickName)

        #create Leaderboard frame
        global frmLeaderboard
        frmLeaderboard = tk.Frame(root, bg = BackCol)
        frmLeaderboard.place(x = 0, y = 0, width = 645, height = 207)
        frmLeader.place(x = 0, y = 500, width = 645, height = 207)

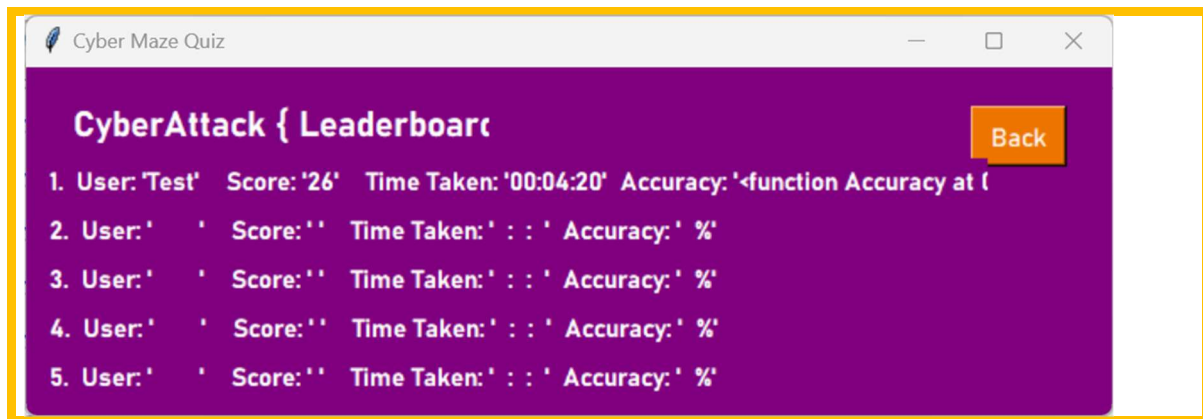
        #Title Label
        Nickname = NickName[:len(NickName)]
        print(Nickname)
        Nickname = str(Nickname), " Leaderboard"
        lblTitle = tk.Label(frmLeaderboard, text = (Nickname),
                             bg = BackCol, fg = TextCol,
                             anchor="w", font=('Bahnschrift', 16, 'bold'))
        lblTitle.place(x = 25, y = 15, width = 250, height = 36)

        #back Button
        btnBack = tk.Button(frmLeaderboard, text = "Back",
                             bg = BtnCol ,fg = TextCol,
font=('Bahnschrift', 11) )
        btnBack.place(x = 561, y = 23, width = 57, height = 36)

        #results
        displaceY = 25 #offset by 29 each time
        for count in range(0,5):
            print(Top5)
            print(Top5[count])
            User, Score, Time, Acc = Top5[count]
            if User == " ":
                User = "      "
            if Time == " ":
                Time = " : : "
            if Acc == " ":
                Acc = " %"
            displaceY = displaceY + 29
            Stats = "%i. User: '%s' Score: '%s' Time Taken: '%s'"
Accuracy: '%s' " %((count+1),User,Score,Time,Acc)
            label = "lbl%i"%(count)
            label = tk.Label(frmLeaderboard, text=(Stats),bg = BackCol,
                             fg = TextCol,anchor="w",
font=('Bahnschrift', 11, 'bold'))
            label.place(x = 11, y = displaceY, width = 560, height = 26)

        #Event Handlers
        btnBack['command'] = Temp_Back

        clbGames.set('View specific game Leaderboard...') #reset clb
        return
```



37.7.9.Evaluation

This is my first attempt at creating the User Leaderboard GUI and there are a few corrections that I will need to make:

- Fix the title so that “Leaderboard” is not in squiggly brackets
- Align the start of each of the stats so that it looks neat.
- Don’t allow any overlap of the back button

37.7.10. Corrections

To fix the title: I realise that I had accidentally made it a list instead of concatenating it, so I fixed this as shown below:

```
Nickname = str(NickName) + " Leaderboard"
```

To align the stats: I have decided to add 5 frames inside the leaderboard frame to iterate through them and move the frames if needed. I have also realised that if the 5 frame solution is implemented, I no longer need to have place fillers in the results, as they will always be in the same place.

I have also decided that for the User Leaderboard, the results list should only be ordered by highest score as from a user’s perspective that is the most important value.

Corrected and Edited Version

The code below includes the corrections outlined above:

```
#Leaderboard
def ViewLeader():
    NickName = clbGames.get()
    if not UserAdmin(Username):
        #call from function in callbacks
        Top5 = Top5Out(NickName)

        #create Leaderboard frame
        global frmLeaderboard
        frmLeaderboard = tk.Frame(root, bg = BackCol)
```

```

frmLeaderboard.place(x = 0, y = 0, width = 645, height = 207)
frmLeader.place(x = 0, y = 500, width = 645, height = 207)

#Title Label
print(NickName)
Nickname = str(NickName)+ " Leaderboard"
lblTitle = tk.Label(frmLeaderboard, text = (Nickname),
                    bg = BackCol,fg = TextCol,
                    anchor="w", font=('Bahnschrift', 16,
'bold'))
lblTitle.place(x = 25, y = 15, width = 250, height = 36)

#back Button
btnBack = tk.Button(frmLeaderboard, text = "Back",
                    bg = BtnCol ,fg = TextCol,
font=('Bahnschrift', 11) )
btnBack.place(x = 561, y = 23, width = 57, height = 36)

global frmLeaderboard1, frmLeaderboard2, frmLeaderboard3,
frmLeaderboard4, frmLeaderboard5
frmLeaderboard1 = tk.Frame(frmLeaderboard, bg = BackCol)
frmLeaderboard1.place(x = 0, y = 50, width = 560, height =
26)

frmLeaderboard2 = tk.Frame(frmLeaderboard, bg = BackCol)
frmLeaderboard2.place(x = 0, y = 79, width = 560, height =
26)

frmLeaderboard3 = tk.Frame(frmLeaderboard, bg = BackCol)
frmLeaderboard3.place(x = 0, y = 108, width = 560, height =
26)

frmLeaderboard4 = tk.Frame(frmLeaderboard, bg = BackCol)
frmLeaderboard4.place(x = 0, y = 137, width = 560, height =
26)

frmLeaderboard5 = tk.Frame(frmLeaderboard, bg = BackCol)
frmLeaderboard5.place(x = 0, y = 166, width = 560, height =
26)

#results
for count in range(0,5):
    print(Top5)
    print(Top5[count])
    User, Score, Time, Acc = Top5[count]
    #placeholders no longer needed due to the separated
    #variables

    First = ["User: '%s'" % User,"Score: '%s'" % Score,"Time
Taken: '%s'" % Time,"Accuracy: '%s'" % Acc]
    Nums = ["1. ", "2. ", "3. ", "4. ", "5. "]
    if count == 0: #to change the frame meaning I can
iterate through the count.
        frame = (frmLeaderboard1)
        print(frame)

```

```

elif count == 1:
    frame = (frmLeaderboard2)
elif count == 2:
    frame = (frmLeaderboard3)
elif count == 3:
    frame = (frmLeaderboard4)
elif count == 4:
    frame = (frmLeaderboard5)

lbl = tk.Label(frame, text = (Nums[count]),
               bg = BackCol,fg = TextCol,
               anchor="w", font=('Bahnschrift', 11, 'bold'))
lbl.place(x = 11, y = 0, width = 50, height = 26)

# Create the Label
label1 = tk.Label(frame,text=First[0],
                 bg=BackCol,fg=TextCol,anchor="w",font=('Bahnschrift', 11, 'bold'))

# Place the Label
label1.place(x=61, y=0, width=150, height=26)

# Create the Label
label2 = tk.Label(frame,text=First[1],
                 bg=BackCol,fg=TextCol,anchor="w",font=('Bahnschrift', 11, 'bold'))

# Place the Label
label2.place(x=200, y=0, width=150, height=26)

# Create the Label
label3 = tk.Label(frame,text=First[2],
                 bg=BackCol,fg=TextCol,anchor="w",font=('Bahnschrift', 11, 'bold'))

# Place the Label
label3.place(x=295, y=0, width=150, height=26)
#300 works

# Create the Label
label4 = tk.Label(frame,text=First[3],
                 bg=BackCol,fg=TextCol,anchor="w",font=('Bahnschrift', 11, 'bold'))

# Place the Label
label4.place(x=450, y=0, width=110, height=26)
count = count+1

#all lables in each column are in line.

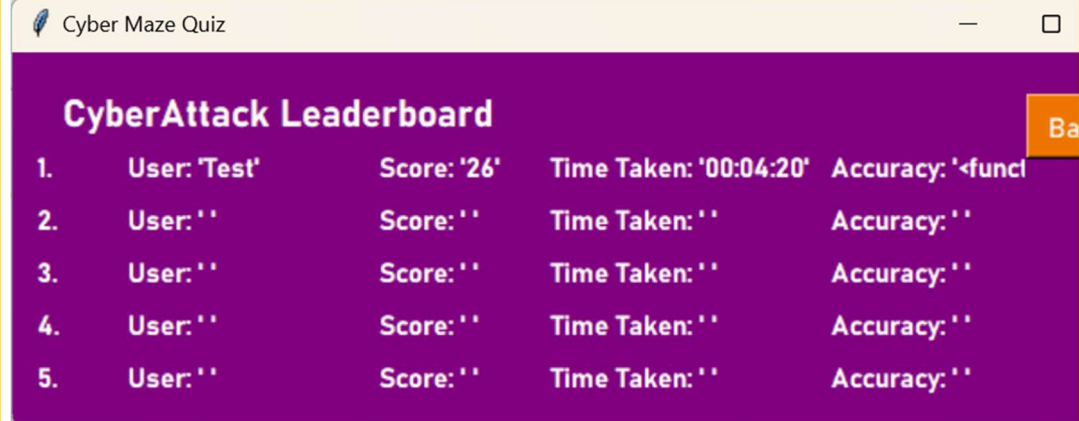
#Event Handlers
btnBack['command'] = Temp_Back

clbGames.set('View specific game Leaderboard...')
#reset clb

```

```
return
```

This is the edited version where I have used frames to separate the variables. I have also removed the place holding variables because the different stats are now separated anyway.



Looking at the GUI output above, I have noticed that although they are evenly spaced, the Accuracy doesn't seem to be correct. I will now go back to the PlayGame01 file to try to identify the error and fix this in the Play Game section.

I have fixed this error in the Play Game GUI development because I located the error in the Save_to_DB function that I had developed at the end of the Play Game GUI. I thought the placement of the error resolution explanation made more sense to be recorded in this project alongside the development of the function that contained the error.

37.7.11. Event Handlers

An event is when something occurs that is expected but it is not known when it will happen, for example the user clicking a button. There is only 1 button to consider in this frame:

btnBack

When clicked,
go to the Leaderboard menu GUI

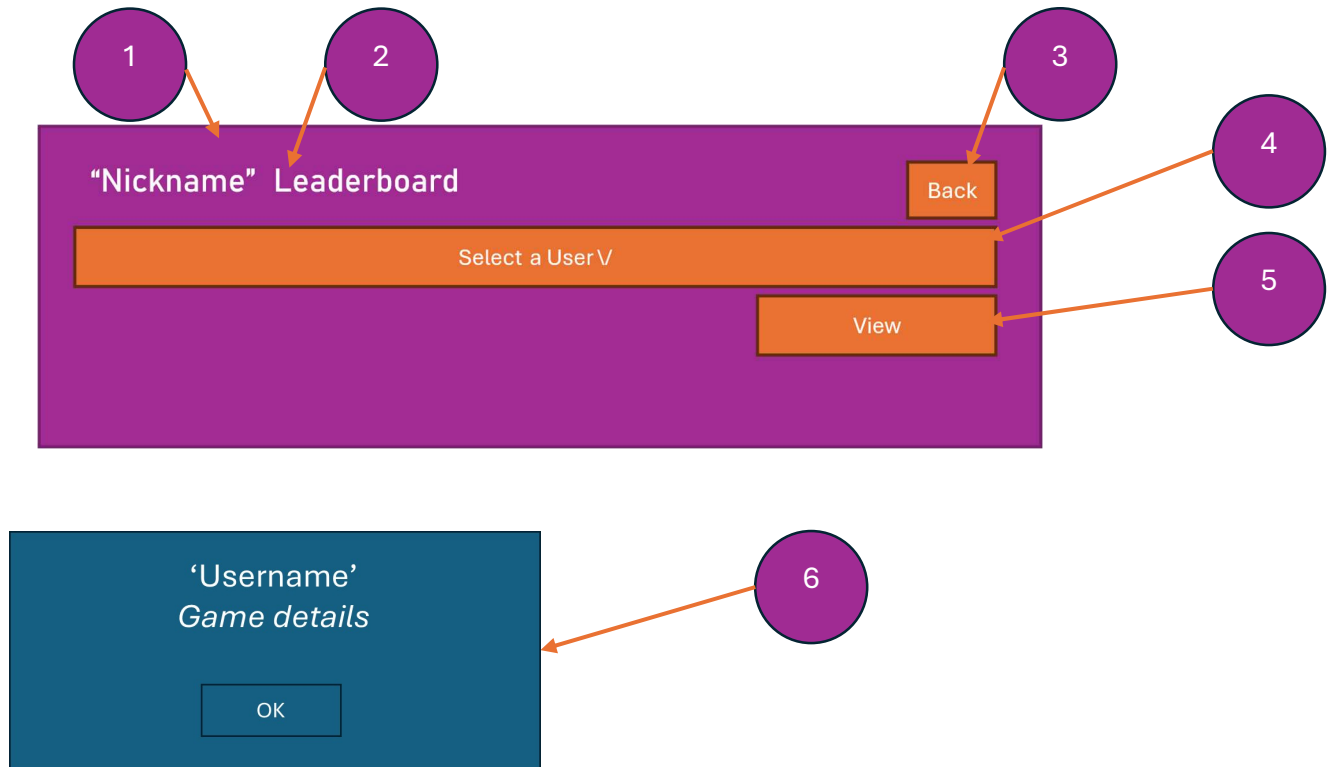
This button is simple to code as it is very similar to all the other back buttons

```
Temp_Back():  
frmLeaderboard.destroy()      #delete the leaderboard so that it can be upda  
frmLeader.place(x = 0, y = 0, width = 645, height = 207)      #move leaderbo  
width, height = 645, 207  
root.geometry("{}x{}".format(width,height)) #reset window size just in cas
```

This procedure will also be called by the Admin leaderboard as the process is the same.

37.7.12. Coding the Admin Leaderboard

I have created a simple design for the Admin Leaderboard GUI within the Leaderboards GUI and this is based on my previous GUIs (specifically the Leaderboards and Play Game GUIs) shown in the simple plan below:



ID	Name	Widget	Justification
1	frmLeaderboard	Frame(container)	All the other widgets are held within the frame. The widgets are then easy to move within the frame for best layout on the screen.
2	lblTitle	Label	Read only title for this GUI so the user knows where they are at in the system.
3	btnBack	Button	A Button is a good option for the Stakeholder to go back to the Options GUI.
4	clbUsers	ComboBox	A drop down menu for User selection meaning the Admin Stakeholder can only choose to view one users stats at a time.
5	btnView	Button	A Button is a good option for the stakeholder to view a specific users stats.
6	msgUserMazeDeets	MessageBox	A Message box specifically designed to allow the Admin user to view detailed stats about individual players

37.7.13. Admin Leaderboard GUI Development

```

else:

    def UserMazeDeets():
        UserGame = clbUsers.get()      #get the user selected
        for count in range(0, (len(UserDetails))):      #get all the values for the
user selected
            if UserGame in UserDetails[count][0]:
                User, Score, Time, Accuracy, Easy, Medium, Hard, Complete =
UserDetails[count]
                print(UserDetails[count] , ":)")
                print(User, NickName, User, Score, Time, Accuracy)
                mbox.showinfo("%s' details for '%s' Maze"%(User, NickName), "User:
%s'\nScore: '%i'\nTime: '%s'\nAccuracy: '%s'\nAmount Easy: %i\nAmount Medium:
%i\nAmount Hard: %i\n%s'"%(User, Score, Time, Accuracy, Easy, Medium, Hard,
Complete))      #message box to display results
                return

        UserDetails = LeaderA(NickName)      #get the details of all users for the game

        #frame for new gui
        #global frmLeaderboard
        frmLeaderboard = tk.Frame(root, bg = BackCol)
        frmLeaderboard.place(x = 0, y = 0, width = 645, height = 207)
        frmLeader.place(x = 0, y = 500, width = 645, height = 207)

        #Title Label
        Nickname = NickName[:len(NickName)]
        print(Nickname)
        Nickname = str(Nickname)+ " Leaderboard"
        lblTitle = tk.Label(frmLeaderboard, text = (Nickname),
                             bg = BackCol,fg = TextCol,
                             anchor="w", font=('Bahnschrift', 16, 'bold'))
        lblTitle.place(x = 25, y = 15, width = 400, height = 36)

        #back Button
        btnBack = tk.Button(frmLeaderboard, text = "Back",
                             bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )
        btnBack.place(x = 561, y = 23, width = 57, height = 36)

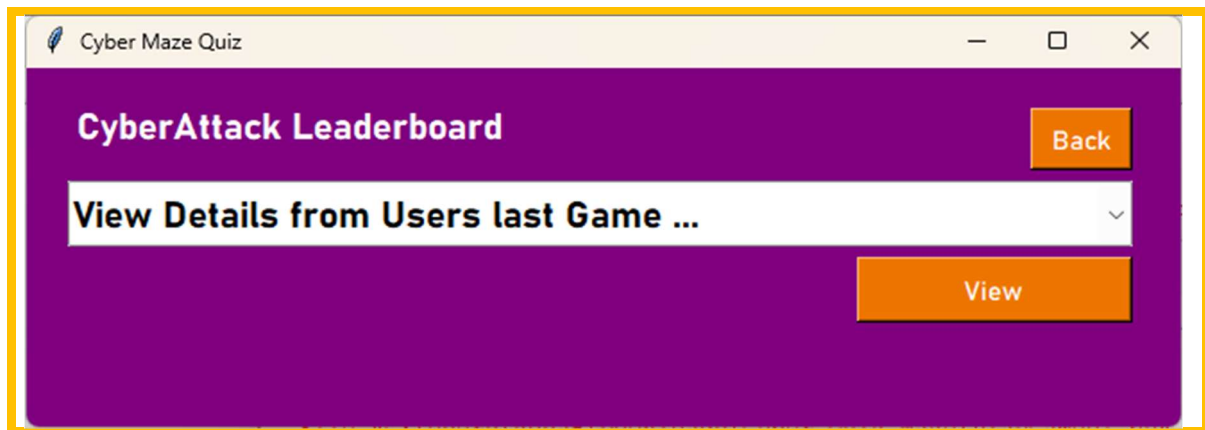
        UserD = []      #array to fill with usernames
        for count in range(0, (len(UserDetails))):
            UserD.append(UserDetails[count][0])
        Leader= sorted(UserD)      #sort alphabetically
        clbUsers = ttk.Combobox(frmLeaderboard, values = Leader,
                                text = "user details", state= "readonly",
                                font=('Bahnschrift', 16, 'bold'))
        clbUsers.set('View Details from Users last Game ...')
        clbUsers.place(x = 23, y = 65, width = 595, height = 38)

        #view button
        btnView = tk.Button(frmLeaderboard, text = "View",
                             bg = BtnCol ,fg = TextCol, font=('Bahnschrift', 11) )
        btnView.place(x = 464, y = 109, width = 154, height = 38)

        #Event Handlers
        btnBack['command'] = Temp_Back
        btnView['command'] = UserMazeDeets

        clbGames.set('View specific game Leaderboard...')      #reset clb
        return

```



37.7.14. **Evaluation**

This is my first attempt at creating the Admin Leaderboard GUI and it is based on the previously corrected Leaderboards GUI, so there aren't any corrections required.

37.7.15. Event Handlers

An event is when something occurs that is expected but it is not known when it will happen, for example the user clicking a button. There are 2 button to consider in this frame:

btnBack

When clicked,
go to the Leaderboard menu GUI

This button uses the same procedure as the User Leaderboard

```
Temp_Back():  
frmLeaderboard.destroy()      #delete the leaderboard so that it can be updated no  
frmLeader.place(x = 0, y = 0, width = 645, height = 207)      #move leaderboard to  
width, height = 645, 207  
root.geometry("{}x{}".format(width,height)) #reset window size just in case.
```

btnView

when clicked,
get the username from clbUsers
Search the User_Maze table for Maze(previously selected) and User selected
Get the stats for the User
Display the User stats in a Message box.

```
def UserMazeDeets():  
    UserGame = clbUsers.get()      #get the user selected  
    for count in range(0, (len(UserDetails))):      #get all the values for the user sele  
        if UserGame in UserDetails[count][0]:  
            User, Score, Time, Accuracy, Easy, Medium, Hard, Complete = UserDetails[count  
            print(UserDetails[count] , ":")  
            print(User, NickName, User, Score, Time, Accuracy)  
    mbox.showinfo("%s' details for '%s' Maze"%(User, NickName), '''User: '%s'\nScore: '%s'  
            %(User, Score, Time, Accuracy, Easy, Medium, Hard, Complete))  
    #message box to display results  
    return
```

This even handler gets the User details and displays then in an onscreen message box.

37.8. After completing my GUIs, I resized my root window to the correct size and centred it within my full screen.

To centre it I tried to guess the position needed for my screen, but then realised it would be different on different screens so I found a reference that I used to do this:

<https://www.geeksforgeeks.org/how-to-center-a-window-on-the-screen-in-tkinter/>⁵³

```
def center_window(window):
    #ref: https://www.geeksforgeeks.org/how-to-center-a-window-on-the-screen-in-tkinter/ 53
    window.update_idletasks()
    width = window.winfo_width()
    height = window.winfo_height()
    screen_width = window.winfo_screenwidth()
    screen_height = window.winfo_screenheight()
    x = (screen_width - width) // 2
    y = (screen_height - height) // 2
    window.geometry(f"{width}x{height}+{x}+{y}")

def Initialise():
    #Procedure to create window container for all GUIs
    global root          #standard name for window

    root = tk.Tk() #call library function

    root.title("Cyber Maze Quiz")    #Title for window

    width, height = 645, 207
    root.geometry("{}x{}+{}+{}".format(width,height, 0, 0)) # in the centre of the screen
    center_window(root)
    #root.eval('tk::PlaceWindow . center')

    return #end procedure
```

This was the final outcome for this solution, and it worked!

After Finalising the GUIs with my stakeholders, I went back and completed my test plans according to my finished GUIs.